

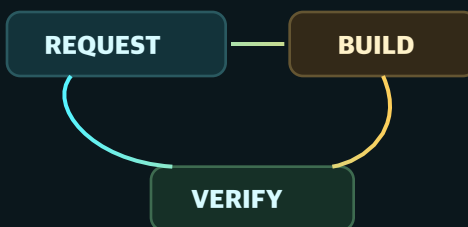
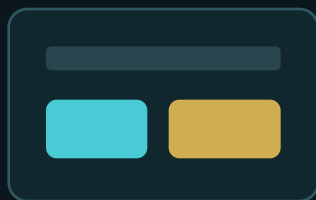
AGENTIC CODING IN THE AI ERA

Agentic Coding Tauri 2

Request · Build · Verify with AI agents

Production desktop apps without losing control

Rust · SvelteKit · Tauri · Real deployment



Ted Park

Request | Build | Verify | Repeat

Agentic Coding Tauri 2

Request · Build · Verify —Shipping six projects with AI agents

Copyright © 2026 Ted Park.

All rights reserved.

First edition, 2026.

Set in Apple SD Gothic Neo | Menlo.

Contents

Preface	1
Educational materials and scope	1
What this chapter covers	2
Learning goals	2
While writing this book	2
Moving the bar for “done”	3
What this book covers	4
What this book does not cover	4
Why the stack feels unfamiliar	4
The six projects at a glance	5
How to read this book	6
Overall structure	6
Prerequisites	7
The stack at a glance	8
Development environment	9
A note from the author	10
What to remember from this chapter	10
Chapter 1. Verification Matters More Than Speed —Why You Need to Stay in Charge	12
What This Chapter Covers	13
Learning Goals	14
1.1 Why AI Coding Without a Plan Doesn’t Last	14
1.2 Why Now, of All Moments	15
1.3 Before: I Typed Every Line Myself	15

1.4 Now: The AI Writes, I Decide	16
1.5 Why “You Just Need to Read It”Is Only Half Right	17
1.6 Role Shift: From Code Review to Writing Requirements	18
1.7 One Scene —the readbooks.ai PDF Parser Episode	19
1.8 Six Case Studies That Keep Coming Back	19
1.9 What You Walk Away With	20
Key Takeaways	21
Chapter 2. Request, Build, Verify —The Shape of the Loop	22
How This Chapter Flows	23
Learning Goals	23
Tools Used in This Chapter	23
2.1 One Loop, Only the Size Changes	24
2.2 Step 1 —Request	25
Requests that are too short	25
Requests that are too long	26
2.3 Step 2 —Build	26
2.4 Step 3 —Verify	27
2.5 Step 4 —Iterate	27
2.6 Why “One Perfect Prompt Builds the Whole App”Is a Fantasy	28
2.7 The Skeleton-First Strategy —Top-Down AI Development	28
Stage 1: Skeleton —the architecture prompt	29
Stage 2: Build —one module at a time	30
Stage 3: Bug fixes —compile, run, fix	30
Stage 4: Feature QA —verify each feature end to end	31
Stage 5: Polish —edge cases, performance, UX	31
Why the order matters	32
2.8 A Real Example —The Mandai DB Schema Loop	32
Cycle 1 —request	32
Cycle 1 —build	33
Cycle 1 —verify	33
2.9 The Basic Prompt Template: What / Why / Constraints / Format	34

2.10 Things to Watch Out For While Looping	35
What You Must Remember From This Chapter	36
Corrected second-cycle SQL	36
Chapter 3. AI Coding Tools —A Prompt-Only Development Loop	37
How this chapter flows	38
Learning goals	38
3.1 The scope of this chapter	38
3.2 The prompt-only loop —developing without typing code yourself	39
Step 1: Write a natural-language prompt	39
Step 2: Let the AI generate the code	39
Step 3: Review and verify behavior	40
Step 4: Accept, reject, or fix	40
Step 5: Verify	40
Step 6: Improve with evidence	41
3.3 Prompt patterns —a practical toolbox	41
Pattern 1: Skeleton prompt —architecture before code	41
Pattern 2: Precisely scoped fixes	42
Pattern 3: Spec-first prompt —signature before implementation	42
Pattern 4: Negative constraints —what not to do	43
Pattern 5: Pin context with file references	43
Pattern 6: Phase-aware prompts —name the current phase	44
Combining patterns	44
3.4 A full session —building the Pomodoro timer command	45
Round 1—initial request	45
Round 2—resolve the missing type	46
Round 3—scope issue plus cancellation	47
Session summary	47
3.5 Summary	48
Minimal verification commands	48
Chapter 4. Tools and Environment —Claude Code, Cursor, Codex, GitHub Copilot	49
How this chapter flows	50
Learning goals	50

4.1 Four tools, four approaches	51
4.2 When to use which	52
4.3 A minimal Tauri 2 + SvelteKit + Rust environment	53
Pinning versions	53
The required install steps	53
Setting up the AI tools	54
Initializing a SvelteKit project with Tauri	55
When installation fails—the three most common causes	56
4.4 The first 30 minutes—from scaffold to first commit	57
Quick checks (5 minutes)	57
Project setup (10 minutes)	57
Your first productive change (15 minutes)	58
Confirmation checklist	59
4.5 <code>.cursorrules</code> / <code>CLAUDE.md</code> —putting the rules files to work	60
<code>CLAUDE.md</code> (for Claude Code)—a real example	60
<code>.cursorrules</code> —same idea, different filename	61
What the rules files give you	61
4.6 Constraints to give the AI—name the stack and versions	61
4.7 Going further: MCP, Agents, slash commands	62
4.8 Five principles for choosing tools	62
What to take away from this chapter	63
Chapter 5. Tauri 2 Fundamentals—Core Patterns and Avoiding AI Mistakes	65
How this chapter flows	66
Learning goals	67
5.1 Tauri 2 Architecture Overview—Why a Clear Boundary Matters	67
Why a WebView instead of native UI	68
5.2 Commands—The <code>#[tauri::command]</code> Pattern	68
A rule for naming commands	68
The smallest example	68
Commands that can fail	69
Data with multiple fields	70
5.3 State— <code>tauri::State<T></code>	71

The basic pattern: <code>RwLock</code>	71
Choose locks by critical section, not by a blanket rule	72
Resources that are expensive to initialize	73
5.4 Events — <code>emit</code> and <code>listen</code>	73
Sending from backend to frontend	73
The streaming pattern	74
5.5 Plugins —The <code>tauri-plugin-*</code> Ecosystem	75
Plugins commonly used in AI apps	75
5.6 Permissions and Capabilities —The Tauri 2 Boundary	76
Check plugin permissions and custom commands separately	77
5.7 Frontend —SvelteKit 2 + Svelte 5 plus Tauri	77
Svelte 5 runes and Tauri events	78
Collecting command calls in an API wrapper	79
5.8 Project Structure —A Reference Layout	79
5.9 A Mental Model for AI-Assisted Tauri 2 Development	80
Prompting strategy	80
5.10 Verification Checklist	81
Chapter 6. Designing Mandai —How Big a Chunk Can You Hand to AI?	82
What This Chapter Covers	83
Learning Goals	84
6.1 What Mandai Was at the Start	84
6.2 The First Mistake —“Build the Whole Thing in One Shot”	85
Problem 1. UUIDs for cell IDs	85
Problem 2. Pomodoro sessions weren’t tied to tasks	86
Problem 3. The folder structure looked right but was hollow	86
6.3 Sizing —The Chunk the AI Can Handle at Once	86
Cell ID scheme —settled with two follow-ups	87
Pomodoro session —following the analytics scenario	88
AI chat —being offered the options	88
6.4 Turning the AI’s Counter-Questions Into a Design Tool	89
6.5 Three Layers: Screen, Data, Command	90
Layer 1: Define the screen	91

Layer 2: Data requirements	92
Layer 3: Enumerate the commands	92
6.6 The AI Chunk-Size Rules	93
6.7 When the Plan Changes Mid-Build	94
Pivot 1: cell ID collisions	94
Pivot 2: a real-time timer	95
When pivots happen too often	96
6.8 Summary	97
Chapter 7. The Database Schema —Your First Request ·Build ·Verify Loop	99
Why this chapter is told as “a story about loops”	100
How much of this should you type yourself?	100
How the chapter flows	100
Learning goals	101
7.1 Why I picked DuckDB	101
Look at the relationships first —seven tables	101
Why DuckDB	102
DuckDB vs SQLite —where AI tends to slip	102
7.2 The first loop —drafting the request	103
Open the connection once and share it	103
Sending the schema request	104
Reviewing the draft —what’s missing	105
7.3 The second loop —filling the gaps	105
The key tables, revised	106
Indexes —start from the queries you’ll run often	107
7.4 The third loop —the migration system	108
7.5 Testing the schema —using AI to check quality	109
7.6 A command that worked in one pass —db_query_json	110
7.7 DuckDB query patterns for Mandai	111
A note on soft delete and joins	111
7.8 The TypeScript type layer —where AI really helped	112
7.9 Summing up the loops in this chapter	113
Sorting database work by loop count	113

7.10 Registering the commands on the Rust side	114
7.11 What's next	114
What to remember from this chapter	114
Chapter 8. Integrating AI APIs —Writing AI Code with AI	116
The Flow of This Chapter	117
Learning Goals	118
What to Look at on Your First Pass	118
8.1 Write Down the Requirements First	119
8.2 Multi-Provider Abstraction —Rust Traits	119
8.3 Event Structure	121
Stream Cancellation	122
8.4 Where SSE Parsing Broke	123
8.5 Error Handling —When Each Provider Fails Differently	124
8.6 Rate Limiting and Cost Control	125
8.7 Response Normalization —A Uniform Shape for the Frontend	125
8.8 The Iteration Summary for This Chapter	125
8.9 How AI Writes Code	126
8.10 What You Must Remember from This Chapter	126
Chapter 9. Mandai Development Mistakes —Where the AI Got It Wrong	128
Flow of this chapter	129
Learning goals	130
9.1 Pomodoro timer —state living in two places	130
The real problem	130
A bad first attempt	131
Trap 1 —CHECK constraint violation	131
Trap 2 —Over-aggressive cleanup, and time-based recovery	132
Trap 3 —Background throttling and task binding	132
Revised prompt template	133
Key lessons	134
9.2 Mandal-Art cell ID —when a generic pattern overrides a domain rule	134
Where it went wrong	135

Why this happens	135
How I fixed it	135
9.3 Productivity score calculation —naming the edge cases makes the AI better	136
Edge cases that came out loop by loop	136
What I took away	137
9.4 Version confusion —mixing Tauri 1.x and 2.x APIs	138
A classic example: window management API	138
Pinning the version in the prompt	138
Common confusion points in v1 -> v2	139
9.5 Prevention: the prompt + review + test trio	140
Fake productivity	140
Tool 1 —Prompt checklist	141
Tool 2 —Review prompt	141
Tool 3 —Test protocol	143
How the three tools relate	143
9.6 Summary and next chapter	144
Next chapter	144
Chapter 10. readbooks.ai —Designing AI Output Quality with Prompt Engineering	146
Flow of This Chapter	147
Learning Goals	148
10.1 Background	148
A provider boundary for cloud and on-device translation	148
On-device, on-demand, and offline describe different things	149
Small models require workload-specific validation	150
10.2 The First Prompt —A Draft That Missed Everything	152
Why the AI Behaves This Way	152
A Real Failure —Five Errors in One Block	152
First Fix —Pin Down the Preservation Boundary	153
Second Fix —Separate “Symbol”from “Content”	154
10.3 Branching by Language —The Illusion of “One Prompt for 36 Languages”	154
Solution —A Base Prompt Plus Per-Language Profiles	155
What a Real Profile Looks Like	156

Building Language Profiles: Using AI as a Research Assistant	156
10.4 Designing Prompts With AI —The Meta Loop	157
What a Meta Prompt Is	157
Reusing the Same AI as a Reviewer	157
Dual Agents —Separating Translator from Reviewer	158
Meta Prompts That Failed	158
Version Control for Prompts	158
10.5 Translation Quality Standards —How to Communicate Them to the AI	159
Where Each Standard Came From	159
Domain-Specific Additions	160
Verification Pipeline —Automatic Checks Plus a 10% Human Review	160
10.6 Wrap-Up of This Chapter’s Loops	161
Three Patterns That Kept Repeating	162
10.7 Next Chapter	162
Summary	163
Chapter 11. Translation Verification —How Do You Trust AI Output?	164
The flow of this chapter	165
Learning goals	165
11.1 The Problem —“Plausible but Wrong”Translations	166
11.2 Back-translation —Trying It by Hand Once	166
11.3 Building the Verification Command —The Simplest Skeleton	167
11.4 Limits of a Single Score —BLEU and Cosine Similarity	168
11.5 Splitting into Three Axes	169
Aligning the Result Type With the Frontend	169
Adding Hallucination Detection	170
Summarizing with a Three-Level Badge	170
11.6 Refining It in Operation	171
11.7 User Feedback Loop	171
The feature	171
Updating Prompts From Real Data	172
Sorting Feedback Into Three Simple Tiers	172
11.8 Visualizing “Verifying AI Output With Another AI”	173

- 11.9 Tips for Applying This to Your Reader App 173
 - Tip 1. No single score. Split into axes. 173
 - Tip 2. Use different models for generation and verification. 174
 - Tip 3. Use three threshold levels. 174
 - Tip 4. Feed user feedback into the prompt. 174
- 11.10 Chapter Summary 174
- 11.11 Looking Back at Part II and Part III 175
- 11.12 Next Chapter Preview 175

Preface

This book explains design and verification through prose, worked examples, and state transitions. Some code excerpts show the structure of the author's applications; they do not constitute the complete source of those products. Independent runnable examples state their prerequisites and expected results.

Educational materials and scope

The optional educational archive is named `tauri2-app-examples.zip`. Its scope is a minimal Tauri app under `tauri/ch05-local-notes/` and an optional Flet exercise under `flet/ear-loop/`. Installation commands, dependency lockfiles, and validation notes belong in the corresponding README files. Access to the author's complete commercial applications or production trading repository is not included.

Check the seller's delivery page for any available attachments or download links. An archive named in this text is not evidence that it has already been attached to your purchase. If a link is unavailable, contact the author through the public profile linked from the book listing. Distinguish explanatory code in the PDF from separately distributed runnable exercises.

What this chapter covers

Before the main text begins, this preface lays out a few things.

- How the author uses AI to build and verify apps
- What this book covers and what it does not
- What you should already know to read it
- The six projects you will build and the technology behind them
- The order in which you should read the book

You do not need to understand everything from the start. Come back here whenever you get lost in the main text.

Learning goals

After reading this preface, you only need to take away three things.

- This book is less about how to use AI tools and more about **how to verify your way to a finished app**.
- You do not need to memorize Tauri 2, SvelteKit, Rust, DuckDB, and the rest from the very beginning.
- You know what order to read the chapters in, and where to come back when you get stuck.

While writing this book

In 2026, AI tools changed quickly. But the reason my way of working changed was not simply that the tools got newer. The scope of what I took on changed.

In January, I was the same kind of hands-on developer I had been for years. I opened a ticket, opened my editor, and typed. Two days for a single feature; three if it was complex. That was the pace I was used to.

This book uses six projects: four desktop apps and two terminal tools. The examples include a productivity app and a TUI—an app that runs inside the terminal screen—for monitoring a trading system.

My skill did not suddenly grow. What changed was the division of labor between what I do and what the AI does.

Moving the bar for “done”

Early in the year, like everyone else, I used AI as a faster autocomplete. I described what I wanted, the AI produced code, and I picked it up and kept going.

The AI was fast. I was slow. Code the AI wrote in three minutes took me an hour and a half to review. I told myself that careful review was a good habit, but the more I repeated it the stranger the bottleneck looked. **I was holding up the whole job by reading code I did not need to read.**

That is when I asked myself a question.

Is my job to confirm that the code is correct, or to confirm that the feature works correctly?

You check the first by reading code. You check the second by running the app. The second one is what actually matters, but I was spending my time on the first.

After that, I changed how I work.

- Write a precise request for the AI.
- Run the feature myself and see what happens.
- Walk through a checklist of common failure cases.
- If it passes, commit. If it fails, paste the failure straight into the next request and loop.

This is not me saying I stopped looking at code. **It is me moving the bar for “done” from how the code looks to how the app actually behaves.**

The result was six projects and this book. I did not get more time. I just stopped spending time where it did not matter.

The AI writes the code; you make the judgment. Unplanned AI coding gets you started fast, but if you do not know what went wrong, you cannot explain it to the AI and you cannot fix it. Chapter 1 goes into detail on where you get stuck and on how the human keeps control of verification.

What this book covers

A single cycle runs through the whole book.

Request -> Build -> Verify

You repeat these three steps over and over while building an app. Chapter 2 explains exactly what each step means and where mistakes tend to creep in. The six projects in this book show how that cycle plays out in real Tauri 2 apps.

What this book does not cover

Knowing what is out of scope ahead of time makes it less confusing when you need to pull in other resources.

- **A Rust syntax tutorial.** This book does not teach Rust from scratch.
- **A Svelte 5 syntax tutorial.** The code uses Svelte 5's new syntax, but we do not teach it from the beginning.
- **A retelling of the official Tauri 2 docs.** This book is not a replacement for or repeat of the official documentation.
- **General prompt engineering.** Non-coding uses like writing and image generation are out of scope.
- **The internals of AI models.** We do not compare how Claude and GPT process text under the hood.

Why the stack feels unfamiliar

It is normal for the technologies in this book to feel unfamiliar at first.

Tauri 2 is a framework that uses Rust and JavaScript/TypeScript together. The two languages run in different environments, so it takes a while to see how they connect. You do not need to grasp the whole structure up front.

This book does not walk through each technology as a separate tutorial. Instead, you run into a real problem while building an actual project, ask the AI, and then run the AI's suggested fix

to see if it works. What you really need to learn is not the syntax of every tool. **It is how to break problems apart and verify them even when the tool is new to you.**

For example, while building the Mandai app—a productivity app for managing tasks and goals on a Mandalart grid—you will learn how to wire in DuckDB, a small embedded database for analytics. In the trading dashboard, you will pull live data over a WebSocket, a connection that lets a server and client keep sending data back and forth without reopening the link. When you get stuck, you hand it to the AI. You do not need to understand everything before you start.

The AI handles a lot of the implementation. If one approach fails, you can tell it to try another. That does not mean you hand every decision to the AI. The AI generates options and tries implementations quickly. **The final call is still yours.**

The AI writes Rust for you, but to spot problems in the generated code, you need to be able to read basic Rust syntax. You do not need to be able to write it from scratch.

The six projects at a glance

The six projects in this book are not examples invented for the sake of explanation. They are apps the author actually uses.

- **Mandai**—A productivity app with a built-in AI coach
- **readbooks.ai**—A PDF translation app built in 2025 with Next.js, rebuilt in this book with Tauri 2
- **Trading Dashboard**—A real-time trading dashboard
- **RL & Backtest Lab**—Reinforcement learning training control
- **TUI Trading Monitor**—A trading monitor that runs in the terminal
- **TUI Log Viewer**—A terminal-based log viewer

Section 1.8 walks through what each project actually does and what role it plays in the book.

How to read this book

Part I (Chapters 1-5) should be read in order. It covers the premise of the whole book, the definition of the cycle, AI coding tool workflows, how to set up your tools, and Tauri 2 basics.

Before you move on to Part II, skim **section 1.8 (the six case studies)** first. The chapters that follow will make much more sense once you know the context behind what you are asking the AI to do.

Parts II-VI (Chapters 6-17) are each built around a single app and stand on their own. You can pick the parts you care about.

- Interested in the Mandai app? -> Part II
- Interested in AI translation and output quality? -> Part III
- Interested in the trading dashboard? -> Part IV
- Interested in how to feed domain knowledge to the AI? -> Part V
- Interested in terminal apps? -> Part VI

Each part opens with an introduction to the cycle pattern you will learn in that app. The apps in Part II and after do connect to each other in places, but the structure does not force you to read them all in order. The first chapter of each part re-establishes the context, so you can start with the app you care about most.

Part VII (Chapter 18) is the chapter to consult when you apply this method to your own app. Read it once on your first pass, and again when you actually start your own project.

Overall structure

The book is organized into seven parts. Each part has you build a real app while picking up another piece of working with AI. **Bolded names** are the projects you will build yourself.

Part	Chapters	Theme
I	1 -5	Why use AI, the feedback cycle, the Copilot workflow, development tools, Tauri 2 basics
II	6 -9	Mandai —design, database, AI API, recovering from AI mistakes
III	10 -11	readbooks.ai —prompting and verifying results
IV	12 -14	Trading Dashboard —scaling up, real-time processing, deployment
V	15	RL & Backtest Lab —injecting expert domain knowledge into the AI
VI	16 -17	TUI Apps —shipping terminal UI apps fast
VII	18	Applying this to your own app

Prerequisites

To get through this book, you need the following background.

- **You can read Rust and TypeScript code.** You do not need to be able to write it from scratch. It is enough to look at the AI's output and roughly tell what the code does.
- **You are comfortable with Git and the terminal.** If creating branches and splitting commits is second nature, you can drop the workflow in this book (write code -> AI reviews -> revise, repeat) straight into your routine.
- **You have used at least one AI coding tool.** Some real work with Claude Code, Cursor, GitHub Copilot, or ChatGPT will do. If you have not, spend a week on a toy project with an AI and then come back.

On the other hand, you do **not** need these: Tauri 1 experience, SvelteKit experience, or Rust experience. Each chapter explains what it needs along the way.

The stack at a glance

Here are the core technologies used in the book. You do not need to learn each one in depth. **For now, just get a sense of what each one does and move on.**

Technology	Role	Used in
Tauri 2	A framework for building desktop apps with web tech (HTML/CSS/JS). The installers are small.	Chapters 2-14
SvelteKit 2 + Svelte 5	The frontend framework for the screens in this book.	Chapters 2-14
Rust	The language for the Tauri backend. You will rarely write it yourself; being able to read it is enough.	Chapters 2-17
DuckDB	An analytical database that runs from a single file, with no server needed.	Chapters 7-13
ratatui	A Rust library for drawing UI in the terminal.	Chapters 16-17
Python	Reinforcement learning training scripts. Runs as a separate process alongside the Rust app.	Chapter 15 only
shadcn-svelte	A set of Tailwind-based Svelte UI components.	Chapters 6-14

Technology	Role	Used in
Claude Code	An AI coding tool that runs in the terminal. The most-used tool in this book.	All chapters
Cursor	A code editor with built-in AI autocomplete. Good for quick edits to a single file.	All chapters

How each one works in detail, and how some compare to alternatives (for example, Tauri versus Electron, or Svelte 5’s rune syntax), is re-explained the first time it shows up in the main text.

Development environment

All the code and screenshots in this book were produced in the environment below. You do not need to pin yourself to the exact same versions, but this gives you a reference point when you get stuck.

Item	Version
OS	macOS 15.7.5 (Sequoia)
Rust	1.95.0
Node.js	24.15.0 LTS
pnpm	10.33.2
Tauri	2.11.0
SvelteKit	2.59.0 + Svelte 5.55.0
DuckDB	1.5.2
Python	3.14.4 (reinforcement learning chapter only)

Item	Version
Claude Code	Latest as of May 2026
Cursor	3.0
VSCode	Latest (1.99+)

Note: pnpm 9.x reached end of support on April 30, 2026. You need to use pnpm 10. Node.js 22 LTS also works, but this book is written against Node.js 24 LTS.

Most of the examples work on Windows and Linux as well. Where a platform needs different steps, the chapter calls it out.

A note from the author

This book follows, in order, my own experience of building with AI.

I have tweaked a prompt, retried, and then in the end just edited the code by hand. You should not do that. I have also taken plausible-looking but wrong code from an AI and committed it without even running it. When that happens, errors pile up like a snowball and you can end up throwing away the whole codebase. The way you learn in this book follows the actual thinking I went through.

This is the record of building six apps while holding down a day job and raising a 150-day-old baby. Prompts written in a cafe at lunch, code checked on the train home, commits pushed after the late-night feeding.

Some scenes have been lightly rearranged for flow. The lessons and the code are real.

I did not write this book because I had time. I wrote it because I wanted to say that if you do not have time but you have an AI agent, that can be enough.

What to remember from this chapter

The preface ran long, but before you move on to the main text, only four things really need to stick.

- **The core is the Request -> Build -> Verify cycle.** You do not read the code end to end; you run the app and see what it actually does.
- **Six real apps** are how you practice the cycle. The author uses every one of them daily.
- **Part I in order**, but from Part II onward you can pick whichever app interests you.
- **You do not need to learn every technology up front.** When you get stuck, ask the AI and run the result yourself. That loop is the learning.

Now Chapter 1 takes up the question: “If the AI is writing the code, why is it the human who gets stuck?”

Ted Park 2026

Chapter 1. Verification Matters More Than Speed —Why You Need to Stay in Charge

Part I —Foundations (Chapters 1-5) This part covers how to develop alongside AI. Why use AI (Chapter 1), what loop you move through (Chapter 2), which tools you use and how (Chapters 3-4), and the core patterns of Tauri 2 (Chapter 5). These five chapters are the foundation of the whole book. On a first read, don't try to memorize every tool name. Instead, focus on internalizing the loop: “ask, build, verify, ask again.”

At first I thought asking the AI to “build it” was the whole job. I’d paste the code as-is, the screen would render, and I’d move on to the next feature. For a few days, it really did feel fast. Hours of typing disappeared. Drafts that used to take hours came out in minutes. But the wrong decisions piled up just as fast as the speed.

For example, even a one-line request like the one below produced code that actually ran.

```
Build a trading dashboard. Connect to a Python FastAPI backend over  
WebSocket,  
with live charts. Use Tauri 2.
```

The screen looked convincing. The trouble came right after. A connection dropped and never reconnected. The state got tangled and I couldn’t tell which part was off. Asking the AI to “fix it” produced more plausible-looking code, but the same kind of problem kept showing up in different shapes.

The lesson was simple. Code I hadn’t verified myself wasn’t code I controlled. Whether the AI wrote it or a human wrote it didn’t matter. If I had merged it without checking what behavior I expected or under what conditions it would fail, it was a black box inside my system. Fast generation didn’t mean I was in charge.

So my question changed. Instead of “build this,” I started with “how do I confirm this behavior is actually right?” I ran the output, narrowed down where it failed, and folded that evidence back into the next request. I repeated this loop. This chapter is about that shift. There’s something more important than the fact that AI writes code quickly: how a developer keeps the steering wheel inside that speed.

What This Chapter Covers

This chapter walks through, in order, why verification matters more than speed in AI coding, and how the developer’s role changes.

- Why AI coding without a plan doesn’t last
- Why now, of all moments
- Before: I typed every line myself
- Now: the AI writes, I decide
- Why “you just need to read it” is only half right
- Role shift: from code review to writing requirements

- One scene —the Korean PDF parser episode
- Six recurring case studies in this book
- What you walk away with

Learning Goals

- Understand why AI is not just fancy autocomplete.
 - Know where AI coding without a plan tends to break down.
 - Tell apart where “you just need to read AI code” is true and where it isn’t.
 - Understand why the developer’s job shifted from typing to verifying and judging.
 - Get a preview of the six case studies covered in this book.
-

1.1 Why AI Coding Without a Plan Doesn’t Last

When you code with AI but no plan, it looks fine at first. The screen renders, buttons click, API responses show up in the console. That makes it easy to put off verification. The thought “let me wire up the next feature and clean this up later” follows naturally.

The problem is the code you stack up that way. The moment something breaks, you can’t tell where the fault is. Say a WebSocket reconnection (a WebSocket is a two-way real-time channel) fails. Is it a frontend state issue? A Rust-side issue? A Python backend issue? It’s hard to tell at a glance. When my dashboard froze, I spent a long time suspecting the wrong layer. The real cause was simple: I had been committing AI output without checking what conditions made it fail.

After that I changed how I worked. The AI may write the code, but deciding whether to move to the next step is the developer’s job. That decision has to rest on verification results, not gut feel. The **Request, Build, Verify loop** (request -> generate -> verify) that recurs throughout this book is the minimum scaffolding for keeping that control. Hand the speed to the AI, but keep the judgment for yourself.

1.2 Why Now, of All Moments

AI existed before. Autocomplete existed before. So why 2026 in particular? Three things lined up at the same time.

First, the models changed. Starting in late 2024, AI models like Claude began to read and understand multiple files at once. By 2026, they can read files, edit code, and run tests within a single flow of conversation. They aren't simple autocomplete anymore—they've become tools you can hand a small unit of development to.

Second, Tauri 2 reached general availability (GA). The installers are small and memory use is low. That's a real difference compared to Electron-based apps, which routinely run well past 100 MB. The technical details come in Chapter 5, but what matters here is Tauri's structure. The screen is built with web technology, while anything that needs file access or system permissions is handled by a Rust command. With roles split this cleanly, you can tell the AI exactly which side to touch.

Third, Svelte 5 runes arrived. Runes are Svelte 5's new reactivity syntax—symbols like `$state` and `$derived` that begin with `$`. Chapter 4 covers the details. For now, remember just one thing: state management that took several composed hooks in React becomes a shorter, simpler pattern in Svelte 5. The simpler the pattern, the fewer ways the AI can slip. That's why frontend code quality goes up.

1.3 Before: I Typed Every Line Myself

In the days before AI, development was mostly done with fingers. You'd read a JIRA ticket, open your editor, and your browser had a row of Stack Overflow tabs. You had to type out even the obvious code, and spending half a day on a single component was normal. The top productivity tool of that era was autocomplete—type a few letters and the IDE filled in the rest. That was it.

Ninety percent of the code passed through my hands. Function names, variable names, repeated conditionals, even import statements—I typed them all. There was an upside. When you write by hand, you sit with the structure longer. But the downside was clear too. Typing ate so much time that you often shoved implementation forward before the design was

thought through. “Let’s make it work first and refactor later” wasn’t a plan—it was a confession that there was no time. And in most cases, “later” never came.

1.4 Now: The AI Writes, I Decide

What changed in 2026 isn’t that AI started doing my job. The AI took over the typing, and I started deciding the direction. My hands got less busy, but I still have to keep choosing what to ask for and where to stop and verify. If anything, judgment happens more often.

Here’s how it looks for one task. One morning I wrote an idea in my notebook: “Mandal-Art goal tree + Pomodoro + AI coaching -> store locally in DuckDB -> wrap the whole thing in Tauri 2.” After lunch I opened the Claude Code terminal and dropped in a first prompt.

This prompt is a short statement of “what to build” and “what constraints to follow.”

Build a Mandal-Art 9x9 grid component with Tauri 2 + SvelteKit 2 + Svelte 5. Clicking a cell opens an edit modal. When the center cell changes, have the AI suggest candidates for the surrounding empty cells. Use only Svelte 5 runes. Split the code by file.

Within three minutes, a first draft of the component, store, and Tauri command for AI suggestions appeared. When I ran it, two things bothered me. First, I had asked for the new syntax, but old syntax was mixed in. Second, the AI call ran directly from the frontend—that would expose the API key.

I put the problems I’d observed straight into the next prompt.

Two fixes.

1. Use only `$state`, `$derived`, `$effect`. No reactive variables declared with ``let``.
2. Move the AI call into a Tauri command. The API key must not be exposed to the frontend.

Three more minutes of edits and both issues were resolved. I committed. Work that would have taken six hours before was done in under twenty minutes.

But those twenty minutes don’t mean “the AI handled everything on its own.” I described what I wanted, ran the result, found the wrong spots, and asked again. The typing volume shrank, but sorting out what I wanted and judging the result was still my job.

If I sort what the AI got right on the first try, what it got wrong, and what I made it fix, it looks like this:

Category	Examples
Got it right	9×9 grid layout, Tailwind class combinations, keyboard navigation skeleton
Got it wrong	Old Svelte syntax mixed in, AI call from the frontend, sloppy type handling
Made it fix	Unified to new runes syntax, moved AI call into a Tauri command, cleaned up error types

This three-row table repeats for every feature. My day, in truth, is filling out this table.

The point isn’t that the AI is perfect from the start. The point is that I can run short ask, short fix, short verify loops very fast. That’s why this is faster than before. The ability to nail it on the first try matters less than the ability to narrow down what went wrong and immediately try again.

1.5 Why “You Just Need to Read It”Is Only Half Right

Looking back, **I never needed to read the code line by line to find bugs.** But at the start, I didn’t know that.

This claim is half right and half wrong.

Start with the right half. You no longer need to memorize all of Rust. The AI writes the code. You read it. As long as you can ask “is this handling correct?”that’s enough.

But stopping there gets you in trouble. **The bar for “I can read it”is much higher now.** You have to be able to tell whether the AI’s code has a problem. You don’t need to know Rust today. By Chapter 5, you’ll start to recognize signals like “this kind of code is dangerous.”That level is enough.

The two abilities you need are: recognizing warning signs, and judging whether “this result is what I asked for.”

A note: This was what I thought going into the Mandai project. By the end, reading code was not the bottleneck. The compiler and behavior testing caught the bugs. Rust’s checking tools (`cargo clippy`) warn you about dangerous patterns, and behavior tests

reveal hidden errors. Knowing which patterns are dangerous and why still matters. But reading code line by line to find bugs turned out to be unnecessary. Chapter 2 shows the loop where behavior testing replaces code review.

So how do you verify? Not by reading code, but by **putting the execution result back into the next request as evidence**. If a Korean PDF comes out garbled, paste a few of the garbled lines straight into the next prompt. Section 1.7 shows how this works in practice.

1.6 Role Shift: From Code Review to Writing Requirements

In 2020, the thing I did most each day was **write code**. In 2026, the thing I do most is **write prompts**.

Writing prompts means writing requirements. It's not like jotting a quick note after a meeting. A prompt for the AI has to be far more **precise**. People interpret "you figure it out" themselves to some extent. The AI doesn't. It looks like it understood, then much later it hands back something completely different.

Before, you reviewed after building. Now, you have to remove the ambiguity before building. If you don't separate the requirements from your own judgments first, the AI will fill the blanks its own way.

So every day I sort these four items in my head.

- **What:** a function, a component, or the whole app.
- **Why:** where it fits inside the larger feature.
- **Constraints:** language, libraries, performance targets, file layout.
- **Output shape:** code blocks, file lists, only the changed parts, depth of explanation.

Once it becomes routine, you can pack the four into one sentence. Chapter 2 treats this as part of the loop.

The old **code review** was reading someone else's code after the fact. The new **writing requirements** is sorting out your head before the AI builds anything. Instead of fixing a wrong build, you reduce the chance of a wrong build in the first place. The order is flipped—that's the biggest change, and it's why working with AI raises productivity.

1.7 One Scene —the readbooks.ai PDF Parser Episode

While rebuilding readbooks.ai in Tauri 2, I needed a feature that pulled text out of PDFs. The first prompt was short.

A Rust-based Tauri command that extracts text from a PDF.

Claude produced about twenty lines of code using the pdf-extract library. I ran it. English PDFs worked. Korean PDFs came back as **boxes of garbled characters**.

In the old days, I'd have typed "rust pdf korean encoding" into a search box. Instead, I took what I had observed and asked again.

The Korean PDF comes out garbled. Here's the first page of the sample input:
<30 lines of garbled text>

List three candidate causes and a quick fix for each,
then pick the most realistic one and give me the corrected command.

Claude listed three candidate causes, then picked the most likely one and suggested switching to a different library or calling out to an external tool. The diagnosis was accurate.

What matters here isn't that Claude was wrong on the first try. What matters is that **I took what I observed and put it back into the next prompt** to sharpen the request. In the old days, I'd have searched for "how do I fix this." Now, **I show the AI exactly what I saw and build the next hypothesis on top of it.**

That's the pattern this book repeats. A short first request. Pasting your results back in as evidence. Listing candidate hypotheses. Picking the realistic one. It's not about expecting answers from the AI one-way. This flow becomes the foundation of a repeatable way to work.

1.8 Six Case Studies That Keep Coming Back

The six projects introduced in the preface —Mandai, readbooks.ai, Trading Dashboard, RL & Backtest Lab, TUI Trading Monitor, TUI Log Viewer —keep reappearing throughout the book. Here I'll touch only on the parts that connect to this chapter's theme.

Each of the six started from a different problem. Mandai froze the UI every time a task was canceled. `readbooks.ai`, as we just saw, garbled the characters in Korean PDFs. The trading dashboard never reconnected after a real-time connection dropped. RL Lab left processes lingering in memory after they had stopped.

They share one thing. The AI was only useful when the problem was visible. If you don't know what's wrong, you can't even describe it to the AI.

The case studies in this book aren't a highlights reel. They're material for showing how to narrow down failures, how to turn what you find into a request, and where a human still has to step in and decide.

1.9 What You Walk Away With

You won't finish this book having memorized every Tauri 2 API. You won't become a Rust expert either. That's not the goal.

What you take away is **the ability to build your own apps in this same way**. Five things, specifically.

1. **The muscle for writing precise requests.** Turning the "I want something like this" in your head into a prompt the AI can actually execute.
2. **An eye for judging AI output.** Telling correct code from incorrect code, and catching code that's correct but dangerous. Knowing in advance the traps that beginners hit in Tauri 2.
3. **A rhythm that keeps the development loop unbroken.** Accepting that the first try won't be perfect, and closing each cycle quickly.
4. **Six real reference projects.** What the AI wrote and what I fixed are both still in there. Patterns you can reuse almost as-is when you build something similar.
5. **Judgment about which interface to pick.** GUI when user experience matters, TUI (terminal-based UI) when speed matters. Which way you go shapes every prompt that follows.

None of these five are tied to a specific stack. Even without Tauri, even without Rust, what stays with you is the way of requesting, judging, and verifying.

Key Takeaways

- The heart of AI coding isn't "build it fast"—it's "verify it yourself." Code you haven't verified isn't yours.
- AI coding without a plan is fast at first, but the moment something breaks, you don't know where to start looking.
- In 2026, the model, Tauri 2, and Svelte 5 all stabilized at the same time, making it the right moment to weave AI into real development.
- The developer's work has moved from typing to **ask -> check the result -> ask again**.
- "You just need to read AI code" is only half right. You memorize less syntax, but you now need to feed execution results back into the next request as evidence.
- This book follows six real cases to show how that way of working actually plays out.

Chapter 2 walks through how to run this loop in practice, step by step—the conditions that start each step, the conditions that end it, and the bad habits that will shake a project if you repeat them.

Chapter 2. Request, Build, Verify —The Shape of the Loop

Every chapter in this book runs on the same loop. The code changes from chapter to chapter. Sometimes you'll design a DuckDB schema, sometimes you'll fix a WebSocket reconnection, sometimes you'll run a Python process inside a Tauri app. But the way you work doesn't change. You make a request, you receive what the AI produced, you run it to confirm what it does, and you request again —but only for the parts that failed.

The core difficulty from Chapter 1 was that **the AI produces plausible-looking results faster than a human can verify them**. If you let that gap stay open, unverified code piles up underneath the next feature, and by the third feature you can no longer tell which layer broke things. That's why this chapter defines the loop tightly on purpose. Each step has an entry condition and a stopping point. The project chapters that follow are records of this loop applied to real

cases. You need to understand the loop first; then you'll see why I phrase certain requests the way I do and why I stop to verify at specific points.

How This Chapter Flows

- Learning goals
- Tools used in this chapter
- One loop, only the size changes
- Step 1 —Request
- Step 2 —Build
- Step 3 —Verify
- Step 4 —Iterate
- Why “one perfect prompt builds the whole app” is a fantasy
- The skeleton-first strategy (top-down AI development)
- A real example —the Mandai DB schema loop
- The basic prompt template
- Things to watch out for while looping

Learning Goals

- Get comfortable with the core loop used throughout this book: **request -> build -> verify -> iterate**.
- See the principle behind each step and the antipatterns that tend to trip people up.
- Understand why “one perfect prompt builds the whole app” is a fantasy.
- Learn the **skeleton-first** strategy: settle the overall shape first, then drop into modules, fix bugs, verify features, and polish at the end.
- Walk through Mandai's first DB schema as a full end-to-end loop.
- Practice the **What / Why / Constraints / Format** prompt template yourself.

Tools Used in This Chapter

Tool / Library	Version	Role in this chapter
Tauri	2.11.0	The target framework for the Rust module and command structure in the skeleton-first example
SvelteKit	2.59.0	The frontend framework whose routes and components are shaped by the skeleton prompt
DuckDB	1.5.2	The database used in the Mandai schema loop walkthrough
tokio	-	Mentioned by name in the async command example. Actual code comes in later chapters
thiserror	-	Mentioned by name in the skeleton-first example. Actual code comes in later chapters

2.1 One Loop, Only the Size Changes

The way you work with the AI across this entire book boils down to one thing.

```
Request --> Build --> Verify
                |
                ↓
        (if needed) Request again --> ...
```

That one line is the working model for the whole book. You use it when you write a single function, and you use it when you design the architecture of an entire app. What changes is how long one trip around the loop takes. Small loops take a few minutes; big ones can take more than a day. Either way, **you do not move to the next step without verifying first.**

Loops come in roughly three sizes.

- **Small loops (5-15 minutes):** “Add a keyboard shortcut to this Svelte component.”

- **Medium loops (30 minutes to 2 hours):** “Move the PDF upload pipeline in readbooks.ai over to a Tauri command.”(readbooks.ai is a PDF translation app I built. A **Tauri command** is the bridge that runs a Rust function when the frontend calls `invoke("name")`.)
- **Large loops (a day to a week):** “Lock down the shared layout and chart components for the 19 screens of the Trading Dashboard.”

The bigger the loop, the longer each step takes, and the more small loops nest inside it. Knowing which size loop you are currently running keeps you from losing track of the larger flow.

2.2 Step 1 —Request

A request is how you hand the AI context, constraints, and a goal. You write it in natural language, but what it contains is closer to a technical spec. A good request doesn’t only say “what to build.”It also says why it’s needed, where it plugs in, which approaches to avoid, and what shape the output should take. When any of this is missing, the AI fills the blanks itself. If those guesses don’t match your project, the next two or three loops will be spent patching in the missing context.

A good request usually meets three conditions.

1. **It includes context.** Where in the codebase does this feature live? What does the surrounding code do?
2. **It states constraints.** “Use Svelte 5 runes only.”“Do not stuff everything into one file.”
3. **It pins down the output shape.** A code block? Three files? “Code only, no explanation”is one way to say it.

Let’s see what it looks like when these three are missing, through two antipatterns.

Requests that are too short

This is the mistake I see most often.

Write a DB command in Tauri

Given this, the AI fills in everything that isn’t there. The kind of DB, the name of the command, how errors are handled—all chosen by the AI. The problem is that even when those

choices are wrong, the code looks fine on the surface. Your project uses DuckDB, but you get a SQLite example back. The next few rounds aren't spent building the feature; they're spent retroactively patching in context you should have given in the first place.

Requests that are too long

The opposite —packing too much into one request —is just as risky.

Build a Tauri 2 app from scratch. Use SvelteKit (Svelte 5), embed DuckDB, hook up OpenAI, build a Mandal-Art UI, add Pomodoro, and include a GTD workflow. Keep every file short and add comments. Use Tailwind too...

When you ask for ten things at once, you usually get ten shallow things. Each piece looks plausible alone but the pieces don't fit together. The DB schema doesn't match the UI state. Tauri command names don't line up with the frontend that calls them. If you'd split this into five smaller requests, you'd have been able to verify at each step.

2.3 Step 2 —Build

The build is the AI's output, but it's still closer to a "this should work" claim than a confirmed fact. It might be code, or it might be a design diagram or a file tree. Getting output doesn't end the loop —not until the verification step confirms the claim.

How you check it depends on what the output is. For design artifacts like architecture sketches, DB schemas, and file trees, you **read** them first. Adding or removing a module is far cheaper than rolling back 200 lines of code. For implementation code, though, don't sit and read line by line —**run it first**. Code that looks natural on the page often turns out to be wrong at runtime.

The habit of "read everything before running" recreates the code-review bottleneck I described in the preface. The AI writes code faster than I can read it line by line. The compiler and a short hands-on test catch the same bugs I'd hunt for manually, faster and more consistently. You still need to be able to understand code. You just don't need to read every line before doing anything else. **Run first, then read around the points where it failed.** The next section goes into this "confirm the claim by running it" step in detail.

2.4 Step 3 —Verify

Verifying means running, testing, and reviewing. The usual order is:

1. **Compile and type-check.** For Rust, `cargo check` (a fast pass that compiles but doesn't produce an executable). For TypeScript, `tsc --noEmit` (type-checks without writing output). If either fails, paste the compiler output verbatim back into a new request.
2. **Did it honor the constraints?** Run the behavior you specified to confirm.
3. **Are existing features still healthy?** Run nearby features and check for regressions.
4. **Edge cases.** Try three things by hand: invalid input, boundary values, and error paths. For important features, have the AI write unit tests.

For a small loop, one manual run is enough. For a large loop, write the tests first and have the AI implement to match. The mechanics of these tools (`cargo check`, `tsc --noEmit`, test setup) are covered later alongside environment setup.

Skimming the code and saying “looks right” isn't verification. Even if the code looks clean, a missing capability file or an event name that doesn't match the frontend will only show up when you actually run it. If you skip verification and stack two builds back to back, the first error gets buried under the second change. Add a third feature and finding which change broke what will take longer than building the features did. **Verification isn't a one-time step at the very end. It's what closes the loop between every build.**

2.5 Step 4 —Iterate

When you iterate, only re-request the parts that were wrong. Don't ask for the whole thing to be rewritten.

Fix only these two things in the code above.

1. <specific point 1>
2. <specific point 2>

Leave the rest unchanged.

“Leave the rest unchanged” is the key line. Without it, the AI will edit parts it didn't need to touch, and you have to re-verify code you just confirmed.

If three iterations with the same request all fail, stop iterating and rewrite the request from scratch. The cause is usually one of three things.

- You left out a constraint.
- You didn't give enough context.
- You asked for too much at once.

Pasting more error messages on top is slower than rewriting the requirements and constraints.

2.6 Why “One Perfect Prompt Builds the Whole App” Is a Fantasy

The reason the loop has to be split into four steps is that doing it in one shot doesn't actually work. X (formerly Twitter) is full of posts claiming you can just write a great prompt and an app pops out. In practice it doesn't. Three reasons:

1. **Context has a ceiling.** No matter how long the prompt is, you can't fit an entire app into one.
2. **Ambiguity compounds.** When you stack ten natural-language requirements, the first ones get reinterpreted by the time the AI reaches the last ones.
3. **It's hard to verify.** A one-shot output has to be checked end to end, and when something is wrong, locating the cause is painful.

These three issues mostly disappear once you break the request into smaller pieces. That's why I rely on **short requests, many times**. I cap each request at no more than three requirements. The “skeleton-first strategy” in the next section is that same principle applied at the scale of a whole app.

2.7 The Skeleton-First Strategy —Top-Down AI Development

This is the most important idea in the book. Everything else is detail wrapped around this.

Tauri app, TUI tool, backend service —whatever you're building with AI, settle the skeleton before the details. The full strategy has five stages.

```
Stage 1: Skeleton    — architecture, file layout, interfaces, data flow
Stage 2: Build       — implement each module, one at a time
Stage 3: Bug fixes   — compile, run, fix all the errors
Stage 4: Feature QA  — verify each feature end to end
Stage 5: Polish      — edge cases, performance, UX
```

It's a funnel from large decisions down to small ones.

The AI doesn't know this order. Ask for a feature and it tries to produce structure, logic, edge cases, and error handling all in one go. The output looks finished but it's fragile. Change one thing and three other places break.

When you, the developer, restrict the scope of each stage, the quality goes up. While the skeleton is being designed, don't let the AI write code. While a module is being implemented, don't let it touch other modules.

Stage 1: Skeleton —the architecture prompt

The first prompt is never “Build X.” It's “Design the skeleton of X.”

Here's the actual skeleton prompt I used for the Mandai project. Mandai is a Tauri app that combines GTD-style task management (a productivity method built around capturing, processing, and organizing tasks) with DuckDB (a lightweight database that embeds inside the app).

The point of this prompt is to **get structure back, with no code**.

[Context]

I'm building a Tauri 2 + SvelteKit (Svelte 5 runes) app called Mandai. It combines Mandal-Art (a 9x9 goal grid), GTD tasks, a pomodoro timer, and multi-AI chat (OpenAI, Claude, Gemini).

[Request]

Design the project skeleton.

- Every Rust module in src-tauri/, with each module's role (one line each)
- Every Svelte route in src/routes/, and the screen it renders
- Every Tauri command name, with its input and return type
- DuckDB tables (name + columns, no SQL yet)

[Constraints]

- Do not write implementation code
- Do not write SQL
- Output structure only

The most important lines are the constraints. Drop “Do not write implementation code” and the AI throws in code without being asked. Then you’re back in code-review mode and you lose the whole benefit of the skeleton stage.

What you get back is a **blueprint**, not a product. Blueprints are easy to read and easy to change. If the AI added a module you don’t want, you delete a single line —not 200 lines of code.

Changes at the skeleton level are cheap. Changes at the code level are expensive. Lock down the skeleton before any code gets written.

Stage 2: Build —one module at a time

A minimum pomodoro request can be: “Implement `start_pomodoro(minutes)` and `stop_pomodoro(session_id)`. Reject zero minutes and duplicate starts; return a session ID, start time, and expected end time. Stopping the same ID twice must end it only once. Use `tokio::time` and `thiserror`. First tabulate expected behavior for normal completion, duplicate starts, invalid IDs, and app restart.” Clear inputs, failures, and recovery matter more than function count.

Once that module is built and verified, move on to the next. Repeat the same pattern. **One module, deeply, verified, then next.**

Stage 3: Bug fixes —compile, run, fix

After a few modules are in, compile the whole project. It’s normal for the build to fail at this point. Because the AI wrote each module in isolation, import paths are off, types don’t line up across module boundaries, and function signatures have drifted.

Bug-fix prompts are different from build prompts. **They’re precise and short.** Two things matter:

- **Paste the compiler output verbatim.** Don’t paraphrase it.
- **Pin it to “only these errors.”** Leave that out and the AI starts modifying healthy functions.

The project fails to compile with the following errors:

```
error[E0308]: mismatched types
  --> src/pomodoro.rs:42:12
     expected `i64`, found `String`
```

Fix only these errors. Don't touch anything else.

Stage 4: Feature QA —verify each feature end to end

Compiling and running isn't the finish line. Use each feature like a real user would, one at a time.

The QA loop:

1. Pick one feature (e.g., “start a pomodoro session linked to a task”).
2. Run it by hand. Does it work?
3. If not, identify the exact failure point and send a prompt that names the symptom.
4. Fix -> re-verify -> next feature.

The point of a bug-report prompt is to write **expected behavior and actual behavior** as clear single lines.

[Bug report]

After starting a pomodoro session for task_id = 5 and stopping it, the session is saved to DuckDB but the task's accumulated_minutes is not updated.

Expected: task.pomodoro_minutes increases by the session length

Actual: task.pomodoro_minutes stays at 0

Show me the stop_pomodoro function. Where should the task update happen?

The last line is the key. Instead of telling the AI to fix the code immediately, you make it **point to the wrong location first**. If it points to the wrong place, you stop right there and ask again.

Stage 5: Polish —edge cases, performance, UX

Polish comes last. The edge cases the happy path didn't cover, the performance issues that only appear on real data, the UX rough spots you notice after an hour of using the app—all of that goes here. The prompt usually numbers the edge cases (for the pomodoro timer, for example: (1) recovering interrupted sessions on app restart, (2) preventing double starts, (3) rejecting 0-minute sessions). Numbering makes it easy to spot anything the response skipped.

Why the order matters

The five stages **depend on the ones before them**. If an earlier stage isn't stable, the later ones wobble.

- Polish before you've fixed bugs and you're polishing code that's about to change.
- Hunt bugs before all modules are in and you may chase one that vanishes once the next module is added.
- Build details before the skeleton is settled and you'll rewrite modules whenever the architecture shifts.

The biggest time-sink in AI-assisted development is mixing the stages up. Following the order doesn't make each stage faster —it cuts down on **rework itself**.

Watch out: the urge to “just fix this one thing real quick” always shows up. Don't fix it on the spot. Write it on a list and handle it in the right stage. The cost of context-switching in AI-assisted development is bigger than it looks.

2.8 A Real Example —The Mandai DB Schema Loop

That's enough theory. Let's look at an actual cycle. This is how I locked down Mandai's first DB schema. Two cycles, about 15 minutes.

I picked the DB schema as the first example for a simple reason: **the code is small enough to fit on one screen, so a full pass around the loop is visible at a glance**. When the request, the output, the check, and the fix all live on the same page, it's the easiest way to learn the loop.

Cycle 1 —request

I fit the whole schema into a short request. The key bits are the **9×9 cell structure**, the **required columns**, and **valid DuckDB SQL**.

Draft the initial DuckDB schema for Mandai.

Mandal-Art is a 9x9 grid. Each cell in the central 3x3 becomes the center of a child 3x3. Each cell has title, description, a done flag,

accumulated pomodoro minutes, and created/updated timestamps.
Use only DuckDB-valid SQL. Include indexes. One file.

Cycle 1 —build

Here’s the core of Claude’s draft. What this code does is **define cells, the parent-child relationship, and the basic metadata columns**.

```
CREATE TABLE cells (
  id INTEGER PRIMARY KEY,
  parent_id INTEGER REFERENCES cells(id),
  position INTEGER NOT NULL,
  title TEXT, description TEXT,
  done BOOLEAN DEFAULT false,
  pomodoro_minutes INTEGER DEFAULT 0,
  created_at TIMESTAMP DEFAULT now(),
  updated_at TIMESTAMP DEFAULT now()
);
CREATE INDEX idx_cells_parent ON cells(parent_id);
```

There are three lines worth pausing on. `parent_id` (this is what builds the grid’s hierarchy), `position` (must be 0-8 to be valid), and `updated_at` (the default is `now()`, set once —it does not auto-update on writes).

Cycle 1 —verify

Reading through, I spotted three problems.

1. There’s no range constraint on `position`. Only 0-8 are valid, but the schema doesn’t enforce that.
2. DuckDB’s autoincrement-ID syntax isn’t the same as SQLite’s. The draft mixed in SQLite syntax.
3. `updated_at` won’t update automatically. The app has to set it explicitly on every write.

After confirming the three, I didn’t ask for a rewrite. I left the correct parts alone and pointed only at the wrong ones.

Fix three things in that schema.

1. Switch `id` to DuckDB's syntax.
2. Add a `0..=8` range check on `position`. The center is 4.
3. `updated_at` gets set by the app on every write — note that in a comment. Leave the `parent_id` structure alone.

The revised version came back with all three changes applied. That was enough to accept as the v0.1 draft schema. Specific DuckDB syntax like `SEQUENCE` and `CHECK` is covered separately in Chapter 7.

Two cycles, about 15 minutes. Schema v0.1 done. I never expected a perfect result on the first try. I got a draft fast, then pointed at the problems specifically. “Leave the `parent_id` structure alone” also made it clear what shouldn’t move. Domain knowledge isn’t only useful when you’re writing code by yourself from scratch. It’s just as useful for narrowing in on the spots where the AI’s assumptions were off.

The next section organizes this pattern into a template you can use for any task.

2.9 The Basic Prompt Template: What / Why / Constraints / Format

Here’s the four-part template I use every day. Filling in the blanks lets you check whether anything’s missing.

[What]

One line: what you're building.

[Why]

Why it matters. The features it includes and the user flow.

[Constraints]

- Language / version
- Things to avoid
- Performance / security requirements
- Interface with existing code

[Format]

- Output shape (code only / with explanation / diff / file tree)
- File and function naming conventions

Filled in for a Tauri command, it looks like this. What this request produces is **a single Rust command that takes a PDF upload and saves it locally.**

[What]

A Tauri command that takes a PDF upload and saves it locally.

[Why]

Stage 1 of the readbooks.ai translation pipeline. Stages 2-4 (parsing, translation, storage) plug in here. The user drag-and-drops a PDF.

[Constraints]

- Rust, tauri 2.x, async
- Save to app_data_dir/uploads/<uuid>.pdf
- Reject files larger than 50MB
- Define a custom error type with thiserror

[Format]

- A single Rust code block, under 30 lines
- Keep the explanation above the code to 2-3 lines

If you write requests in this shape, the quality stays the same even when they get shorter. Once it's a habit, you can compress the four sections into a single paragraph. When starting a fresh session with the AI, though, it's better to keep the four blocks separate.

2.10 Things to Watch Out For While Looping

- **Double-check the AI's "best practice" claims.** APIs often change after the model's training cutoff. Cross-reference current official docs.
- **If the same fix instruction produces the same result twice, the instruction itself is the problem.** Rewrite it.
- **Save the important prompts.** Keep a prompts.md per app and you'll reuse them two months later for similar work.

What You Must Remember From This Chapter

- Every task in this book follows the loop: **request** -> **build** -> **verify** -> **iterate**. Only the size of the loop changes based on the scale of the work.
- Use the **skeleton-first strategy**: settle the architecture before writing code, build one module at a time, fix bugs once the whole thing is in place, verify each feature end to end, and only then polish. Jumping between stages causes rework to pile up fast.
- There is no “one perfect prompt.” **Short requests, many times** wins on both quality and speed.
- Building without verifying turns iterations into accumulating technical debt.
- Put **What / Why / Constraints / Format** in every request. Once you’re used to it, you can compress it into a paragraph.

Starting next chapter, you’ll apply this loop to actual code. You’ll see the same four steps cycle around in the same shape, every time.

Corrected second-cycle SQL

Use a DuckDB sequence for IDs, constrain position to 0-8, and have the application set `updated_at` on every write. Keep `parent_id` unchanged, then test invalid positions and parent references.

```
CREATE SEQUENCE cell_id_seq;
CREATE TABLE cells (
  id INTEGER PRIMARY KEY DEFAULT nextval('cell_id_seq'),
  parent_id INTEGER REFERENCES cells(id),
  position INTEGER NOT NULL CHECK (position BETWEEN 0 AND 8),
  title TEXT,
  description TEXT,
  done BOOLEAN DEFAULT false,
  pomodoro_minutes INTEGER DEFAULT 0,
  created_at TIMESTAMP DEFAULT now(),
  -- The application supplies updated_at on every INSERT/UPDATE.
  updated_at TIMESTAMP NOT NULL
);
CREATE INDEX idx_cells_parent ON cells(parent_id);
```

Chapter 3. AI Coding Tools —A Prompt-Only Development Loop

Chapter 2 explained what the development loop is. This chapter is about how to actually run that loop in front of a real tool. Understanding the idea in your head and writing a request that AI can immediately follow are two different things.

When I first started using GitHub Copilot, I treated it as an autocomplete tool. I would type a few characters, Copilot would suggest the rest of the line, and I would accept it with the Tab key. But that was not enough for a project like a Tauri app, where the frontend, the Rust commands (the Rust functions that handle system work like file access), and the database all have to line up.

Now, before I press Tab, I settle three things first.

1. Tell the AI what to build.

2. Decide the order in which I want the results.
3. Decide what to check before accepting anything.

These three habits form the core of the prompt-only development loop.

How this chapter flows

- 3.1 The scope of this chapter
- 3.2 The prompt-only loop —six steps
- 3.3 Six prompt patterns I use every day
- 3.4 A full session —building a Pomodoro timer
- 3.5 Summary

Learning goals

- Build a **prompt-only development loop** that keeps quality high even when you do not type the code yourself.
- Learn six prompt patterns that work reliably.
- Walk through building a Tauri command end to end, using only natural-language prompts.
- Learn a way of thinking that survives even when the tool you use changes.

3.1 The scope of this chapter

Tool categories and how to choose between them are covered in Chapter 4. This chapter does not compare features. It focuses on the **loop pattern** that still works after the tool changes.

The examples use Copilot, but the product name is not what matters. These three habits are.

1. Break the request into small pieces.
2. Provide context and constraints together.
3. Verify the result by running it.

These habits work the same way in Cursor or Claude Code.

3.2 The prompt-only loop —developing without typing code yourself

You can build a whole feature without typing a single line of implementation code yourself. That does not mean the developer steps away. Instead of writing code, your job shifts to writing clear requirements, reviewing what the AI produces, and confirming that the result actually runs.

The loop looks like this.

Request -> Review -> Verify -> Improve -> Repeat

Step 1: Write a natural-language prompt

This step is not “having a chat with the AI.” It is **writing a specification**. The AI will not leave ambiguous parts empty. It fills them with its own guesses. If those guesses do not match the conventions of your project, small conflicts will keep popping up in later steps.

That is why the first prompt always includes four things. Here is a template you can copy directly.

Context: [where this fits in the codebase]

Goal: [the specific behavior you want]

Constraints: [what not to do, or which specific patterns to follow]

Format: [how to structure the output — single function, multiple files, etc.]

Leave any of the four blank and the AI’s guess will fill that space.

Step 2: Let the AI generate the code

Wait until the generation is fully finished. If you interrupt halfway, the structure the AI was building gets cut off, and the next response often mixes in different assumptions.

If the direction is obviously wrong, start over with a new prompt. For small mistakes, look at the finished output first and then send a fix request.

Step 3: Review and verify behavior

This is the most important step. Before you accept the code, **actually run it and confirm the behavior**. Code from AI usually has correct syntax and natural names, so problems are easy to miss when you only read it.

Check four things.

- Does it compile without errors?
- Are the constraints respected?
- Do the edge cases you defined pass?
- Do nearby features still work?

Step 4: Accept, reject, or fix

Once verification is done, pick one of three actions.

- **Accept:** apply it as is.
- **Reject:** the approach is wrong. Start over with a different prompt.
- **Fix:** the direction is right but the details are off. Ask for changes to specific parts.

The reason you do not tweak the generated code by hand is that it makes the starting state unclear for the AI's next round. Keeping the loop clean with explicit fix requests works better.

Step 5: Verify

Same as Chapter 2.

1. **Compile check:** `cargo check`, `npm run type-check`
2. **Manual run:** actually exercise the feature.
3. **Edge case test:** try to break it.

If verification fails, capture the error output and move to step 6. Rather than guessing a fix by hand, it is better to hand the error message back to the AI as evidence.

Step 6: Improve with evidence

The prompt that fixes a failure has a fixed structure.

The code above produced this error: [paste the complete error]

Please fix only these specific issues:

1. [specific problem 1]
2. [specific problem 2]

Leave everything else exactly as it is.

The last line —“**Leave everything else exactly as it is**”—is the key. Without it, the AI may rewrite code that was already working. If that happens, you have to redo all the verification you just finished.

3.3 Prompt patterns —a practical toolbox

If the six-step loop is the big picture, the six patterns in this section are the tools you pull out at each step. **A prompt is a technical document.** Write it as precisely as you would write a function signature or an API spec.

Pattern 1: Skeleton prompt —architecture before code

Before asking for implementation, always ask for the structural design first. If the structure is wrong at this stage, every later round drifts on top of that bad foundation.

Here is the request I send when sketching the Pomodoro module. The crucial line is “**structure only, no code blocks.**”

[Context]

Mandai productivity app. Tauri 2 + SvelteKit 2 + Svelte 5. DuckDB backend.

Current modules: mandal-art CRUD, GTD tasks.

Target to add: Pomodoro timer feature.

[Request]

Design the skeleton of the Pomodoro module:

1. Rust side: all function names, input types, return types (no implementation)
2. Svelte side: all components, props, the Tauri commands they call
3. DuckDB: table names, columns, types (no CREATE TABLE SQL yet)
4. Event flow: which Tauri events are emitted, and who listens

[Output]

Structure only. No code blocks. No implementation.

What you get back is a **blueprint**. Erasing a line of text takes a second; tearing down a function that has already been written takes an hour.

Pattern 2: Precisely scoped fixes

When something does not work, do not just write “it doesn’t work.” Paste the **exact error** and **limit the scope** of what should change.

```
error[E0308]: mismatched types
  --> src/pomodoro.rs:42:12
   |
42 |     return sessions;
   |           ^^^^^^^^ expected `Result<Vec<Session>, String>`,
   |                   found `Vec<Session>`
```

Fix only this return-type mismatch.

Wrap it in `Ok()`.

Do not change the query logic or the `Session` struct.

This prompt follows three rules.

1. **Paste the compiler or runtime output verbatim** —the AI reads line numbers.
2. **State the fix scope** —“fix only this” prevents other parts from being touched.
3. **State what to preserve** —“do not change” blocks unnecessary rewriting.

Pattern 3: Spec-first prompt —signature before implementation

Lock down the function signature (name, inputs, return type) first, then ask for the implementation. If you let the AI choose both the signature and the body at once, you end up with parameter types you did not want. This matters most when the frontend and backend need to agree on the same interface.

Write an implementation for exactly this function signature:

```
pub async fn stop_pomodoro(
```

```

    db: State<'_, DbPool>,
    session_id: i64,
) -> Result<PomodoroResult, AppError>

```

Behavior:

- Query the active session from pomodoro_sessions
- Compute elapsed minutes
- Update the session status to 'completed' and set ended_at
- Update accumulated_minutes on the linked task
- Return a PomodoroResult containing the session summary

Do not change the function signature. Do not add parameters.

The last two lines are the key. If the signature drifts, the calling code drifts with it.

Pattern 4: Negative constraints —what not to do

The AI defaults to the patterns in its training data. Even when you ask for Tauri 2, you may get Tauri 1 patterns; even when you ask for Svelte 5, Svelte 4 syntax may sneak in. Spelling out what you do not want is more reliable.

Generate a DuckDB query command for Mandai.

Must:

- Use `tauri::State` for the database connection
- Return `Result<T, AppError>` with a custom error type
- Use parameterized queries

Must not:

- Use `unwrap()` or `expect()` anywhere
- Use `rusqlite` (use the `duckdb` crate)
- Return raw database errors to the frontend
- Use any Tauri 1 APIs

The **must / must not** format is a checklist the AI can follow mechanically. Writing a must-not list noticeably cuts down the number of fix rounds.

Pattern 5: Pin context with file references

Pointing at the file that holds your conventions is much more accurate than describing those conventions in words. The AI reads the file and infers the pattern.

Look at `#file:src-tauri/src/db.rs` (the database module)

and `#file:src/lib/stores/timer.ts` (the frontend timer store),

then write a new Tauri command `get_pomodoro_stats`:

- Query DuckDB for total sessions, total minutes, and average duration
- Return a struct that matches the TypeScript interface in `timer.ts`
- Use the same error-handling pattern as the other commands in `db.rs`

Each tool uses a different syntax for file references.

Tool	Syntax
GitHub Copilot Chat	<code>#file:path</code> (no space)
Cursor	<code>@filename</code>
Claude Code	just mentioning the filename includes it automatically

Pattern 6: Phase-aware prompts —name the current phase

When you split work into multiple phases, writing the current phase into the prompt keeps the AI from straying out of scope.

[Phase: Bug fix — do not add features]

After compiling the whole project, three errors remain:

1. `src/ai/mod.rs:15` — missing import for `StreamExt`
2. `src/mandal/grid.rs:88` — `Cell` struct needs `#[derive(Clone)]`
3. `src/main.rs:42` — wrong module path in the `register_commands!` macro

Fix all three. Show only the changed lines with three lines of context.
Do not refactor, do not optimize, do not add features.

The `[Phase: Bug fix]` tag at the top is the anchor. Without it, the AI will redesign whole modules while it happens to be fixing one error.

Combining patterns

In practice, most prompts use two or three patterns at once. You might combine phase-aware, file reference, spec-first, and negative constraints in a single prompt. Here is what each pattern prevents.

- Phase-aware -> prevents work from spilling outside the intended scope.
- File reference -> prevents breaking project conventions.
- Spec-first -> prevents the API interface from drifting.
- Negative constraints -> prevents falling into traps you already know about.

Combine purpose, input/output types, constraints, failure behavior, and observable completion checks into one request. Use the verification commands at the end of this chapter.

3.4 A full session —building the Pomodoro timer command

Now let's see how these patterns come together in a single piece of work. We will build a Tauri command for the Pomodoro feature in the Mandai app without typing the implementation ourselves —only by talking to the AI. The whole thing takes three rounds.

Round 1 —initial request

The first request spells out, in one shot, what to build, which arguments it takes, what it should do, and how it should handle errors.

Create a new Tauri command called `start_pomodoro_timer`.

Context: For Mandai, a productivity app with a DuckDB backend.

Args: `goal_id` (String), `duration_minutes` (u32)

Behavior:

- Insert a new session into the `pomodoro_sessions` table
- Start a background timer
- Emit a `'timer_tick'` event every second with the remaining time
- Emit `'timer_done'` on completion
- Return the `session_id`

Error handling: `return Result<String, String>, no unwrap()`

Let's only look at the structure of the AI's response. The excerpt below is just enough to see "what this command does" at a glance —it inserts a session into the DB, emits an event every second on a background task, and sends a completion event when done. Keep the session ID, cancellation state, and task handle together; reject duplicate starts and make stop idempotent.

```
#[tauri::command]
pub async fn start_pomodoro_timer(
    app: tauri::AppHandle,
    goal_id: String,
    duration_minutes: u32,
) -> Result<String, String> {
    let session_id = uuid::Uuid::new_v4().to_string();

    // 1. Insert the session into the DB (INSERT INTO pomodoro_sessions
    ...)
    // 2. Spawn the background timer
    tokio::spawn(async move {
        for remaining in (0..=duration_minutes * 60).rev() {
            app.emit("timer_tick", /* TimerEvent */).ok();
            tokio::time::sleep(Duration::from_secs(1)).await;
        }
        app.emit("timer_done", session_id.clone()).ok();
    });

    Ok(session_id)
}
```

Review —what is right and what is missing: the basic shape is correct. But three things are missing.

1. The `TimerEvent` struct is not defined -> compilation fails.
2. There is no way to cancel the timer mid-run.
3. The `tokio::spawn` block runs on a separate task, so `goal_id` cannot be used inside it directly (ownership problem).

Of those three, the first blocks compilation; the other two are behavior problems. Getting compilation to pass comes first.

Round 2 —resolve the missing type

Start with the `TimerEvent` type that is blocking compilation.

Please define the `TimerEvent` struct used above.

It should include: `session_id` (`String`), `remaining_seconds` (`u32`), `goal_id` (`String`).

Make it `Serialize + Clone` so it can travel as a Tauri event.

The struct that comes back has those three fields and derives `Serialize` (for JSON conversion) and `Clone` (so the same event can be sent to multiple places).

What to check: at this point `cargo check` should pass.

Round 3 —scope issue plus cancellation

Solve the remaining two problems together. By saying “do not touch the parts that already work,” we preserve Round 1’s output.

Please fix only these two things:

1. Clone `goal_id` before spawn and include it in `TimerEvent`.
2. Add a `cancel_pomodoro_timer` command that takes a `session_id` and updates the DB row to `status = 'cancelled'`.

In the timer loop, check the DB status every 10 seconds and exit if it is no longer 'active'.

Do not change anything else.

The added `cancel_pomodoro_timer` is essentially a one-line UPDATE query—it sets the session’s status to cancelled. The actual stopping of the timer happens because the loop running on the background task sees this status change and exits on its own.

What to check: after calling `cancel_pomodoro_timer`, the `timer_tick` events should stop within ten seconds.

Session summary

- **Total rounds:** 3
- **Time taken:** about 20 minutes
- **Lines of implementation code I typed myself:** 0

The AI produced the basic pattern correctly on the first try. The only things that needed extra guidance were sharing data between threads and adding cancellation. This is what a prompt-only loop usually looks like—get the big structure in one go, then clean up the details with narrow follow-up rounds.

3.5 Summary

Two things to remember from this chapter.

- **The six-step loop** —request, generate, review, accept/reject/fix, verify, improve with evidence.
- **The six patterns** —skeleton prompt, precisely scoped fix, spec-first, negative constraints, file reference, phase-aware.

Of the six patterns, the two I reach for most often are **the skeleton prompt** and **negative constraints**. Just sketching the skeleton first and then spelling out what not to do dramatically reduces the number of fix rounds.

Once these two become second nature, you can keep the quality of your output high without typing the code yourself.

Chapter 4 covers the reasons behind the tools I actually use and how to set up your environment. Chapter 5 moves into the basics of Tauri 2.

Minimal verification commands

Run these inside the project and check the visible feature result as well as exit codes:

```
cargo check --manifest-path src-tauri/Cargo.toml
pnpm check
pnpm tauri dev
```

Chapter 4. Tools and Environment —Claude Code, Cursor, Codex, GitHub Copilot

Chapter 3 covered the prompt loop and six patterns. This chapter connects those patterns to actual tools. What matters is not which tool you use, but how you keep the loop going with that tool.

It is hard to settle the tool question with a comparison table alone. Questions like “Is Claude Code or Cursor better?” don’t actually improve your day-to-day development. There is a better

question: looking at the size and risk of the work in front of you, which tool gets you to a result without getting stuck?

Even with the same AI, the environment you need when fixing a single function is different from what you need when reshaping the whole repository. By **feedback loop** here I mean the repeating cycle of asking the AI for something, checking the result, and asking again to fix it. Shorter loops let you catch mistakes sooner. The good tool is not the “smartest tool”—it’s the one that makes today’s feedback loop the shortest.

What you’ll walk away with after this chapter: a working Tauri app skeleton, a project rules file (CLAUDE.md or .cursorrules), and two git commits. In other words, the environment you’ll use from Chapter 5 onward when you start building seriously with AI.

AI coding tools fall into three broad groups based on where they run. There are **terminal (CLI)** tools, tools that run **inside an IDE**, and **cloud-based** tools. Terminal tools are good for handling an entire repository with a single command. IDE tools are good for quick edits right next to the code. Cloud tools run outside your machine, which is what you want when you’d rather delegate the work without touching your local setup.

How this chapter flows

- Four tools, four approaches
- When to use which
- A minimal Tauri 2 + SvelteKit + Rust environment
- The first 30 minutes—from scaffold to first commit
- Rules files (CLAUDE.md, .cursorrules)
- Constraints to give the AI—specify your stack and versions
- Going further: MCP, Agents, slash commands

Learning goals

- Understand how the four tools (Claude Code, Cursor, Codex, GitHub Copilot) differ.
- Set up a minimal local environment with Tauri 2 + SvelteKit + Rust running.
- Write a CLAUDE.md / .cursorrules rules file to raise the quality of AI output.

Cross-reference: This chapter picks up from the prompt loop and six patterns in Chapter 3, and leads into the deeper dive on Tauri 2 in Chapter 5.

4.1 Four tools, four approaches

As of 2026, four AI coding tools are most often compared in practice. Even when the names sound similar, the differences show up first in **where the tool runs** and **how it works**, not in the underlying model.

Tool	Runtime	Model	Main strength
Claude Code	Terminal (local)	Claude (Anthropic)	Long context, full-repo exploration
Cursor	IDE (VS Code fork)	Configurable	Fast inline edits and autocomplete
OpenAI Codex	Cloud sandbox	OpenAI	Isolated execution, doesn't touch your local machine
GitHub Copilot	IDE + Cloud + CLI	Configurable	GitHub issue/PR integration, inline autocomplete

Each tool fits a different spot. Let's look at them one by one.

Claude Code runs in your terminal and can take your whole project folder as context. It's strong at long conversations and at handling many files at once.

Cursor is an IDE built on top of VS Code. It lets you mix agent-driven work and autocomplete on the same screen, and it's fast for short edits.

OpenAI Codex shares its name with the 2021 code-generation model, but it is a completely different product. The defining trait is that it runs in an isolated cloud sandbox, so it doesn't affect your local environment.

GitHub Copilot started as an autocomplete tool and has grown into an agent platform. There are now four ways to use it: Agent mode inside the IDE, a Cloud Agent that handles GitHub issues automatically, a Copilot CLI for the terminal, and the original inline autocomplete. For now, just remember the names and what each does.

In this book I mostly use Claude Code and Cursor. GitHub Copilot and Codex come up as alternatives for specific situations.

Heads up: Running several agents on the same task at the same time tends to cause conflicts, especially when two agents touch the same file. Pick one tool per task, and make the switch clean when you change.

4.2 When to use which

Choosing a tool is not a matter of taste —it’s a matter of **the unit of work**. The question to ask is: “How wide does this task need me to look at once?”

When Claude Code fits

Adding new features that touch multiple files, large refactors, laying out the initial structure of a project. The same goes for debugging when you need to read logs, stack traces, and the source together. Because it can see the whole file tree, it’s easier to trace causes. The trade-off is that the wider scope means you have to give it a clear **verification standard**. Without that, it can change many files and still drift away from what you actually asked for.

When Cursor fits

A single function inside one file, an explanation of a specific snippet, a one-line autocompletion —anything where **the location to edit is already obvious**. When the change spans many directories, the cursor-centric view starts to feel too narrow.

When OpenAI Codex fits

When you’d rather not touch your local environment and want to run things in an isolated cloud workspace. It’s useful when you want to hand off long-running work without loading down your local machine.

When GitHub Copilot fits

Inline autocomplete is the fastest way to type out repetitive boilerplate. When you want a GitHub issue to be turned into a PR automatically, you use the Cloud Agent.

In my own work, Claude Code takes up more than half of the day. Cursor comes next, and the rest is whatever fits the situation. The more I’m building a new app from scratch, the more Claude Code I use; the more I’m polishing existing screens, the more Cursor I use.

4.3 A minimal Tauri 2 + SvelteKit + Rust environment

This section is a collection of commands you'll type when you set up a Tauri 2 development environment for the first time. I'll explain why each one is needed.

Pinning versions

Before installing anything, you need to pin versions exactly. **This is especially important for AI-assisted development.** The AI has been trained on code from many different versions, so if you don't say which version you want, it can generate Tauri 1.x code (which won't run today), Svelte 4 stores, or older SvelteKit routing. This book uses the following versions as its baseline.

- Tauri: 2.11.0
- SvelteKit: 2.59.0
- Svelte: 5.55.0 (rune syntax)
- Node.js: 24.x LTS
- Rust: stable channel, 1.95.0

The required install steps

You only have to run each step once. If a command finishes without hanging, it succeeded.

1. The Rust toolchain

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source ~/.cargo/env
rustup default stable
```

After installing, `rustc --version` should report 1.95.0 or higher.

2. Node.js (using fnm for version management)

```
curl -fsSL https://fnm.vercel.app/install | bash
source ~/.zshrc
fnm install 24
fnm use 24
```

```
fnm default 24
```

3. Package manager (pnpm recommended)

```
npm install -g pnpm@latest
```

4. The Tauri CLI

```
cargo install tauri-cli@2.11.0
pnpm add -D @tauri-apps/cli@2.11.0
```

5. Platform-specific requirements

Platform	Install command
macOS	<code>xcode-select --install</code>
Linux (Ubuntu/Debian)	<code>sudo apt install libwebkit2gtk-4.1-dev libayatana-appindicator3-dev librsvg2-dev patchelf</code>
Windows	Microsoft C++ Build Tools, WebView2 (usually preinstalled on Windows 11)

Setting up the AI tools

```
# Claude Code CLI
npm install -g @anthropic-ai/claude-code
claude --version

# Codex CLI
npm install -g @openai/codex
codex --version
```

You can download the Cursor IDE from cursor.com. It imports your existing VS Code settings automatically.

Account login is available. Set API-key environment variables only when choosing separate API billing.

```
export ANTHROPIC_API_KEY="your-key-here"
export OPENAI_API_KEY="your-key-here"
```

Heads up: Never put API keys directly in your code, and never commit them to git. Use environment variables, or a `.env.local` file that is listed in `.gitignore`. When an AI tool suggests putting the key directly in the source, refuse every time.

Initializing a SvelteKit project with Tauri

Scaffolder versions and Tauri CLI versions are independent. The old `create-tauri-app@2.11.0` instruction does not identify a published scaffolder version. Create SvelteKit first, then add Tauri:

```
pnpm dlx sv create my-app
cd my-app
pnpm install
pnpm add @tauri-apps/api
pnpm add -D @tauri-apps/cli@2.11.4 @sveltejs/adaptor-static
pnpm tauri init
```

Choose SvelteKit minimal and TypeScript. For tauri init, use `http://localhost:5173`, `pnpm dev`, `pnpm build`, and frontend output `../build`. A WebView does not bundle the SvelteKit server, so configure static SPA output:

```
import adapter from '@sveltejs/adaptor-static';
import { vitePreprocess } from '@sveltejs/vite-plugin-svelte';

const config = {
  preprocess: vitePreprocess(),
  kit: {
    adapter: adapter({ fallback: 'index.html' })
  }
}
```

```
};

export default config;
```

Disable SSR at the root layout:

```
// src/routes/+layout.ts
export const ssr = false;
```

Check `build.frontendDist` is `../build`, then run `pnpm tauri dev`. The SvelteKit screen should open in a Tauri window without console errors.

When installation fails —the three most common causes

If the window doesn't open or the build fails, beginners almost always hit one of these three problems.

- **Rust path not set up:** If `rustc --version` doesn't work, run `source ~/.cargo/env` again or open a new terminal.
- **WebView2 missing (Windows 10):** Tauri uses the system webview. Windows 11 usually has it preinstalled, but Windows 10 doesn't. Download the “Evergreen Bootstrapper” from Microsoft's official site.
- **Missing Linux libraries:** If packages like `libwebkit2gtk-4.1-dev` are missing, you'll get compile errors during `pnpm tauri dev`. Re-check the `apt install` command above.

Note: Cross-platform builds (for example, building a Windows binary from macOS) and release automation are covered separately in Chapter 12. For now, all you need is **your computer running the app**.

4.4 The first 30 minutes —from scaffold to first commit

Use the SvelteKit-plus-Tauri setup in section 4.3, then follow this checklist.

Quick checks (5 minutes)

1. Test the dev server

```
cd your-app-name
pnpm install
pnpm tauri dev
```

The app should open and show “Welcome to Tauri!” The browser console should have no errors.

2. Test the build

```
pnpm tauri build
```

The first run takes 2-3 minutes while Rust compiles. If it finishes without errors, the executable is in `src-tauri/target/release/bundle/`.

3. Check IDE integration

Open the project in Cursor or VS Code. You should see three things: - Rust syntax highlighting in the `src-tauri/` folder - Svelte syntax highlighting in the `src/` folder - No TypeScript errors

Project setup (10 minutes)

4. Create the rules file

First, create `CLAUDE.md` at the project root. When Claude Code starts a session, it **automatically reads the `CLAUDE.md` in your working folder and includes it as system context**. In other words, you don’t have to repeat the same rules in every prompt —what you write in this file becomes a baseline agreement.

Project name**## Stack**

- Tauri 2.11.0, SvelteKit 2 + Svelte 5, TypeScript, DuckDB
- Svelte 5 rune syntax only (\$state, \$derived, \$effect)

Structure

- Frontend: SvelteKit (src/)
- Backend: Rust Tauri commands (src-tauri/src/commands/)
- Database: access DuckDB only via Tauri commands

Rules

- No direct API calls from the frontend
- All external data goes through Tauri commands
- All command returns use Result<T, AppError>
- No console.log in production code

Prohibited

- No Svelte stores or legacy reactive statements
- No unwrap() in Rust (use proper error handling)
- No hardcoded file paths (use the Tauri path API)

In the same place, create a `.cursorrules` file too. Cursor reads it for the same purpose. The contents should be the same kind of rules as in `CLAUDE.md`.

5. Initialize git and make the first commit

```
git init
git add .
git commit -m "Initial Tauri + SvelteKit scaffold"
```

Your first productive change (15 minutes)

6. Write your first custom Tauri command

Before writing any code yourself, check that the AI actually follows the Tauri patterns. Send the AI this prompt.

Create a simple Tauri command called "get_app_info" that returns basic app metadata.

Return type: `Result<AppInfo, AppError>` where `AppInfo` includes:

- `app_name: String`
- `version: String`
- `platform: String`

Follow the project rules in `CLAUDE.md`.

7. Test the new command

```
pnpm tauri dev
```

Calling this command from the frontend should produce no errors.

The first places to check when AI code doesn't work

- File location:** Is the new file in the folder declared in `CLAUDE.md` (`src-tauri/src/commands/`)?
- Return type:** Does it end in `Result<T, AppError>`? Is there any stray `unwrap()`?
- Bindings:** Did the TypeScript caller for the frontend land in `src/lib/api/` as well?
- Versions:** Did the AI mix in Svelte 4 stores or Tauri 1.x patterns? Even one of these being off will break the build. Before rewriting code, walk through this list first.

8. Commit the working command

```
git add .
git commit -m "Add get_app_info Tauri command"
```

Confirmation checklist

After 30 minutes, check the items below.

- ☒ Both dev server and build commands work
- ☒ A rules file exists that constrains how the AI behaves
- ☒ There are two commits (scaffold + first feature)
- ☒ You have confirmed the AI follows the rules
- ☒ No TypeScript or Rust compile errors

The outcome: every AI task from here forward continues with the patterns you set up here.

Heads up: Don't skip the verification steps. Setup issues need to be caught before you've piled on 500 lines of code. If the AI keeps generating code that doesn't compile or doesn't run, finding the cause later costs far more time.

4.5 .cursorrules / CLAUDE.md —putting the rules files to work

These two files hold **the rules the AI follows by default**. You don't have to explain them in every prompt; the AI just observes them. In 4.4 we saw the basic template for a new project. Here we look at how a real shipped project's rules file gets polished.

CLAUDE.md (for Claude Code) —a real example

Place it at the repository root and Claude Code reads it automatically when a session starts. Below is an excerpt from the CLAUDE.md in the Mandai project.

```
# Mandai

## Stack
- Tauri 2, SvelteKit 2 + Svelte 5, TypeScript, DuckDB (duckdb crate)
- AI providers: OpenAI, Anthropic, Gemini (unified command)

## Rules
- Svelte 5 runes only ($state, $derived, $effect). No legacy store syntax.
- Tauri commands live under src-tauri/src/commands/*.rs.
- Errors funnel into MandaiError, which is based on thiserror.
- No AI calls from the frontend. Always go through a Tauri command.

## Prohibited
- No console.log in commits.
- No `unwrap()` in production code (tests are fine).
```

With this file in place, you no longer need to repeat “use Svelte 5 runes only” or “no unwrap” in every prompt.

.cursorrules —same idea, different filename

The concept is identical to `CLAUDE.md`. Only the filename differs. Cursor reads `.cursor-rules`; Claude Code reads `CLAUDE.md`. If your project uses both tools, keep both files in place and make sure their core rules match.

Different apps may need extra rules. A real-time data app, for example, may add app-specific rules like “manage WebSocket connections in the Tauri backend” and “the frontend receives data through Tauri events.” Pull the common rules from your new-project template, then add only the app-specific patterns.

What the rules files give you

- Prompts get shorter. You stop explaining “in this repository...”.
- Fewer loop iterations, fewer AI mistakes.
- When you hand the repository to a teammate, **the AI rules double as team rules**.

Update the rules file whenever you upgrade dependencies or change the structure. A stale file leads the AI to write outdated patterns.

4.6 Constraints to give the AI —name the stack and versions

Once the rules file is in place, the next question is **what constraints to spell out in each prompt**. The core rule boils down to one line.

When you give the AI a task, name **the technologies and versions to use, and the patterns it should assume**.

If you don’t add constraints, the AI defaults to the most common pattern in its training data (often Electron, React, PostgreSQL). Adding a line or two at the top of the prompt —“use Tauri 2”, “Svelte 5 rune syntax only”, “the backend is Rust”—is enough to block these wrong defaults.

Turn task constraints into observable completion criteria. For a database: “The second launch does not rerun an applied migration.” For WebSockets: “After three disconnections, exactly

one connection remains active.”For a UI: “Saving and inspecting errors are possible with the keyboard alone.”Chapters 7, 13, and 14 cover database, recovery, and deployment boundaries. Define how success is observed before choosing a tool.

Heads up: Tool and tech choice constraints need to be given **explicitly and repeatedly**. Without constant reminders, the AI falls back on the most common pattern as its default.

4.7 Going further: MCP, Agents, slash commands

New terms are piling up around AI tools. On the first read, just learn the names and what they do —come back when you actually need to use one.

MCP (Model Context Protocol) —a standard adapter the AI uses to connect to outside tools and data (file systems, git, databases, web search, and so on). With it plugged in, a short request like “check the schema of the cells table in this project”works, because the AI reads it directly. You’ll spend less time copy-pasting schemas.

Agent —an AI that plans and runs multiple steps on its own. Claude Code’s default behavior is already this kind. “Add tests to this repository”triggers planning, execution, and verification on its own. Just be aware: when all the steps run in a batch, it’s hard to see where a mistake happened, so it’s safer to use a mode where all the output is visible.

Slash commands —**saved prompts with names**, like `/commit` or `/review-pr`. Worth creating once you find yourself repeating the same request five times or more. Don’t bother for a one-off request.

Repository-structure decisions (monorepo vs. separate repos) also don’t need attention on the first read. Start your first app as a single repository, and only worry about a monorepo once you have clearly shared code. We come back to this in the operations chapters later in the book.

4.8 Five principles for choosing tools

1. **Write the rules file first.** When you create a new repository, write `CLAUDE.md` before writing code. The quality of every prompt you send the AI is mostly decided in this file.

2. **Review your setup once a month.** AI tools change fast. The setup that feels optimal today may be stale a month from now.
3. **Always pin versions.** If you don't say which version, the AI picks one on its own. When that version doesn't match your project, bugs appear.
4. **Test the setup immediately.** A flawed initial setup compounds problems. Confirm that dev, build, and the AI integration actually work before building features.
5. **Maintain the rules file alongside the code.** When you upgrade a dependency or change the structure, edit `CLAUDE.md` too. A stale file leads the AI into outdated patterns.

What to take away from this chapter

- **Different tools shine at different jobs.** Big structural work calls for Claude Code, one-file editing calls for Cursor, repetitive input calls for autocomplete. It's a division of labor, not a competition.
- **You must tell the AI what to use.** Say nothing, and the AI defaults to the most common pattern in its training data (Electron, React, PostgreSQL).
- **You must pin versions.** If you don't write things like Tauri 2.11.0 and SvelteKit 2.59.0, you end up with mismatched code mixed together.
- **CLAUDE.md and .cursorrules give you the biggest payoff.** They free you from repeating the same rules in every prompt.
- **In 30 minutes you should have the environment set up, the rules file written, and the first commit made.** That pattern becomes the baseline for every task that follows.
- For MCP, Agents, and slash commands, knowing the names and what they do is enough for now. Come back when you need them.

If I had to sum it up: **AI code quality is decided more by the constraints you give than by which model you pick.** An ordinary AI with good constraints beats an excellent AI with none.

What you can now do: – Pick an AI tool for each kind of task (structural work vs. fine edits vs. repetitive code) – Set up a complete Tauri 2 + SvelteKit development environment with pinned versions – Write rules files that raise the quality of AI code and cut down on rewrites – Avoid the most common mistakes: version mismatches, missing constraints, a bad scaffold – Establish productive patterns in the first 30 minutes of a new project

What's next: Chapter 5 dives deep into the fundamentals of Tauri 2. We look at the architecture, the command system, and the state-management patterns that hold up an AI-driven workflow.

Cross-reference: The workflow patterns set up in this chapter lead into rapid prototyping in Chapter 6 and the production deployment strategies in Chapter 12.

Chapter 5. Tauri 2 Fundamentals —Core Patterns and Avoiding AI Mistakes

Of all the frameworks in this book, Tauri 2 is the trickiest to direct an AI through. On the surface it looks like a simple combination of a web frontend and a Rust backend, but in practice your app's stability and security come down to how strictly you respect the boundary between those two runtimes.

Tauri runs on top of two independent runtimes. Rust handles the backend process, and a WebView running SvelteKit handles the frontend. The two do not share memory; they only exchange messages through an **IPC bridge** (an inter-process communication channel). Each

time data crosses the boundary, it is converted to and from JSON (serialized and deserialized). This boundary is not just an organizational line —it is the core of Tauri’s security design, and it is where AI tools make the most mistakes. An AI may try to call functions directly across the boundary, expose secret keys to the frontend, or write async code that compiles fine but stalls under load.

This chapter is less a Tauri syntax cheat sheet than a **map of the boundary**. In the end, deciding what belongs on which side is the developer’s job.

When you first encounter Tauri, it helps to think of it as a “small web app plus a safe system backend.”The screen is built with web technology, and sensitive work like files, databases, processes, and the keychain happens through Rust commands. The frontend only sends requests; the actual privileges sit with the backend.

The rule for drawing the line is simple. **Anything that changes what the user sees on screen belongs in the frontend. Anything that touches the operating system or your data belongs in a Rust command.** Changing a button color or filtering a list should end in Svelte. Saving a file, opening DuckDB, or reading an API key from the keychain belongs in a Rust command.

A restaurant analogy works well: the frontend is the dining room the customer sees, and the Rust backend is the kitchen and the safe. The customer can pick from the menu and place orders, but they cannot open the refrigerator or pull a card out of the safe themselves. Order slips go to the kitchen through a designated window, and only the finished dish comes back out. Tauri’s `invoke()` is that order slip.

How this chapter flows

- 5.1 Architecture overview —why the boundary matters
- 5.2 Commands —how the frontend calls the backend
- 5.3 State —sharing data between commands
- 5.4 Events —pushing data from backend to frontend
- 5.5 Plugins —bundles of common functionality
- 5.6 Permissions and capabilities —Tauri 2’s new security model
- 5.7 Frontend —connecting with SvelteKit 2 + Svelte 5
- 5.8 Project structure
- 5.9 A mental model and prompting strategy

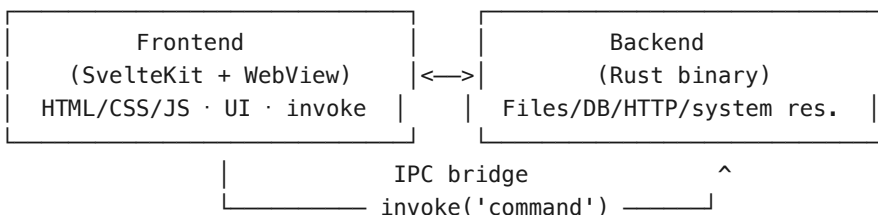
- 5.10 Mistakes AI makes most often
- 5.11 Summary and quick reference

Learning goals

- Understand Tauri 2's two-runtime architecture (Rust backend plus WebView frontend).
- Learn the pattern of exposing a backend function to the frontend with `#[tauri::command]`.
- Share data between commands using `tauri::State<T>`.
- Stream backend progress to the screen in real time using the event system.
- Understand Tauri 2's permission model (capabilities) and catch missing-permission errors quickly.
- Recognize the patterns AI gets wrong most often in Tauri 2 code.

5.1 Tauri 2 Architecture Overview —Why a Clear Boundary Matters

Let's start with a picture of how the two processes connect.



When the frontend calls `invoke('get_user_data')`, the request flows through these steps.

1. `invoke()` packs the function name and arguments into JSON.
2. The IPC layer carries that data to the backend.
3. Rust runs the `get_user_data` command.
4. Rust sends the result back.

5. The frontend receives the response as a Promise.

The flow resembles an HTTP API call, but no network is involved. Inside the same app, the WebView and the Rust process exchange messages over an internal channel.

This architecture suits AI-assisted development well because **the boundary is sharp**. If you say, “Write a backend command that saves user settings,” the AI has a clear target. In Electron, both the main and renderer processes are JavaScript, so it is easy for an AI to blur the line between frontend and backend. In Tauri 2 the language difference between Rust and JavaScript enforces the separation.

Why a WebView instead of native UI

Tauri 2 uses the WebView that ships with the operating system (WebKit on macOS, WebView2 on Windows, WebKitGTK on Linux). That lets you build the UI with HTML, CSS, and JavaScript, and a single codebase runs on every platform without any platform-specific UI code.

5.2 Commands —The `#[tauri::command]` Pattern

A command is how the frontend calls a backend function. File system access, database queries, and HTTP requests all run as commands.

A rule for naming commands

Do not tie a command name directly to a UI event. `onSaveButtonClick` is bound to a specific button, while `save_user_settings` keeps working even if that button disappears.

A command name should describe a business action, not a screen event.

The smallest example

What this code does: the frontend sends a name, Rust builds a greeting string, and Rust sends it back. It contains all three pieces a Tauri command needs —the `#[tauri::command]` attribute, registration in `invoke_handler`, and the frontend `invoke()` call.

```
// src-tauri/src/lib.rs
#[tauri::command]
pub fn greet(name: String) -> String {
    format!("Hello, {}!", name)
}

pub fn run() {
    tauri::Builder::default()
        .invoke_handler(tauri::generate_handler![greet])
        .run(tauri::generate_context!())
        .expect("error while running tauri application");
}
```

The frontend calls it by the same name.

```
// src/lib/api.ts
import { invoke } from '@tauri-apps/api/core';

export async function greet(name: string): Promise<string> {
    return await invoke('greet', { name });
}
```

What to check: the name must match in all three places —the Rust function name, the entry in `generate_handler![]`, and the string passed to `invoke()` on the frontend. If any one of them is off, you get a runtime command not found error. **A name mismatch is the bug you will hit most often when adding a new command.**

Commands that can fail

In a real app, most commands are asynchronous and can fail. A file might be missing, permissions might be wrong, or the network might drop. Instead of returning only a success value, make failure explicit with `Result<T, String>`.

```
#[tauri::command]
pub async fn read_config(path: String) -> Result<String, String> {
    if !std::path::Path::new(&path).exists() {
        return Err(format!("File not found: {}", path));
    }
}
```

```

    }
    tokio::fs::read_to_string(&path).await
        .map_err(|e| format!("Failed to read file: {}", e))
    }

```

The `read_config` snippet illustrates Result handling only. A product must normalize paths under its allowed app-data directory and reject traversal or symlink resolutions outside that directory. Do not permit arbitrary frontend-supplied absolute paths.

On the frontend, catch that failure with `try/catch`.

```

try {
  const text = await invoke<string>('read_config', { path });
} catch (error) {
  console.error('Failed to read config:', error);
}

```

Data with multiple fields

For multi-field data like settings, orders, or sessions, define a struct and derive `Serialize` and `Deserialize`. Tauri converts it automatically, so the frontend and backend share the same shape of data.

```

use serde::{Deserialize, Serialize};

#[derive(Serialize, Deserialize)]
pub struct UserSettings {
    theme: String,
    auto_save: bool,
    recent_files: Vec<String>,
}

#[tauri::command]
pub async fn save_user_settings(settings: UserSettings) -> Result<(),
String> {
    // Save to a file or database
    Ok(())
}

```

What to check: any new command (such as `save_user_settings`) also needs to go into the registration list, like `invoke_handler![greet, save_user_settings]`.

AI prompting tip: instead of “Make commands to manage user settings,” include the struct definition (field names and types) in your prompt. The results will be much more consistent.

5.3 State — `tauri::State<T>`

When several commands need to share data, use Tauri’s built-in state management. It is cleaner than a global variable, and it is safe under concurrent access from multiple threads.

Think of state as a shared locker inside the app. When several commands need the same settings, database connection, or cache, they pull it from the locker instead of creating a new one each time. Because multiple commands can reach into the locker at once, you need a lock.

The basic pattern: `RwLock`

What this code does: a cache that commands share is registered once when the app starts, and each command pulls the value out with `State<'_, AppState>`.

```
use std::sync::RwLock;
use std::collections::HashMap;
use tauri::State;

pub struct AppState {
    pub cache: RwLock<HashMap<String, String>>,
}

#[tauri::command]
pub async fn set_cache_value(
    key: String, value: String, state: State<'_, AppState>
) -> Result<(), String> {
    let mut cache = state.cache.write()
        .map_err(|e| format!("Lock error: {}", e))?;
    cache.insert(key, value);
    Ok(())
}
```

```

}

// Register the state when the app starts
fn run() {
    tauri::Builder::default()
        .manage(AppState { cache: RwLock::new(HashMap::new()) })
        .invoke_handler(tauri::generate_handler![set_cache_value])
        .run(tauri::generate_context!())
        .expect("error");
}

```

`RwLock` is the lock that keeps shared data safe. Only one writer can hold it through `write()`, while many readers can hold it at the same time through `read()`.

Choose locks by critical section, not by a blanket rule

Neither `std::sync::RwLock` nor `tokio::sync::Mutex` is always correct. A standard lock is simple for brief synchronous cache access. A Tokio lock may be appropriate when exclusive ownership must span asynchronous work. Avoid keeping an unnecessary lock across `.await`.

This example holds shared state while waiting for the network, making other commands wait:

```

// Holds shared state while waiting for network I/O.
use tokio::sync::Mutex;
pub struct BadState { data: Mutex<Vec<String>> }

#[tauri::command]
pub async fn bad_command(state: State<'_, BadState>) -> Result<(),
String> {
    let mut data = state.data.lock().await; // asynchronous lock
    data.push("item".to_string());
    send_to_server(&data).await?;          // still holds the lock
    Ok(())
}

```

Update briefly, clone a snapshot, and release the lock before awaiting the network. `send_to_server` represents the application-specific I/O function.

```
// Release the lock before awaiting I/O.
use std::sync::RwLock;
pub struct GoodState { data: RwLock<Vec<String>> }

#[tauri::command]
pub async fn good_command(state: State<'_, GoodState>) -> Result<(),
String> {
    let snapshot = {
        let mut data = state.data.write()
            .map_err(|e| format!("Lock poisoned: {}", e))?;
        data.push("item".to_string());
        data.clone()
    }; // lock released here
    send_to_server(&snapshot).await?;
    Ok(())
}
```

RwLock suits read-heavy access; Mutex suits exclusive access. Move slow synchronous database work to a worker thread or `spawn_blocking` rather than blocking an async executor.

Resources that are expensive to initialize

For something like a DuckDB connection that takes time to build, you want to create it once for the whole app. Chapter 7 covers that case along with the DuckDB example. For now, just remember **there is a separate pattern for “build once and share across commands.”**

5.4 Events —emit and listen

Commands are how the frontend asks the backend. But sometimes the backend needs to speak first—for AI response streaming, or for the progress of a long-running process. Instead of having the frontend keep asking “Are we done yet?”, the backend **pushes** events out.

Sending from backend to frontend

What this code does: for ten seconds it sends a progress update to the frontend once per second, and at the end it sends a completion event.

```

use tauri::{AppHandle, Emitter}; // Emitter provides emit()

#[tauri::command]
pub async fn start_long_process(app: AppHandle) -> Result<(), String> {
    tokio::spawn(async move {
        for i in 0..10 {
            tokio::time::sleep(tokio::time::Duration::from_secs(1)).await;
            let _ = app.emit("process_progress", (i + 1) as f64 / 10.0);
        }
        let _ = app.emit("process_complete", ());
    });
    Ok(())
}

```

If it fails, look here: forgetting `use tauri::Manager` makes `app.emit()` fail to compile.

The frontend receives events with `listen`. Detach the listener when it is no longer needed to avoid a memory leak.

```

import { listen } from '@tauri-apps/api/event';

$effect(() => {
    const unlistenPromise = listen<number>('process_progress', (event)
    => {
        console.log('진행:', event.payload);
    });

    // Clean up even if registration completes after disposal.
    return () => { void unlistenPromise.then((unlisten) => unlisten());
    };
});

```

The streaming pattern

The same pattern works for streaming an AI response to the screen. Each time the model produces a token, push it as an event, and send a completion event when the stream ends.

```
#[tauri::command]
pub async fn stream_ai_response(prompt: String, app: AppHandle) ->
Result<(), String> {
    let mut stream = client.chat().create_stream(prompt).await?;
    while let Some(chunk) = stream.next().await {
        if let Some(content) = chunk?.choices[0].delta.content {
            let _ = app.emit("ai_chunk", content);
        }
    }
    let _ = app.emit("ai_complete", ());
    Ok(())
}
```

This pattern shows up again in Chapter 6 (timer notifications), Chapter 8 (AI response streaming), and Chapter 10 (capturing process output).

5.5 Plugins —The tauri-plugin-* Ecosystem

Tauri 2 ships common functionality as plugins. Each plugin bundles Rust commands and, when needed, a frontend JavaScript API.

Plugins commonly used in AI apps

Plugin	Use case
tauri-plugin-shell	Spawning processes, running Python scripts
tauri-plugin-store	Saving settings like user preferences
tauri-plugin-dialog	File and folder picker dialogs
tauri-plugin-fs	Reading and writing files
tauri-plugin-http	HTTP requests from Rust
tauri-plugin-websocket	WebSocket connections

Adding a plugin takes two steps. Add the dependency in `Cargo.toml`, and register it in your Rust code.

```
fn run() {
    tauri::Builder::default()
        .plugin(tauri_plugin_store::Builder::new().build())
        .plugin(tauri_plugin_dialog::init())
        // ...
}
```

Security note: `tauri-plugin-store` writes data to disk as JSON. For values that must not leak, such as API keys, encrypt them separately or store them in the system keychain through `tauri-plugin-stronghold`.

5.6 Permissions and Capabilities —The Tauri 2 Boundary

Capabilities restrict which Tauri core and plugin APIs the WebView frontend may call, with permissions and path/URL scopes per window or WebView. Custom Rust commands registered with `generate_handler!` are callable from app windows by default; capabilities do not automatically inspect Rust code inside them. They cannot compensate for unsafe custom command implementations.

```
{
  "$schema": "../gen/schemas/desktop-schema.json",
  "identifier": "main",
  "windows": ["main"],
  "permissions": [
    "core:default",
    "fs:allow-read-text-file",
    "fs:allow-write-text-file",
    "dialog:allow-open",
    "dialog:allow-save",
    "http:default",
```

```

    "store:default"
  ]
}

```

A plugin's default permission bundle varies by plugin. Check generated schemas or permission tables instead of inferring breadth from the name. Select required actions, such as `fs:allow-read-text-file`, and narrow path/URL scopes too.

Check plugin permissions and custom commands separately

The following frontend call uses the filesystem plugin and therefore needs its permissions and path scope:

```

import { readTextFile } from '@tauri-apps/plugin-fs';
import { BaseDirectory } from '@tauri-apps/api/path';

const text = await readTextFile('notes/today.md', {
  baseDir: BaseDirectory.AppData
});

```

Without permission, the Promise rejects with a permission error. Check plugin registration, permission identifiers, and allowed paths together. Error wording varies by version; choose identifiers from the generated schema rather than copying error text blindly. Custom Rust commands performing the same work must validate paths, allowlists, and authentication themselves.

5.7 Frontend —SvelteKit 2 + Svelte 5 plus Tauri

The frontend of a Tauri app is a SvelteKit application running inside the WebView. Add the Tauri bridge package and you get access to the system APIs.

```
pnpm add @tauri-apps/api
```

There are three imports you reach for most often.

```
import { invoke } from '@tauri-apps/api/core';
import { listen } from '@tauri-apps/api/event';
import { getCurrentWindow } from '@tauri-apps/api/window';
```

Svelte 5 runes and Tauri events

What this code does: it receives progress updates pushed from the backend into `$state` and reflects them directly on the screen. `$derived` computes the displayed text automatically.

```
<script lang="ts">
import { invoke } from '@tauri-apps/api/core';
import { listen } from '@tauri-apps/api/event';

let progress = $state(0);
let isProcessing = $state(false);

async function startProcess() {
  isProcessing = true;
  try {
    await invoke('start_long_process');
  } catch (error) {
    isProcessing = false;
    console.error(error);
  }
}

let progressPercent = $derived(Math.round(progress * 100));

$effect(() => {
  const subscriptions = [
    listen<number>('process_progress', (e) => { progress =
e.payload; }),
    listen('process_complete', () => { isProcessing = false; }),
  ];
  return () => {
    for (const pending of subscriptions) {
      void pending.then((unlisten) =>
unlisten()).catch(console.error);
    }
  }
});
```

```

    };
  });
</script>

<div>{isProcessing ? `처리 중... ${progressPercent}%` : '준비'}</div>

```

Collecting command calls in an API wrapper

If you call `invoke()` directly inside components, typos in command names spread everywhere. Wrap every command call in a function under `src/lib/api/` and you become resistant to change.

```

// src/lib/api/files.ts
import { invoke } from '@tauri-apps/api/core';

export const FilesAPI = {
  readDirectory: (path: string) =>
    invoke<FileItem[]>('read_directory', { path }),
  readFile: (path: string) => invoke<string>('read_file', { path }),
  writeFile: (path: string, contents: string) =>
    invoke<void>('write_file', { path, contents }),
};

```

The component just calls `FilesAPI.readFile(...)`. If a command name changes, you fix one place and you're done.

5.8 Project Structure —A Reference Layout

All six projects in this book use the same structure.

```

my-tauri-app/
├── src-tauri/                # Rust backend
│   ├── src/
│   │   ├── lib.rs           # Entry point
│   │   ├── commands/       # files.rs, ai.rs, database.rs ...
│   │   ├── state/          # app_state.rs
│   │   └── utils/

```

```

|   ├── capabilities/main.json
|   ├── Cargo.toml
|   └── tauri.conf.json
├── src/                                # SvelteKit frontend
|   ├── lib/
|   │   ├── api/                       # files.ts, ai.ts ...
|   │   ├── stores/                   # app.svelte.ts, files.svelte.ts
|   │   └── components/
|   ├── routes/
|   └── app.html
├── package.json
└── vite.config.ts

```

When you ask Claude Code for work, name the directory at the same time.

“Add a command that saves user preferences in `src-tauri/src/commands/database.rs`, and also add a wrapper function in `src/lib/api/index.ts`.”

Now the AI does not have to guess where the code goes.

5.9 A Mental Model for AI-Assisted Tauri 2 Development

Tauri 2 enforces role boundaries, which actually helps when working with an AI. The structure itself enforces these rules.

- The frontend cannot touch the file system directly -> it must go through a command.
- The backend cannot touch the DOM directly -> it must notify through an event.
- Data is always converted when it crosses the boundary.
- Permissions must be declared in code ahead of time.

These rules are baked into the code itself, so code that violates them simply does not work from the start.

Prompting strategy

Even for the same feature, the prompt you write changes the result.

- ☒ "Add a save-to-file feature."
- ☑ "In `src-tauri/src/commands/files.rs`, add a Rust command that writes text content to a file path. Include error handling for invalid paths and permission issues. Then create a TypeScript wrapper in `src/lib/api/files.ts` that calls this command."

The more you spell out which side does what, the more closely the code matches the structure.

5.10 Verification Checklist

- Register every new command in `generate_handler!`, including `save_user_settings`.
- Check frontend plugin permissions and path/URL scope; custom Rust commands validate their own inputs.
- Keep lock lifetimes short and choose synchronous or asynchronous locks for actual access requirements.
- Register listeners before starting work. Dispose them on teardown, including late registration.
- Match event payload types across Rust and TypeScript. Here `process_progress` is a number in `[0,1]`, and `process_complete` clears the processing flag.
- For high-volume streams, batch updates every 20-50ms or use a Tauri Channel instead of a global event per token.
- Use SvelteKit 2 with Svelte 5; verify static-adaptor configuration and disable SSR for native-only routes.
- Test command failure, rapid mounting/unmounting, and repeated execution.

The optional educational archive contains `tauri/ch05-local-notes/src/App.svelte`, a standalone Svelte 5 + Vite + Tauri 2 exercise. It is not the complete Mandai or SvelteKit application. The body explains the command, `app-file`, event cleanup, and error-handling boundaries; the exercise README supplies installation and completion checks.

Chapter 6. Designing Mandai —How Big a Chunk Can You Hand to AI?

Part II —Mandai (Chapters 6-9) Now we actually build an app. Chapters 1-5 covered principles and tools; Part II shows how they work across a real project. The order is design (Ch. 6) -> database (Ch. 7) -> AI APIs (Ch. 8) -> mistakes and lessons (Ch. 9). You'll watch the loop spin in practice. On a first read-through you don't have to memorize every feature of Mandai. Just hold onto the shape: "Instead of handing the whole app to AI at once, split it by screen, data, and command."

When you start a project, the first question is not “What prompt should I use?” There’s an earlier one: **What am I actually building?**

What happens if you ask the AI before answering that question? The AI produces something quickly, without direction. The filenames look plausible and the folder structure looks tidy, but the pieces don’t fit together.

What about handing over too much at once? Ship the whole app in one prompt and the AI can’t keep it all in its head. Chop it into pieces that are too small and the context disappears. The AI ends up writing one part with no memory of the part before it.

Mandai was a good project for finding that balance. Mandal-Art, GTD, Pomodoro, and AI chat all look like independent features. But inside a real product they needed to flow as a single goal-management loop. This chapter walks through the process of designing Mandai while searching for the chunk size the AI could digest in one bite.

There is almost no code in this chapter. Instead it follows the order in which Mandai’s skeleton took shape. We look at **what I sent, what the AI replied, and how that reply changed the design**. What you should walk away with is not a finished folder layout. It’s a feel for the questions you need to ask when you trade design back and forth with the AI.

A cooking analogy: tell the AI “make me dinner” and it picks the menu, ingredients, and cooking order on its own. Keep saying “dice the onion to 1cm” and you lose sight of the whole dish. A good chunk is more like “We’re making curry —first, propose the ingredient list and the cooking order.” There’s a clear direction, and the work in front of you is small enough to handle in one pass. Designing Mandai worked the same way.

What This Chapter Covers

We start by shaping the app called Mandai, then steadily narrow the unit of work we can hand to the AI.

- What Mandai was at the start
- The first mistake —“build the whole thing in one shot”
- Sizing —the chunk the AI can handle at once
- Turning the AI’s counter-questions into a design tool
- Three layers: screen, data, command

- The AI chunk-size rules
- When the plan changes mid-build

Learning Goals

- Get a feel for how big a job the AI can take in one bite, and where the limit is.
- Learn to use the AI's clarifying questions as a tool for making design decisions.
- Pick up the strategy of splitting an app into three layers: screen, data, and command.
- Walk through the early Mandai design process and see how I structured the app with the AI.

6.1 What Mandai Was at the Start

Mandai bundles four features into one productivity tool. Listed out, it sounds like a familiar app. The point, though, isn't to use the four features separately—it's to connect them as a single flow. Goals, today's tasks, focused time, and AI chat all stay linked instead of sitting in their own silos.

- **Mandal-Art (9×9 goal grid):** one central goal with eight sub-goals around it. Each sub-goal expands into its own 9-cell grid. $9 \times 9 = 81$ cells total.
- **GTD:** today's task list. Each task is bound to one of the 81 cells.
- **Pomodoro:** 25 minutes of focus, 5 of rest. Finished sessions get recorded in DuckDB.
- **AI API integration:** one chat window that switches between OpenAI, Claude, and Gemini. Streaming supported.

The reason for fusing the four is simple: **a Pomodoro session only matters if it's tied to one of my 81 cells.** Logging focus time alone isn't enough. The record needs to show which goal and which area of that goal I spent time on, so I can look back later and see where my energy actually went.

Existing apps don't make that link. Todoist has no goal tree, Forest has no tasks, and Notion doesn't bake Pomodoro into the default flow.

So I built it myself. That decision became the yardstick for every later design choice. Mandai isn't a simple timer app—it ties a goal structure to time records, and I had to keep reminding the AI of that.

6.2 The First Mistake —“Build the Whole Thing in One Shot”

My first prompt looked like this.

“Build me an app combining Mandal-Art, GTD, Pomodoro, and AI chat with Tauri 2 + SvelteKit 2 + Svelte 5. Use DuckDB, hook up four AI providers, and support streaming.”

Claude's reply was what you'd expect. A 30-line folder structure appeared, followed by a 300-line code block dumped straight into `lib.rs`. Half of it didn't compile, and the other half wasn't what I wanted. Mandal-Art cell IDs were chosen at random, and the Pomodoro table didn't match the task table.

I couldn't just use that output. But it wasn't useless either. It exposed exactly which gaps were sitting in my design.

There were three core problems in what the AI produced. More than the code, what matters is seeing **which decisions were made badly**.

Problem 1. UUIDs for cell IDs

The AI gave each cell a long random identifier (UUID). It also let the “which goal does this cell belong to” field stay empty. Both were wrong.

- **UUIDs throw away the position.** Meaning like “the fifth cell of the third area in the center” is gone.
- **A blank parent goal means cells can exist that belong to nothing.** The $9 \times 9 = 81$ structure isn't enforced by the data itself.

What I wanted was a human-readable ID and cells that always belonged to some goal.

Problem 2. Pomodoro sessions weren't tied to tasks

The session table the AI produced didn't include a link to "which task this session belongs to." Instead it just had a free-form note string.

That single omission breaks the whole intent of the app. The core decision —"a Pomodoro session only matters if it's tied to one of the 81 cells"—was missing. Sessions get logged, but you can't tell later which cell or goal the time went into.

Problem 3. The folder structure looked right but was hollow

Files like `models.rs`, `migrations.rs`, and `connection.rs` were auto-generated. The names looked respectable, but the migration files were full of "fill in later" comments and the database path was hard-coded. A clean-looking skeleton with no real meat inside.

The request was sloppy, and the AI filled the gaps on its own. The trouble was that the values it filled in didn't match the product's core intent. The cell ID system and the session-to-task link are central pieces of the app's structure. Get them wrong early and you'll keep returning to square one even after finishing all the screens.

Watch out: Vague requests are an invitation for the AI to fill the gaps with its own judgment and produce a plausible-looking answer. The AI doesn't know what matters. Assume the first response has at least three wrong decisions in it.

The lesson is simple. **Don't hand the AI "the whole app" at once.** Before you pass anything off, split the work yourself. An unsplit request doesn't give the AI freedom —it leaves the product's core decisions for the AI to guess.

6.3 Sizing —The Chunk the AI Can Handle at Once

The second session opened like this.

"The app plan is done. From now on I'll ask exactly one thing at a time. First, listen to the overall structure, then throw back up to 5 questions you want to ask. Don't write code."

That single line changed how Claude replied. Instead of code, Claude came back with questions. Five of them.

1. What's the scheme for Mandal-Art cell IDs? (e.g., 1–3–5 vs `center.sub3.cell5` vs UUID)
2. When a Pomodoro session ends, which task does it attach to —picked at start, or selected at the end?
3. Is AI chat history per task, per cell, or per app?
4. Does it work offline? If so, how do AI calls behave?
5. Where do settings (API keys, etc.) live —`keyring`, a regular file, or DuckDB?

Three out of five were things I hadn't decided yet. The AI found the gaps before I did. These were exactly the spots the AI had filled in by itself during the first session.

When Claude doesn't ask first, you can prompt it directly: "Don't write code. Find three things in this design that I haven't decided yet." That one sentence shifts Claude from writing code to hunting design gaps.

Cell ID scheme —settled with two follow-ups

The first thing we narrowed down was the cell ID scheme. I answered: "Cell IDs follow {center_idx}-{sub_idx}-{cell_idx}. They're strings. $9 \times 9 = 81$ of them."

Claude asked back: "What about the center itself (center)? Is it a virtual cell like 0–0–0, or a separate table?"

I had missed that. The central goal isn't a cell, but I was subconsciously treating it like one. "The center is a separate goal table. Cells reference it through a goal_id FK."

Claude asked again: "If the goal is separate, what about multiple active goals? Does a cell belong to exactly one goal, or can it span several?"

One answer, and a new question appeared. That's the repeating pattern. Every time I answered, two or three new questions came back. The AI keeps hunting for design gaps.

My answer: "Only one goal is active at a time. A cell belongs to exactly one goal via goal_id FK."

Two exchanges nailed down the cell schema. A more substantial result than the 300 lines Claude wrote on its own in the first session.

Pomodoro session —following the analytics scenario

Next we settled which task a Pomodoro session attaches to. I answered: “The task is picked when the Pomodoro starts.”

Claude asked back: “What if you run a 25-minute session with no task —handling email, for instance?”

Fair point. “Then the task is optional. `task_id` is nullable, and a `note` column takes its place.”

Claude pushed further:

“If `task_id` is nullable, how does analytics handle it? Do you want to see ‘three hours on email this week’ and ‘three hours on untracked work this week’ separately?”

I hadn’t thought about the analytics case.

My answer: “Yes, notes should be searchable. Add a category enum: ‘task’, ‘email’, ‘meeting’, ‘other’.”

That one decision later shaped the entire layout of the session history screen. Because the AI pushed back, I made a future decision now instead of later.

AI chat —being offered the options

The third question was the scope of AI chat. I first answered: “AI history is flat across the whole app.”

Claude didn’t just accept that. It threw a scenario at me:

“Say you asked ‘how do I solve this?’ from cell 2–3–4. Later, you’re on cell 5–1–2 and you see the same question in your history. Now the context is wrong.

Options: A) Per-cell history (1 thread per cell = 81 threads) B) Per-goal history (1 thread per goal) C) App-wide history with context tags (`#cell:2-3-4`, `#task:write-tests`) D) Hybrid: app-wide but with context filters”

Claude offered options I hadn’t thought of. Option C was the answer. Flat storage, rich tagging, filterable views.

That one loop defined the entire chat database schema. Three tables: `messages`, `contexts`, and a junction table `message_contexts`.

Watch out: The first data structure that comes to mind is almost always wrong. The AI catches the use cases you missed. Push “and then what?” at least three times before locking in a schema.

6.4 Turning the AI's Counter-Questions Into a Design Tool

The three loops above all share something. I answer, Claude asks again, and that question exposes a gap in the design. You can produce that flow on purpose.

The template I use looks like this.

“Audit this design for holes. Follow these rules: 1. Don't write any code. 2. Find three places where I'm avoiding a decision. 3. For each one, suggest two or three alternatives.”

With this template, Claude turns into an alternative generator. It lists choices I didn't know I had, I pick one, and I write down why in my notes. Those notes become the architecture document.

Here's how it actually played out. I gave Claude this design.

“Mandai has five screens: Today, Archive, Goals, Settings, Chat. The Today screen shows the 81 cells of the active goal as a 9×9 grid. Clicking a cell reveals its tasks; clicking a task starts a Pomodoro.”

The audit produced three places I hadn't decided.

Gap 1: Navigation state – Option A: Browser-style routing (/today, /archive, etc.) – Option B: Tab-based navigation (SvelteKit stores) – Option C: Modal overlays (one main screen, others as popups)

Gap 2: Cell interaction model – Option A: Single click to open, double click to edit – Option B: Right-click context menu for every action – Option C: Action buttons appear on hover

Gap 3: Timer state persistence – Option A: Timer resets when the app closes (session-only) – Option B: Timer survives restart (saved to DB) – Option C: Timer warns before app close (user choice)

I hadn't decided any of these. What looked like a design was actually a draft full of unspoken assumptions.

My decisions: 1. **Navigation:** Browser-style routing. The back button works, deep links work, and it feels like an app. 2. **Cell interaction:** Single click to open, right click for the context menu. First-time users find it instantly, and power users get an efficient shortcut. 3. **Timer restoration:** When the app reopens, the user can pick up the session in progress. But if they try to close the window mid-session, warn them first. A 25-minute focus log shouldn't disappear just because someone closed the app by accident.

I ran this audit on Mandai roughly ten times. The decision list that came out looks something like this.

- Cell ID: {center_idx}-{sub_idx}-{cell_idx} string
- The goal table is separate from the cell table
- Pomodoro task_id is nullable, with a category enum
- AI history is app-wide with context tags (#cell, #task, #goal)
- Navigation: SvelteKit routing (/today, /goals, etc.)
- Cell click: single click to open, right click for context menu
- Timer restoration: DB-backed; reopening the app continues the session in progress
- Offline: failed AI calls save to a local draft; on reconnect, offer retry
- API keys: OS keyring (not tauri-plugin-store)
- Migrations: a _migrations table in DuckDB, checked at app start

I dropped this list **into Cursor's .cursorrules**. Since then, Claude has not written code that violates these decisions.

A single audit takes roughly ten minutes per major design area. In return, it saves hours later. Design holes you catch in session 2 don't turn into bugs in session 20.

6.5 Three Layers: Screen, Data, Command

When I ask the AI for a feature, I use a three-layer template.

1. **Screen** —what the user sees and clicks.
2. **Data** —DB tables and TypeScript types.

3. **Command**—Rust functions (`#[tauri::command]`) that connect screen to data.

Ask for all three at once and the AI loses focus. That’s why the order matters. **First lock down the screen, then pull the data it needs out of that screen, then list the commands that handle that data.**

Here’s how the pattern played out on the Today screen.

Layer 1: Define the screen

The layout went out as an ASCII mockup instead of code. You can draw a mockup in text without any drawing tools, and the AI doesn’t get confused. The mockup below is exactly what I sent to Claude.

```
+-----+
| MANDAI                                [Timer] 18:42 |
+-----+-----+
| 9x9 grid          | Tasks for selected cell |
|                   |                         |
| [Done][Wait][Run] | [ ] Task 1      0/3      |
| [Wait][Done][Wt]  | [ ] Task 2      2/3      |
|                   | [Done] Task 3    3/3      |
|                   | [+ New task]           |
+-----+-----+
```

The interaction rules went in as text alongside the mockup.

- Click a cell -> show its tasks below
- Click a task -> start a Pomodoro
- During a session, the timer shows in the top right
- Progress bars show each cell’s task completion

Looking at the mockup, Claude confirmed two things.

“1. What do we show when no cell is selected—all 81 cells’ tasks, or an empty view? 2. How does the task progress bar update—live during a session, or only after a session ends?”

These were undecided. My answer: “No cell selected = empty task list with a hint, ‘Click a cell to see its tasks.’ Progress bars update only after a session completes.”

Layer 2: Data requirements

Once the screen was locked, Claude pulled out the data it needed:

“What this screen needs: - Active goal: `goal_id`, `title` - All 81 cells of that goal: `cell_id`, `title`, `task_progress` (completed/ total) - Tasks of the selected cell: `task_id`, `title`, `completed`, `cell_id` - Active session: `session_id`, `task_id`, `started_at`, `target_minutes` - Session counts: today’s count, this week’s count”

I changed one thing: “Don’t pre-compute `task_progress`. The DB returns raw counts only: `completed_tasks`, `total_tasks`. The frontend computes the ratio.”If the database decides the display format, every screen change forces me to rewrite the query.

Claude asked a follow-up: “Do completed tasks count toward the progress bar, or only active ones? And what’s the definition of ‘this week’—Monday-start or rolling 7 days?”

Another edge case I’d missed. “All tasks count toward progress, whether completed or not. ‘This week’is Monday 00:00 to Sunday 23:59 in the user’s time zone.”

Layer 3: Enumerate the commands

With the screen and data settled, the Tauri command list fell out naturally. There’s one command per type of data the screen needs.

Claude proposed six. The names alone tell you what each one fetches.

Command	Returns	Purpose
<code>get_active_goal</code>	<code>Goal</code>	The single currently active goal
<code>get_goal_cells(goal_id)</code>	<code>Vec<CellWithProgress></code>	The 81 cells’text and progress counts
<code>get_cell_tasks(cell_id)</code>	<code>Vec<Task></code>	Tasks in the selected cell
<code>get_active_session</code>	<code>Option<PomodoroSession></code>	The session in progress, if any
<code>get_session_counts</code>	<code>SessionCounts</code>	Session counts for today/this week
<code>start_pomodoro(task_id, minutes)</code>	<code>String</code>	Start a new session

Only `get_session_counts` got a signature tweak. If it only accepts fixed time buckets like today/this week, you can't add ranges like "last 30 days" later without changing the signature. I switched it to take a time range as an enum (a type that bundles several choices into one).

The idea is simple. The caller says which range it wants as a value, and the command counts sessions in that range and returns the result. The enum has three options: `Today`, `ThisWeek`, and `Custom` (where the caller passes a start and end date). **Adding a new variant to the enum doesn't change the command's signature.** If a "last 30 days" screen comes along later, I add an enum variant—the command stays untouched.

Specify each command as an input/output/error contract. For example, `create_goal` accepts a title and returns `id`, `title`, and `created_at`, distinguishing an empty title from a database failure. Serialize states as fixed values such as "running", "paused", and "completed", with matching Rust and TypeScript definitions. Using translated display labels as state values breaks the contract when language changes.

That's the pattern: **screen -> data -> command**. Each layer comes out of the one before it. The AI works well when it handles one layer at a time. Give it all three at once and the output is a smear.

Six commands means **six Request-Build-Verify loops**. Up to 30 minutes per command. One screen = half a day.

Watch out: Bouncing between layers confuses the AI. Ask one prompt about screen layout and DB schema and you'll get an answer that's half UI framework, half SQL. One layer per conversation.

6.6 The AI Chunk-Size Rules

Pulling the experience together, I landed on three rules.

Rule 1. One screen at a time, or one command at a time. Whole app >> one feature >> one screen >> one command. The further right you go, the easier it is to hand off to the AI.

Rule 2. Ask for questions before you ask for code. The first five to ten minutes go to letting Claude ask me things. That surfaces the gaps in the design. The time you spend there comes back, doubled, as time you don't spend debugging.

Rule 3. I make the decisions, and I record why in `.cursorrules`. The AI is good at generating alternatives but can't decide which to use. I pick, and I write down why. That keeps Claude from wavering on the same question in the next session.

Bigger chunks fail faster. After six projects, my rough sense of the limits looks like this.

- **Whole app (around 1000 lines, 3-4 hours):** nearly always fails. Don't bother.
- **Feature-level (e.g., "all of Pomodoro," 1-2 hours):** only useful for early exploration.
- **One screen (30-45 minutes):** the standard unit of work. The one I use most.
- **One command (5-15 minutes):** split here when the screen-level chunk stalls.
- **One function (2-5 minutes):** only when I'm debugging a specific piece of logic.

When in doubt, I ask myself: **can I describe this chunk in one sentence? Can I test it independently? Will I still remember the requirements after a coffee break? Does it require just one decision?** If any answer is "no," the chunk is too big.

On the other side, if the AI repeats the same question for every chunk or two chunks share most of their logic, the chunks are too small. Merge them.

The right chunk size is **the largest one that produces working code in one pass**. It varies by domain and model, but it's usually smaller than you'd expect.

6.7 When the Plan Changes Mid-Build

Sections 6.2-6.6 assume the design holds steady from start to finish. In practice, you often find new requirements while actually building. The AI surfaces edge cases too. The first plan getting bent out of shape is unavoidable.

It happened twice while I was building Mandai. Here's how I changed the design in order, while keeping development moving.

Pivot 1: cell ID collisions

Three weeks into the build, I hit a problem with the cell ID scheme. The original plan had cell IDs in the form `{center_idx}-{sub_idx}-{cell_idx}`, with each index between 1 and 9. (The final implementation switches to 0-8 —see the schema in Chapter 7.)

The problem: While implementing goal switching, I realized a user could create multiple goals that all contained a cell like 1–3–5. Cell ID values weren’t unique across goals, so tasks under “goal A’s cell 1-3-5” mixed with tasks under “goal B’s cell 1-3-5.”

I could have rewritten the whole schema, but that meant redoing 15 DB commands from scratch. Instead I asked the AI what my options were.

“I have a design problem. Cell IDs like ‘1–3–5’ collide across goals. I need unique cell references without altering the existing task and session tables. What are the options?”

Claude proposed three:

- A) **Prefix with goal_id:** `goal_uuid_1–3–5` (unique, but ugly)
- B) **Composite key:** keep `cell_id` as ‘1–3–5’ and always query as (`goal_id`, `cell_id`) together
- C) **Hierarchical ID:** `{goal_idx}–{center_idx}–{sub_idx}–{cell_idx}`

Option B didn’t touch table structure; it only added a condition to queries. To make sure later work didn’t break this, I added four lines to `.cursorrules`.

- Cell IDs stay as strings in the form ‘1–3–5’.
- Cell IDs are unique only within a single goal.
- Every cell query must include a `goal_id` condition.
- The composite index is on (`goal_id`, `cell_id`).

These four lines tie down every future cell query. When Claude is building a new screen and writes `WHERE cell_id = ?` alone, that violates the rule, and the rules file catches it.

The migration took two hours instead of two days. The win came from keeping the change small.

Pivot 2: a real-time timer

In the original plan, the Pomodoro timer ran inside the browser. Close the app and the session went with it.

The problem: Users kept closing the app mid-session by accident. A 25-minute session vanished with a single `Cmd+W`. For a productivity app, that’s hard to accept.

The usual fix is a background service that survives the app’s shutdown. But Tauri 2 doesn’t support background services the same way on every platform.

If you phrase the requirement as “the timer must keep running after the app closes,” the design direction is blurry. The more accurate phrasing is closer to “when the app reopens, restore the session that was in progress.” The point isn’t that the timer ticks on its own—it’s that the start time gets saved so the next launch can compute the remaining time.

Claude proposed a pattern that uses the database not as plain storage but as **a way to restore state**. The flow:

1. On session start, write a session record with a `started_at` timestamp.
2. Update `last_heartbeat` every 30 seconds.
3. On app restart, check whether any session has `ended_at IS NULL`.
4. If there is one, show a dialog: “A session was in progress. Continue (X minutes left) / Mark as complete / Discard.”
5. If the user picks “Continue,” compute remaining time as `now - started_at` and resume the timer.

The pattern works for a simple reason. Instead of saving the remaining time every second, you save **only the start time, once**. The time the app spent closed gets covered by a single `now - started_at` calculation. Session continuity is preserved without a background service.

The implementation took roughly an evening. The restore dialog asking “continue the session in progress?” landed well with users. More than just an added feature, it gave them trust that a closed window wouldn’t break their flow.

When pivots happen too often

Two pivots in a project is normal. Five is a signal that the original design is fundamentally off. Here’s how to tell the two apart.

Normal pivots (keep building): – The problem comes from one specific constraint (ID collisions, persistence requirements, etc.) – The blast radius is narrow and the fix direction is clear – The AI proposes two or three concrete solutions – The fix actually improves the overall design

Pivots that are warning signs (restart planning): – Several problems tangle together – The AI replies with more clarifying questions than answers – A “simple” fix touches five or more DB tables – You find yourself explaining the same core concept again and again

If you see the warning-sign pattern, **go back to section 6.4 (design audit)**. Run the hole-finding template against the current state. Repeated pivots usually mean there's a deeper architecture problem underneath.

6.8 Summary

The thing to take away from this chapter is simple. **Don't hand the AI too much at once. Ask for questions before you ask for code. You make the decisions, and you write down the reasons.**

Key idea: **using the AI lowers the cost of pivoting. It doesn't eliminate it.** Pivot early and often—but stay alert for the case where the design itself is wrong.

That agility comes from picking the right chunk size. The AI can build a chunk of any size, but the chunk size decides the quality of what comes back. Too big, and it invents constraints that aren't there. Too small, and context disappears.

What we learned:

- **The AI's wrong guesses reveal the gaps in your design.** Every spot Claude filled in arbitrarily during the “build it all” prompt was a requirement I'd left unspecified.
- **Early on, questions are worth more than code.** A single prompt —“ask me five questions, don't write code”—turns Claude from a code generator into a design completeness checker.
- **The AI's counter-questions surface edge cases.** Every response brings two or three new questions. That's not a bug; it's the AI pointing at scenarios you never spelled out.
- **The architecture audit template finds deferred decisions.** Run “find three places where I'm avoiding a decision,” and the AI points to them and lays out the options. You pick.
- **Layering keeps the AI's responses focused.** Separate screen -> data -> command into separate conversations so the AI handles one thing at a time.
- **Chunk-size baseline: one screen = four to six commands = half a day.** That's the unit where you keep momentum and can still change direction.
- **Small pivots are normal.** In AI-assisted development, course corrections aren't a sign of failure—they're an expected part of the flow. But if small pivots keep landing on the same spot, that's a sign of a structural issue beneath the surface.

- **Documenting decisions keeps things steady.** Write design choices and their reasons into `.cursorrules`. The AI stays consistent across sessions, and you don't have the same discussion twice.

Next: We now have a system for breaking large features into chunks the AI can digest. We've also picked up how to use the AI's counter-questions as a design tool and how to change plans without losing momentum. In Chapter 7 we apply this process to actual DuckDB schema design —the first complete Request-Build-Verify loop.

Chapter 7. The Database Schema —Your First Request · Build · Verify Loop

Database design is usually the first place you get stuck when building with AI. It isn't because the AI can't write SQL. A simple `CREATE TABLE` comes out fine on the first try. The hard part is somewhere else: the schema (table structure) you settle on in this first loop becomes the foundation for every feature that comes after. If you get the relationships between tables wrong here, you'll end up fixing the screens, the command-handling code, the migrations (the work of moving existing data into a new structure), and the analytics queries all at once.

Mandai's schema only settled down after three rounds of revisions. The first version wasn't obviously wrong. It compiled, it ran. The trouble only showed up after I had added three more features on top. What matters in this chapter is not just the final schema but the process: what I asked the AI for, and what I noticed while reviewing the answer. The goal is to put you in a position to run the same loop on your own project.

Why this chapter is told as “a story about loops”

Before diving in, here’s a one-page view of where each of the three loops ended and where another loop was needed. When the body of the chapter starts to feel abstract, come back to this table.

Round	What I asked for	What Claude produced	What I caught during review
1	Draft schema for seven tables	Basic CREATE statements	Missing session end time, cell ID validation, soft-delete strategy
2	Fill in those three gaps + indexes	Constraints and composite indexes added	_migrations had no uniqueness guarantee
3	A Rust migration runner	Worked on the first try	None

Schema design takes two or three loops; the boilerplate pieces (the migration runner, the query helpers, the type layer) finish in one loop. That distinction is the main message of this chapter.

How much of this should you type yourself?

On the first read, follow schema decisions and failure handling. A migration checks the version, runs unapplied SQL, and records successful application. Distinguish JSON null, numbers, strings, and booleans when passing parameters; reject unsupported values. The examples explain this contract rather than provide a procedure for copying an entire commercial app.

How the chapter flows

1. **Why DuckDB** —why I picked a database that’s strong at analytical queries
2. **The first loop** —get a draft schema and figure out what decisions are missing
3. **The second loop** —fill in session end times, cell ID validation, and soft delete (a way to mark data as deleted without actually removing it)
4. **The third loop** —finish the migration system in a single pass
5. **Verification** —use AI-generated tests to confirm the constraints actually fire

6. **Wiring** —connect a Rust command and a TypeScript type layer to the frontend
7. **The loop pattern** —sort out which tasks finish in one loop and which need two or three

Learning goals

After this chapter you'll be able to:

- Explain how DuckDB differs from SQLite and why you should spell out that difference in your AI prompt.
 - Apply the loop where you don't try to nail the schema in one shot, but fill in the missing decisions step by step.
 - Design CHECK constraints (rules that stop invalid values from being written), foreign keys, and indexes to match your query patterns.
 - Wire Tauri commands to a TypeScript type layer for type-safe database access.
-

7.1 Why I picked DuckDB

Look at the relationships first —seven tables

Before column names, look at how the tables relate. The core flow in Mandai is this:

goals — 1:N — cells (81 squares) — 1:N — tasks — 1:N — sessions
(Pomodoro)

|
chats (AI conversation) — attaches to a goal/cell as
context
settings (key-value) — app-wide settings
_migrations (version tracking)

A single goal (goals) has $9 \times 9 = 81$ cells (cells) under it. Each cell holds today's tasks (tasks), and each task accumulates Pomodoro sessions (sessions). AI chats (chats) attach to all of this as context. Once you have this picture in your head, the need for foreign keys and indexes becomes obvious, and it becomes clear why you can't make these decisions by looking at one table in isolation.

Why DuckDB

SQLite seems like enough at first. If all you’re doing is storing data, SQLite is more familiar and has more documentation. But over time Mandai starts to ask analytical questions: “Which cell got the most focus time in the last three months?”“What’s the median focus time per weekday?”These aren’t data-storage questions, they’re data-analysis ones. DuckDB is built for the analysis side.

DuckDB is a columnar engine. Where a regular database stores data row by row, DuckDB stores it column by column. That makes aggregate queries —sums, averages, anything that sweeps across an entire column —fast. The SQL syntax isn’t far from SQLite, so there isn’t much new to learn.

The reason the database chapter feels harder than it looks is something else entirely: how you design the tables becomes the policy of the app. Should `task_id` be allowed to be empty? When you “delete”something, do you actually remove it, or just mark it deleted? Choices like these ripple through the screens, the analytics queries, and the migrations. It isn’t just personal preference.

DuckDB vs SQLite —where AI tends to slip

Watch out: Most AI models default to SQLite syntax. If you don’t spell out the DuckDB-specific features you want, you’ll get plain SQL that doesn’t take advantage of DuckDB’s strengths.

The first time I asked Claude for a “database schema,”the answer leaned toward SQLite. The types and functions that are useful for analysis weren’t there.

Area	DuckDB	SQLite
Timestamps	TIMESTAMP	DATETIME
Large integers	BIGINT	INTEGER
Analytical functions	<code>percentile_cont()</code> , <code>stddev()</code> built in	Usually have to write your own
Arrays	Native support	None

Just adding a single line to the prompt —“Use DuckDB syntax, not SQLite patterns”—changes the types and the example queries the AI picks. Feeling that shift firsthand is one of the most practical takeaways of this chapter.

7.2 The first loop —drafting the request

Before asking Claude for a schema, I needed to decide how Tauri would connect to DuckDB. Whether each command opens a new connection or the whole app shares one changes how error handling looks.

Add the dependency to `src-tauri/Cargo.toml`. The bundled feature compiles DuckDB into the app itself, so users don’t have to install anything separately.

```
[dependencies]
duckdb = { version = "1.5", features = ["bundled"] }
```

Open the connection once and share it

I open one connection for the whole app and share it. The code below makes a single DuckDB connection at startup, keeps it in global state, and then runs migrations.

```
static DB: OnceLock<Mutex<Connection>> = OnceLock::new();

pub fn init_db() -> Result<(), String> {
    let conn = Connection::open_in_memory()
        .map_err(|e| format!("DB connection failed: {}", e))?;
    DB.set(Mutex::new(conn)).map_err(|_| "DB already
initialized".to_string())?;
    run_migrations().map_err(|e| format!("migration failed: {}", e))?;
    Ok(())
}
```

`OnceLock` is a Rust type that holds a value initialized exactly once across the whole app, and `Mutex` is a lock that lets only one thing access the value at a time. If you open a new connection

for every command, it gets slow; if several connections touch the same file at the same time, lifetime management gets messy. So the app opens a single connection at startup and every command borrows it briefly afterward.

Difference from Chapter 5: Chapter 5 recommended `RwLock`. We use `Mutex` here because the DuckDB connection itself doesn't allow concurrent access. A good rule of thumb: `RwLock` for ordinary data structures, `Mutex` for resources that genuinely allow only one access at a time.

Watch out: In a real shipped app you should not use `Connection::open_in_memory()`. The data disappears the moment the app closes. You need to point at a real file path and handle the case where the file doesn't exist yet on first launch.

Sending the schema request

With the connection style decided, I asked Claude for the schema. The prompt didn't just list table names—it described **how the app uses the data**. If you only throw table names at the AI, it can't tell whether each table is just for storage or for analysis, so it picks columns for “standard CRUD.” Writing out the usage flow lets it think about foreign keys, timestamps, and index candidates up front.

Write me a DuckDB schema for the Mandai app. Seven tables.

- goals: top-level goals
- cells: 9×9 = 81 squares, FK to goal
- tasks: today's tasks, FK to cell (optional)
- sessions: Pomodoro sessions, FK to task (optional)
- chats: AI chat messages
- settings: key-value
- _migrations: migration tracking

Explicit foreign keys. DuckDB syntax. CREATE TABLE only — no INSERT examples.

Context: This is a productivity app. A goal has a 9x9 grid of cells (like a Mandala chart).

The user adds tasks to cells, runs Pomodoro sessions on tasks, and chats with an AI

about progress. Analytical queries need to aggregate sessions by time range and compute focus-time ratios.

Reviewing the draft —what’s missing

In Claude’s draft, the `goals` and `cells` tables came with foreign keys, `NOT NULL` constraints, and sensible defaults right out of the box. That part was good. But three missing decisions showed up during review.

1. **`sessions` has `start_at` but no `end_at`.** There’s no way to compute total focus time, and if the timer ends mid-way, the session stays open forever.
2. **`cells.id` has no rule for how it’s formed.** In Chapter 6 we decided the format would be `{center_idx}-{sub_idx}-{cell_idx}` (for example `0-1-4`), but the schema doesn’t enforce that.
3. **The soft-delete strategy isn’t consistent.** `goals` has a `deleted_at` column but `tasks` doesn’t. If I delete a task, what happens to the sessions connected to it?

All three are decisions I never finished making in Chapter 6. I had forgotten, and Claude didn’t catch it either. **Decisions you missed earlier surface during the review. That’s exactly what the first loop is for.**

7.3 The second loop —filling the gaps

Quick reminder of the rule from Chapter 6: A cell ID has the form `{center}-{sub}-{cell}`, and each index is between 0 and 8. Example: `0-1-4` means center area 0, sub-area 1 inside it, cell 4 inside that. Translating this rule into a database constraint is the heart of this section.

For the second loop I didn’t ask for the draft to be rewritten from scratch. I kept the parts that were good and pointed at exactly the three missing decisions.

Three fixes needed:

1. Add ``end_at` TIMESTAMP NULL`` and ``ended_reason` TEXT`` (completed / skipped / interrupted) to ``sessions``. `NULL end_at` means the session is still running.
2. Add a `CHECK` constraint on ``cells.id``: the format is "0-1-4", i.e. ``{center}-{sub}-{cell}``.
3. Standardize soft delete as ``deleted_at` TIMESTAMP NULL``. Only on ``tasks`` and ``goals``.
Sessions are historical data so they stay; cells are never deleted (they're structural).

Rewrite only those three tables. Also suggest indexes that match the query patterns.

If I had just said “add soft delete,” Claude would have picked the approach itself. You have to spell out the exact approach and the reasoning to get the AI to follow it.

The key tables, revised

The heart of the revision is the `CHECK` constraints that landed on `sessions` and `cells`. They make sure bad data **cannot enter the database in the first place**.

```
CREATE TABLE cells (
  id TEXT PRIMARY KEY,
  goal_id INTEGER NOT NULL,
  center_idx INTEGER NOT NULL,
  sub_idx INTEGER NOT NULL,
  cell_idx INTEGER NOT NULL,
  FOREIGN KEY (goal_id) REFERENCES goals(id),
  CHECK (id = printf('%d-%d-%d', center_idx, sub_idx, cell_idx)),
  CHECK (center_idx BETWEEN 0 AND 8),
  CHECK (sub_idx BETWEEN 0 AND 8),
  CHECK (cell_idx BETWEEN 0 AND 8)
);

CREATE TABLE sessions (
  id INTEGER PRIMARY KEY,
  task_id INTEGER NULL,
  start_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  end_at TIMESTAMP NULL,
```

```

ended_reason TEXT NULL,
target_minutes INTEGER NOT NULL DEFAULT 25,
FOREIGN KEY (task_id) REFERENCES tasks(id),
CHECK (ended_reason IS NULL OR ended_reason IN
('completed', 'skipped', 'interrupted')),
CHECK (end_at IS NULL OR end_at >= start_at),
CHECK (target_minutes > 0)
);

```

The lines that matter: the CHECK constraints at the bottom. They keep cell indexes inside the 0-8 range, prevent a session from ending before it started, and force the target minutes to be positive. Claude added safeguards I hadn't even asked for.

Indexes —start from the queries you'll run often

Indexes work best when you list the queries you'll run most often first and build them to match. Here are indexes shaped around the three most common queries in Mandai.

```

CREATE INDEX idx_cells_goal ON cells(goal_id);
CREATE INDEX idx_sessions_task_start ON sessions(task_id, start_at
DESC);
CREATE INDEX idx_sessions_completed ON sessions(ended_reason,
start_at);

```

If I had been working alone I'd have made one index per column. Claude thought about the actual query patterns first and built composite indexes instead.

Watch out: You don't need a lot of indexes up front. Each additional index slows down writes and uses more disk space. DuckDB doesn't support some index options (partial indexes that target only certain rows, for example), so you'll handle that kind of filtering in your WHERE clauses instead.

The second review also revealed one more problem. The `_migrations` table doesn't guarantee execution order, so if the migration runner runs twice there's nothing stopping a duplicate insert.

7.4 The third loop —the migration system

The third loop isn't about the schema itself; it's about the system that runs it. **What is a migration runner?** It's code that manages “which SQL to run, in what order, and only once.” When the app updates and the schema changes, the runner automatically brings each existing user's database up to the new structure.

The request was simple: “A function that takes a DuckDB Connection, runs only the migrations that haven't been applied yet in order, and records what it applied in `_migrations`. Core logic in ten lines or fewer.”

The result **worked on the first try**.

```
pub fn run_migrations(conn: &Connection) -> duckdb::Result<()> {
    conn.execute_batch(include_str!("../migrations/000_meta.sql"))?;

    const MIGRATIONS: &[(&str, &str)] = &[
        ("001", include_str!("../migrations/001_init.sql")),
        ("002", include_str!("../migrations/002_add_indexes.sql")),
    ];

    for (version, sql) in MIGRATIONS {
        let applied: i64 = conn.query_row(
            "SELECT COUNT(*) FROM _migrations WHERE version = ?",
            [version], |row| row.get(0))?;
        if applied == 0 {
            conn.execute_batch(sql)?;
            conn.execute(
                "INSERT INTO _migrations(version, applied_at) VALUES
                (?, CURRENT_TIMESTAMP)",
                [version])?;
        }
    }
    Ok(())
}
```

Two lines worth noticing: - `include_str!("...")`: a macro that drops the SQL file directly into the binary at compile time. There's no separate file read at runtime. - `if applied == 0`: the guarantee that “versions already applied get skipped.”

This kind of code follows a fixed pattern, so Claude gets it right in one shot. **It’s the kind of work that’s ideal to hand off in a single request.** On the first read you don’t need to break this function down line by line —just remember that it follows the three-step shape “check version -> run unapplied SQL -> record it.”

Watch out: This approach requires a rebuild every time you add a migration. That’s fine for a desktop app, but in a server environment loading the files dynamically would be a better fit.

migrations/000_meta.sql holds only the _migrations table definition, and migrations/001_init.sql holds every table we built above.

From here, sections 7.5 through 7.8 cover **what comes after the schema**. We verify the constraints with tests (7.5), expose database access through Rust commands (7.6), gather the common query patterns (7.7), and finally write a thin helper on the TypeScript side that receives results type-safely (7.8). Each one is short.

7.5 Testing the schema —using AI to check quality

Before moving on to the query layer, I made sure the schema actually behaved. I asked the AI to “write test INSERTs focused on edge cases and constraint violations.” The key is cases like the ones below, where **an error is the correct outcome**.

```
-- Should fail: invalid cell ID format
INSERT INTO cells (id, goal_id, center_idx, sub_idx, cell_idx, text)
  VALUES ('invalid', 1, 0, 0, 0, 'bad ID');

-- Should fail: cell index out of range
INSERT INTO cells (id, goal_id, center_idx, sub_idx, cell_idx, text)
  VALUES ('9-0-0', 1, 9, 0, 0, 'out of range');

-- Should fail: end time earlier than start time
INSERT INTO sessions (task_id, start_at, end_at) VALUES
  (1, '2026-04-17 10:00:00', '2026-04-17 09:30:00');
```

All three should raise something like `CHECK constraint failed`. If any one of them succeeds, the corresponding CHECK constraint isn't doing its job.

Test successful creation, duplicates, invalid references, migration reruns, and partial failure. One illustrative deletion policy rejects goals with child cells unless the user explicitly requests deleting the children too, then does so in one transaction. On startup, mark interrupted running sessions as needing recovery rather than inventing completion times. Confirm the actual policies in requirements; these examples show the decision order.

7.6 A command that worked in one pass — `db_query_json`

As soon as the schema was settled I wired up a Rust command. This command runs a SELECT query sent from the frontend and returns the result as a JSON array string. The signature looks like this:

```
#[tauri::command]
pub fn db_query_json(sql: String, params: Vec<serde_json::Value>) ->
    Result<String, String>
```

Convert JSON parameters by type: null to SQL NULL, boolean to BOOL, integer to an integer type, finite floating point to a real type, and string to TEXT. Serialize arrays and objects only for columns explicitly designed for JSON. Test integers before floats to avoid losing large-integer precision. Bind values instead of concatenating SQL. Table and column identifiers require an allowlist because value binding does not protect them.

A write-only command, `db_execute`, has a similar signature; it calls `execute` instead of `query_row` and returns the number of affected rows.

Watch out: A general-purpose query command is as dangerous as it is powerful. The frontend can run any SQL, including DELETE and DROP. In production you must split this into separate read and write commands.

7.7 DuckDB query patterns for Mandai

With the schema and the command in place I started writing the actual queries. On a first read, just looking at the one example below —where DuckDB’s strength shows up in a single line — is enough.

```
SELECT
  date_trunc('month', start_at) as month,
  COUNT(*) as total_sessions,
  COUNT(*) FILTER (WHERE ended_reason = 'completed') as completed
FROM sessions
WHERE start_at >= '2026-01-01'
GROUP BY month
ORDER BY month;
```

The line that matters: `COUNT(*) FILTER (WHERE ...)`. To get the same result in SQLite you’d write a long `CASE WHEN` expression. DuckDB lets you write the same conditional aggregate short and readable.

Define the aggregation unit first. To find median focus time by weekday, select completed sessions, derive the weekday in the chosen timezone, group durations, and apply `percentile_cont(0.5)`. Define completion rate as `completed/started` sessions, leaving it undefined when the denominator is zero. State whether date boundaries use UTC or local time so other screens can reconcile totals. Chapter 8 moves to AI API integration.

A note on soft delete and joins

When you join multiple tables, just remember one thing. If you use soft deletes, every analytical query’s join condition or `WHERE` clause needs `deleted_at IS NULL`. Forget it and deleted rows start polluting your stats.

```
LEFT JOIN tasks t ON c.id = t.cell_id AND t.deleted_at IS NULL
```

7.8 The TypeScript type layer —where AI really helped

Putting a TypeScript type layer on top of the database commands was a place where the AI delivered code I barely had to touch. The layer gives the frontend three thin helpers for handling query results in a type-safe way.

```
import { invoke } from "@tauri-apps/api/core";

export async function query<T>(sql: string, params: unknown[] = []):
Promise<T[]> {
  const json = await invoke<string>("db_query_json", { sql, params });
  return JSON.parse(json) as T[];
}

export async function one<T>(sql: string, params: unknown[] = []):
Promise<T | null> {
  const rows = await query<T>(sql, params);
  return rows[0] ?? null;
}

export async function execute(sql: string, params: unknown[] = []):
Promise<number> {
  return invoke<number>("db_execute", { sql, params });
}
```

Each function does one thing: - `query<T>` —returns multiple rows as an array - `one<T>` —returns a single row (or `null` if there's none) - `execute` —write-only, returns the number of affected rows

Because each accepts a generic `<T>`, calling `query<Goal>(...)` returns `Promise<Goal[]>` automatically. The call sites stay simple.

```
interface Goal { id: number; title: string; created_at: string; }

const goals = await query<Goal>("SELECT * FROM goals WHERE deleted_at
IS NULL");
```

```
const activeGoal = await one<Goal>("SELECT * FROM goals WHERE id = ?",
[goalId]);
```

This TypeScript layer didn't need a single loop. One request, done. That's because gluing `invoke`, generics, and JSON parsing together is a fixed pattern. AI is fast and accurate at work where no judgment calls are needed. Conversely, in sections 7.2-7.4, where decisions had to be made, the AI couldn't answer on its own. Those calls had to come from the developer.

Watch out: This layer doesn't validate data at runtime. If the SQL returns columns you didn't expect, you won't get an error—you'll get undefined properties at runtime. In production, pairing this with a runtime validation library like Zod is safer.

7.9 Summing up the loops in this chapter

If you look back at the loop table from the introduction, the pattern of what finished in one loop and what needed multiple loops becomes obvious. Finished in one pass: `run_migrations()`, `db_query_json`, the TypeScript helper, the test cases. Needed two or three loops: the schema design, the constraint spec, the index strategy.

This pattern repeated all over Mandai. When the code has a fixed shape and clear boundaries, the AI finishes it in one shot. When design decisions are mixed in, you need two or three loops. Recognizing this distinction lets you predict, before you even send the prompt, how many turns of conversation you'll need.

Sorting database work by loop count

- **Schema design** = 2-3 loops (lots of business decisions)
 - **Query functions** = 1 loop (established pattern)
 - **Type layer** = 1 loop (fixed shape)
 - **Test generation** = 1 loop (AI is great at edge cases)
 - **Migration system** = 1 loop (Rust pattern is well-known)
-

7.10 Registering the commands on the Rust side

Last step: register the commands in `lib.rs`. Take the `invoke_handler! [...]` we saw in Chapter 5, add the three commands we built in this chapter (`db_query_json`, `db_execute`, `run_migrations_cmd`), and call `init_db()` and `run_migrations()` inside the `setup` block. Tauri enforces this structure, so there isn't much choice involved—every time you add a command, you add one more line in the same place. Use the registration skeleton from Chapter 5 verbatim.

Watch out: If `init_db()` or `run_migrations()` fails, the app panics at startup. In production, database initialization errors need proper handling.

Debugging tip during development: When a panic happens, the message can flash by quickly. Run `cargo tauri dev` with a terminal open so you don't miss the first-run failure message. If the message isn't visible, drop an `eprintln!` inside `init_db()`'s `map_err` temporarily to narrow down which step is failing.

7.11 What's next

The next chapter implements streaming integrations with three AI providers—OpenAI, Claude, and Gemini. It's more complex than the schema. The SSE format, the error format, and the timeouts differ across providers, and some of the implementations took five or more loops.

What to remember from this chapter

After one pass through this chapter, these five points are enough.

1. **Schema design doesn't finish in one shot.** It's normal for the first draft to reveal decisions you forgot to make. The second and third loops are where you fill those gaps.
2. **AI defaults to SQLite.** If you're using DuckDB, say so in the prompt. That's how you unlock the analytical functions, richer types, and JSON functions.

3. **CHECK constraints are rules that stop invalid data at the database itself.** Add the basic ones —cell index range, session end time —from the start.
4. **Use one connection across the whole app.** Share a single connection using the `Once-Lock<Mutex<Connection>>` pattern. Don't open a new connection per command.
5. **Tell apart the work the AI finishes in one go and the work that needs loops.** Boilerplate (the migration runner, query helpers, the type layer) finishes in one pass. Design decisions (the schema, constraints, index strategy) need two or three loops.

The takeaway: Database schema work follows a predictable shape when you collaborate with AI. Design decisions usually take two or three exchanges, but implementation details often finish in one. The streaming AI API in the next chapter has more iterations and trickier fixes.

Chapter 8. Integrating AI APIs —Writing AI Code with AI

Connecting three AI providers (OpenAI, Claude, and Gemini) to a single UI is one of the areas where AI coding tools are both most useful and most full of traps. Every provider has its own authentication scheme, response format, and error shape. The parts spelled out in the docs are handled well by AI. But the parts the docs don't cover —what happens to the HTTP request when the user hits a cancel button, or what to show when the stream stalls —are a different story.

The big picture of this chapter fits in one sentence. **Concentrate the differences in one place in Rust, and let the frontend deal with just three events.**

```
[Frontend (SvelteKit)]
  ask_ai_stream(provider, model, messages)
  |
```

```

      v
[Tauri command] --> [Provider layer (Rust)]
                    - OpenAI
                    - Claude   HTTP stream -> extract text fragments
                    - Gemini

      |
      v
[Frontend listen]
  ai_chunk(text fragment) / ai_done(finished, usage) / ai_error(error)

```

What the user sees on screen is much simpler. You type a question into the chat input, and letters get appended bit by bit to the answer area. If the provider sends three fragments — "Hel", "lo", " world" —the frontend stitches them together and shows "Hello world". The code in this chapter is the work of making that “receive a fragment and append it” flow reliable.

Even though each provider has different response and error formats, the frontend only needs to know about three events: `ai_chunk` (a text fragment), `ai_done` (completion), and `ai_error` (an error). The Rust provider layer absorbs all the differences.

Let’s introduce one new term you’ll meet for the first time in this book. **SSE (Server-Sent Events)** is a way for the server to keep an HTTP connection open and stream the response out a piece at a time. AI providers tend to use SSE so they can send tokens one fragment at a time as they’re generated. The frontend can append each incoming fragment to the screen immediately, without waiting for the whole response, which creates the “feels like it’s being typed” experience.

The most important decision when first building this was: **where do we hide the differences?** Scatter them across the UI and your buttons, chat window, and error toasts get tangled up in provider-specific conditionals. Pull them all into one place in Rust and the frontend just receives the same events. All the user needs to know is whether a response is coming, whether it’s finished, and whether it failed.

The Flow of This Chapter

- Write down the requirements in plain language first
- Multi-provider abstraction —the Rust trait pattern
- The event structure and stream cancellation (the main thread)

- Normalize provider events to delta, done, and error, handling cancellation and completion per stream ID.
- Wrap-up that leads into Chapter 9

Learning Goals

- Understand the Rust trait pattern (a concept like an interface) for wrapping multiple AI providers behind one interface.
- Get comfortable with the streaming flow built on three events: `ai_chunk`, `ai_done`, and `ai_error`.
- Know why per-provider SSE, error, and authentication differences should be absorbed in one place.
- Learn basic security principles such as storing API keys safely.

What to Look at on Your First Pass

This chapter throws Rust async, SSE, per-provider response differences, and cancellation handling at you all at once. Trying to understand everything from the start is hard. On your first pass, just grab these four ideas:

- The UI doesn't have to know about per-provider differences.
- The Rust provider layer absorbs the differences between OpenAI, Claude, and Gemini.
- The frontend only receives three events: `ai_chunk`, `ai_done`, and `ai_error`.
- Operational details like API key storage and cost limits matter, but you can come back to them once you understand the app flow.

You don't need to fully memorize `tokio::select!`, each provider's SSE end signal, or rate limiting right now. The core idea of this chapter is **“hide the differences in one place, keep the screen simple.”**

8.1 Write Down the Requirements First

If you just tell the AI “connect these three providers,” the results are all over the place. Example code from different providers gets mixed up, and error handling looks different every time. So before writing any code, I wrote down the requirements.

- The frontend doesn’t need to know about providers. It uses one command: `ask_ai_stream(provider, model, messages)`.
- Streaming fires `ai_chunk` events via Tauri emit. The frontend receives them with `listen`.
- Token usage is sent once at the end of the stream as an `ai_done` event.
- All errors funnel into a single `ai_error` event.
- API keys are read from the OS keyring. They are never passed in as command parameters.
- Use `cancel_ai_stream(stream_id)` to stop an in-flight request.

I pinned these six lines at the top of the prompt. Claude followed the spec for the entire session. **Lock down short, clear requirements up front and the number of revisions drops by half.** The more stateful and complex the feature —like streaming —the more important it is to nail down event names and responsibility boundaries before you start.

The core design principle is “**protect the frontend from provider changes.**” If a model name changes or a new provider is added, you shouldn’t have to touch the chat UI.

Security: API keys placed in environment variables are visible to every running process. You should use the OS keyring (macOS Keychain, Windows Credential Manager, Linux Secret Service). Tauri’s `tauri-plugin-keyring` standardizes this for you. When prompting the AI, mention keyring usage explicitly.

8.2 Multi-Provider Abstraction —Rust Traits

Let’s first cover why a trait is needed. If you stuff per-provider code straight into the command handler, it balloons like this over time:

```
if provider == "openai" { ... 100 lines ... }
else if provider == "claude" { ... 100 lines ... }
else if provider == "gemini" { ... 100 lines ... }
```

Every new provider makes the same function longer, and fixing one branch tends to break another. Wrap them all behind a common interface, and a new provider just needs to be implemented in its own file against the same agreement.

A Rust **trait is a concept similar to a TypeScript interface**. It acts as a contract that says, “you can be used as a provider only if you implement every feature in this list.” The provider trait in this project needs only four methods.

In TypeScript, it would look roughly like this.

```
interface ChatProvider {
  streamChat(input: ChatInput): Promise<void>;
  parseChunk(event: unknown): string;
  parseUsage(event: unknown): Usage;
  mapError(error: unknown): AiError;
}
```

A Rust trait enforces the same agreement. The only difference is syntax. As a beginner, it’s enough to think of it as “a contract that forces every provider to implement the same four functions.”

Method	Responsibility
stream_chat	Sends the HTTP request, receives SSE events, and emits them
parse_chunk	Pulls a text fragment (delta) out of one event
parse_usage	Pulls out token usage data
map_error	Maps HTTP status and response into a common error format

Once these four are settled, the command handler just looks at the provider string, picks the matching instance, and calls `stream_chat`. In pseudocode, it’s this short:

```
// Command handler: branch once, then only call trait methods
let p: Box<dyn ChatProvider> = match provider {
  "openai" => Box::new(OpenAi),
  "claude" => Box::new(Claude),
  "gemini" => Box::new(Gemini),
```

```

    _ => return Err(unknown_provider()),
  };
  p.stream_chat(model, messages, stream_id, app).await

```

To add a new provider, you add a type that implements the same four methods and extend the `match` by one line. The command handler body and the frontend stay untouched.

A common provider accepts a request and cancellation signal, then emits text delta, completion, or error. Internal events can be `Delta { stream_id, text }`, `Done { stream_id }`, and `Error { stream_id, code, message }`. Convert provider responses inside adapters so the UI does not branch on provider name. Do not expose credentials or raw HTTP responses in error text.

8.3 Event Structure

If the trait is “the container that holds the differences in one place,” the event structure is “the consistent shape that container shows to the outside world.” The two go together. With a trait but inconsistent event names, the frontend wobbles. With unified events but no abstraction inside, the handler balloons.

In the first phase I held off on actual HTTP communication and **first laid out only the Tauri command and event-emission structure**. Two things matter:

- When the command is called, the backend uses `tokio::spawn` (a function from Rust’s async runtime `tokio` that launches work in the background) to kick off the task and returns immediately. **It plays a role similar to creating a Promise in JavaScript and letting it run in the background.** That’s why the frontend doesn’t sit waiting for a response.
- The frontend subscribes to `ai_chunk`, `ai_done`, and `ai_error` events with `listen`.

The core of the frontend listener boils down to four lines:

```

const unChunk = await listen<{ delta: string }>("ai_chunk", e => buf +=
e.payload.delta);
const unDone = await listen<{ usage: Usage }>("ai_done", e => { usage
= e.payload.usage; });
await invoke("ask_ai_stream", { provider: "claude", model, messages });

```

The key is **how buf accumulates deltas**. The `ai_chunk` event does not send the whole message at once. It carries only the newly arrived token fragment. The frontend stitches those fragments together in order to form the complete message. This shrinks the payload and makes the stream feel much faster.

```
ai_chunk: "Hel"      buf = "Hel"
ai_chunk: "lo"       buf = "Hello"
ai_chunk: " world"   buf = "Hello world"
ai_done              stop rendering, save usage
```

As part of the same normalization, **the payload shapes the frontend receives are fixed to four forms**. `ai_chunk` is { `delta` }, `ai_done` is { `usage` }, `ai_error` is { `provider`, `kind`, `message` }, and the cancellation notification is { `stream_id` }. These shapes don't change as providers are added. Claude once tried to stuff `model` and `finish_reason` into `ai_done`, but we decided “leave out fields the frontend doesn't use, and add them when they're actually needed.” **AI tends to add fields by default**. Let it run and the payload keeps growing.

After registering a listener, you must clean it up. The function `listen` returns (`unChunk`, `unDone`) has to be called, or on the next message the same callback piles up two or three times, and the same text appears repeatedly on screen. Listeners also outlive the page and hold on to memory. **There are two cleanup points —right after the stream ends (when `ai_done` or `ai_error` arrives), and when the component unmounts** (`onDestroy` in Svelte, the `useEffect` cleanup in React).

Stream Cancellation

This structure still has one problem. **Once a stream starts, there's no way to stop it**. Even if the user clicks “stop” or navigates away from the page, the background task keeps running. It burns API credits and keeps emitting events to a frontend that has already closed.

The one-line key to the fix is this. **Give each stream its own ID and its own cancellation channel**. With a single app-wide channel, a cancel signal would wake up every stream, and you'd have no way to know which one to stop (this kind of two-tasks-tangled-up situation is called a **race condition**).

Concretely, when a stream starts, generate a UUID as the `stream_id` and register a cancellation channel by ID. When `cancel_ai_stream(stream_id)` is called, only the channel for that ID receives the signal. On the task side, a tool called `tokio::select!` lets

you follow whichever happens first —“cancellation signal arrives” or “HTTP stream completes.” `tokio::select!` is **similar to JavaScript’s `Promise.race`**: it handles whichever of several async operations finishes first. If cancellation wins, the backend reports it with `ai_cancelled`, whose payload carries the `stream_id` so the frontend knows which stream ended.

Create the stream ID, register listeners, then start the request. Starting before registration can lose the first token. Callbacks process only their own `stream_id`; completion, error, cancellation, and view teardown all call the same idempotent cleanup. If registration finishes after disposal, immediately remove the new listener. Chapter 5 uses the same disposed-state principle.

Worth thinking about: If a stream finishes naturally at the same moment a cancellation arrives, `ai_done` and `ai_cancelled` may fire at nearly the same time. The frontend should ignore any further events after the first completion signal.

8.4 Where SSE Parsing Broke

From here through 8.6 are “details you’ll run into once you actually start writing the code, after the first pass.” On a first read, you can move on after grabbing just one big-picture line from each.

The big picture is simple. **Each provider has different end signals, event kinds, and authentication locations.** OpenAI uses the literal string `data: [DONE]` to end the stream, which breaks if you try to JSON-parse it, and usage only appears in the final chunk. Anthropic (Claude) requires branching on the `SSE event: line`, and its output token count is the current response value rather than a cumulative one —don’t add them up. Gemini doesn’t default to SSE but to a **JSON-lines stream** (JSON objects separated by newlines), so you have to add `?alt=sse` to the URL, and auth goes through a query parameter (`?key=...`) rather than a header.

The adapter converts a common request into provider JSON and maps responses back to common stream events. Network chunks are not SSE events: buffer bytes and parse blank-line-delimited events, including UTF-8 characters split across chunks. `reqwest` can handle HTTP and `serde_json` can handle JSON, but field names and end markers must match the provider API version.

Caveat 1: Anthropic calls must include an `anthropic-version` header. Without it you get HTTP 400. Don't hardcode the version string —check the current value in the official docs.

Caveat 2: Gemini passes the API key as a query parameter. It can leak into server logs or HTTP debugging tools. For production, consider OAuth2.

Caveat 3: Token-count accuracy varies by provider. Avoid building billing logic that depends directly on token counts.

You don't have to memorize these three caveats on a first pass. The point is that **stream formats differ across providers, so those differences should be handled at the provider layer, not in the UI.**

8.5 Error Handling —When Each Provider Fails Differently

All three providers have different error response formats. OpenAI uses `error.type` and `error.code`, Anthropic uses `error.type`, and Gemini uses `error.status` and `error.code` —different field names in different places.

If you let those differences leak straight into the frontend, every UI piece grows its own branching code. **The whole point of error normalization is one thing.** No matter what shape the provider's failure takes, the frontend always receives `{ provider, kind, message }`. Define a small set of `kind` values such as `auth`, `rate_limit`, `context`, `server`, and `unknown`, and map into one of them by looking at both the HTTP status code and the provider-specific error type string.

Start error mapping with HTTP 401/403 for authentication or permission, 429 for rate limits, 5xx for server errors, timeout for expiration, and parsing failures for invalid response format. Respect Retry-After and bound retries. Do not repeatedly retry invalid credentials before they change. Show safe explanations and log request IDs plus error categories for diagnosis.

Caveat: Error messages from AI providers can include sensitive information like fragments of API keys or internal service names. Always sanitize them before showing them to users or writing them to logs.

8.6 Rate Limiting and Cost Control

Things to add when operationalizing: AI APIs cost money per call, and each provider enforces separate “requests per minute” and “tokens per minute” limits. The usual approach is a **token bucket** algorithm to manage both limits at once, with per-model pricing kept alongside it to track cumulative cost. Prices and limits change frequently, so don’t hard-code them —keep them in config. And for billing math, use the usage numbers from the response rather than an upfront estimate. On a first pass, just remember “track two limits separately —request count and token count.” The implementation details belong to the operations chapter.

8.7 Response Normalization —A Uniform Shape for the Frontend

The actual normalization of response payload shapes was already covered in 8.3 along with the event structure. Here I want to reinforce why that decision shouldn’t be allowed to slip.

If you let the payload shape drift, the frontend and backend get coupled unnecessarily. By the fifth provider, you’d be touching the chat UI just to add one. So we accept new field requests conservatively —if you can’t answer “where on screen do we use this field right now?”, set it aside. The value of normalization comes from the promise that the shape doesn’t change.

8.8 The Iteration Summary for This Chapter

I have a table of how many revisions happened in each area during this chapter, but the next chapter analyzes that same data in more detail in its opening. **The one line to remember here is this: when the design is solid, most things passed in one or two tries; only the per-provider protocol edge cases drove the revision count up.** Chapter 9 generalizes this pattern.

8.9 How AI Writes Code

This section overlaps with Chapter 9’s topic. The distinction between **areas where AI is strong (idiomatic patterns, abstraction design, data transformations with examples)** and **areas where it stumbles (per-provider protocol edge cases, end signals, authentication locations)**, along with the workaround “for the weak areas, include a summary of the API docs in the prompt,” is covered in detail with examples in the next chapter. Here I’ll just leave one line —**AI code generation works best when requirements are clear and there are enough examples.**

8.10 What You Must Remember from This Chapter

- One sentence: **Concentrate the differences in one place in Rust, and let the frontend receive just three events —`ai_chunk`, `ai_done`, and `ai_error`.**
- Core structure: provider trait (four methods) / Tauri event-based streaming (delta accumulation) / `per-stream_id` cancellation (`tokio::select!`).
- Provider gotchas: OpenAI’s `[DONE]` is a literal string and usage appears only in the last chunk. Anthropic branches on `event:` lines, and output tokens are not cumulative. Gemini defaults to JSON lines, needs `?alt=sse`, and uses a `query-parameter` for auth.
- Operations: API keys in the OS keyring, rate limits tracked as two separate ceilings (requests and tokens), price tables not hardcoded.
- Safety: cleanup after listener registration (on stream end and on component unmount), follow only the first signal when cancellation and normal completion collide, sanitize error messages before exposing them to users.
- Using AI: idiomatic patterns, abstraction design, and data transformations work well. Protocol edge cases are weaker -> **write requirements down first, and for edge-case handling, include a summary of the API docs in the prompt.**

The next chapter (Chapter 9) generalizes the pattern that shows up in the table above. “Where is Claude strong and where does it stumble, and how do you respond when faced with patterns that aren’t in its training data?” And the **trait-based abstraction, the three-event payloads, and `per-stream_id` cancellation** patterns established in this chapter will keep showing up in

the same shape every time you bolt on another external API after Chapter 9. The marks left by those seven rounds become the starting point for the next chapter.

Chapter 9. Mandai Development Mistakes —Where the AI Got It Wrong

Working with AI produces two kinds of results: things the AI got right, and things it got wrong. I learned more from the second kind. The places where I approved code without checking, the places where I noticed a structural problem only after stacking three more features on top — those failures became the raw material for my next attempt.

The earlier chapters covered the flow of collaborating with AI. This chapter shows the other side. It is a record of **the areas where the AI was repeatedly wrong**, and how I caught those mistakes and worked around them. I left out small typos and import errors that were fixed in

one or two tries. I only kept the problems I had to fix five or more times, or the ones where I had to rewrite the requirements themselves. Four traps kept showing up.

1. **State synchronization** —the frontend and the backend see different states.
2. **Implicit rules** —rules that lived only in my head, so the AI could not know them.
3. **Indifference to edge cases** —by default, the AI prefers the happy path (the normal flow with no errors).
4. **Version confusion** —the AI mixes old API patterns with new ones.

A quick note on terms. **State synchronization** is the job of keeping the same piece of information consistent when it is stored in two places (for example, the screen and the database). **Domain model drift** is when the AI applies a popular generic pattern instead of the rule that is specific to our project. Both terms come up again and again in this chapter.

Warning: do not trust an AI “best practice” without checking it. The AI confidently suggests patterns that work in tutorials but fail in real services. Always test the full behavior flow.

This chapter is not an attack on the AI. The opposite, actually. Once you know the spots where the AI tends to fail, you can write more precise requests. “Build a timer” and “if I close and reopen the app, the running timer should be restored” are not the same request. The second one carries information about ownership of state, when to save, and how to restore. Most real-app requirements look like that second one. One more specific sentence changes the direction of the whole implementation.

Reading guide: section 9.1 is the longest; the rest are short. If you get through 9.1, the rest goes fast. 9.1 is long because all of the chapter’s core insight is packed into it.

Flow of this chapter

We start with the heaviest case and work down to lighter ones. At the end, the prevention tools are bundled together.

- **9.1 Pomodoro timer** —what happens when state lives in two places
- **9.2 Mandal-Art cell ID** —the moment a generic pattern overrides a domain rule
- **9.3 Productivity score calculation** —naming the edge cases makes the AI better
- **9.4 Version confusion** —mixing Tauri 1.x and 2.x APIs

- **9.5 Prevention: the prompt + review + test trio**
- **9.6 Summary and next chapter**

Learning goals

- Recognize the four recurring AI mistake patterns (state synchronization, implicit rules, edge cases, version confusion).
- Learn from real cases how each mistake is spotted and fixed with a prompt.
- Build and use your own AI-mistake-prevention checklist, review prompt, and test protocol.

9.1 Pomodoro timer —state living in two places

The key lesson of this section is one line: wherever you store the “time remaining” for a running timer, you must always be able to recompute it from `start_at`. I reached that conclusion after five rounds of trial and error. The story below is how I got there.

Mandai’s Pomodoro feature (the technique of focusing for 25 minutes and then taking a short break, repeated) looks simple from the outside. Press start, count down 25 minutes, save the session to DuckDB when it ends. In practice it is not that simple. Behind the number on the screen, you also need a start time, a target duration, a linked task, an ending reason, a notification, and rules for how to recover state when the app is reopened. On top of that, the user might close the app while the timer is running, and the laptop might suddenly go to sleep.

At first I did not put any of those conditions into the prompt. The AI’s first timer came out at demo quality, and every time I added a real condition another problem surfaced. I ended up fixing it five times. With every fix, the word “timer” got clearer in my head—it is not a plain countdown. A timer is a small system that stores state and can reload that state when needed.

The real problem

The core issue is that Pomodoro state is split across two places.

- **Frontend:** the seconds counting down on the screen.

- **Backend** (the sessions row in DuckDB): the permanent log. `start_at`, `end_at`, `ended_reason`.

Claude first built the timer **only on the frontend** and only wrote to the DB once the session finished. In a demo it looks fine. The countdown ticks down, and when it hits zero a “completed” record is left behind. But once you close the app and reopen it, the running session is nowhere to be found. The real requirement was “if you close and reopen the app, the running timer should keep going.” In other words, the timer state has to live in the DB, where it can be read back to restore the screen.

A bad first attempt

My first prompt was something short like “show a 25-minute countdown and save to the DB when it finishes.” The code Claude produced kept the timer state in a component variable and **only wrote one row to the DB at the moment the session ended**.

```
// Wrong: timer state only lives in the component
let remainingSeconds = 25 * 60;
// One INSERT only on completion -> no persistence while running
```

It works as a demo. But the moment a user closes the app mid-session, the time they had already worked through is not recorded anywhere. A tutorial timer only has to run while the screen is on, but a real-app timer has to keep its current state somewhere even after the screen is off.

After that first attempt, four more rounds of fixes followed. INSERT the row at the start, clean up open sessions on restart, recover by time instead of cleaning up unconditionally, and finally move the timing itself into the Rust backend. Each fix started from a gap left by the previous one. I will collect the details at the end of this section under **Key lessons**. Here I want to walk through three of the traps I hit along the way.

Trap 1 —CHECK constraint violation

When I tried to INSERT a running session at start time, Claude tried to put `ended_reason='running'` into the row. The sessions table from chapter 7 has a

CHECK constraint (a DB rule that only allows specific values) on the `ended_reason` column, restricting it to 'completed', 'skipped', or 'interrupted'. The insert was rejected.

The fix is simple. Leave `ended_reason` as `NULL` at start (= still running), and fill it in only at completion, interruption, or skip. This way you can keep the schema from chapter 7 intact.

Trap 2 —Over-aggressive cleanup, and time-based recovery

I asked for “on restart, clean up any session where `ended_reason` is `NULL`.” Claude wrote a function that marked every open session as `interrupted` whenever the app started. But this was too aggressive. **Even when the app had just briefly gone into the background and come back, the timer was being treated as interrupted.** The user comes back after five minutes and the timer is already dead.

That is when it clicked. Closing the app, minimizing it, the laptop going to sleep, the process being killed—those are all different situations. But in the DB the only signal you have is `end_at IS NULL`.

I changed direction. Stop cleaning up unconditionally, and **recompute based on the start time**. That is, if the time elapsed since `start_at` is past the target, automatically end the session; if there is still time left, recompute the remaining time and bring the timer back to life. The key is one thing—do not trust the stored “remaining time,” recompute it from `start_at` every time.

What to look for: close the app, reopen it five minutes later, and the remaining time on screen should have dropped by five minutes. If it is unchanged, you are still trusting the stored value.

Trap 3 —Background throttling and task binding

The last round revealed two more things.

Background throttling is when the browser engine (WebView) reduces how often JavaScript timers fire while the app is in the background. `setTimeout` is affected the same way. The fix is to **move the timing into the Rust backend**. The Tauri backend is not subject to browser throttling. The backend keeps time and, when it is up, sends an event to the frontend.

Task binding is a different kind of decision. If the user switches tasks during a timer, should `sessions.task_id` follow? Without an explicit rule, the AI guesses differently every time.

Decision: **a session is locked to the task that was active at start time.** This makes productivity reports easier to interpret and keeps the stats consistent.

What to look for: send the app to the background and wait 25 minutes. The notification sound should still play.

Revised prompt template

After five rounds, here is the prompt I wish I had sent on day one.

Build a Pomodoro timer with these requirements:

Persistence:

- On start, immediately save the session to DuckDB (end_at=NULL, ended_reason=NULL)
- On restart, if elapsed time is less than target_minutes, restore the active session
- Only if target_minutes has been exceeded, mark it ended_reason='interrupted'

Timing:

- Use tokio::time in the Rust backend (not subject to browser throttling)
- The frontend only displays; the backend owns the actual timing
- When the backend timer expires, emit the Tauri event 'timer_completed'

Task binding:

- session task_id is locked to whatever task was active at start time
- Changing the task during a session must not affect session.task_id

Tests:

- [] Start the timer, close the app, reopen -> restores with the correct remaining time
- [] App backgrounded for the entire duration -> notification still fires
- [] App crash during the timer -> marked 'interrupted' on next start

Warning: pushing forward with a broken structure is a sunk-cost mistake. If you have looped on the same feature 3 or 4 times and it still does not work, check whether the AI's original approach was fundamentally wrong. Sometimes it is faster to lay out the constraints again and start over.

Key lessons

State synchronization is the job of keeping the frontend timer and the DB row pointing to the same value. The AI alone cannot solve this. A human has to decide three things first: who plays which role between frontend and backend, how to recover when the app exits abnormally, and how to guarantee data consistency. You have to decide those three and tell the AI. Otherwise, the AI writes code that works only when everything is fine.

After five rounds, I built a **timer state synchronization checklist**.

- ☐ Do the frontend timer and the DB row start at the same moment?
- ☐ When the app crashes, what happens to open sessions?
- ☐ What state does the user see when returning from sleep or background?
- ☐ Who plays the completion notification —frontend or backend?
- ☐ If session metadata (task, etc.) changes mid-run, do we ignore the change or apply it?

When I build something similar, I paste these five items at the top of the prompt to start. A lesson from one painful loop becomes a prompt asset I keep reusing. The next section applies the same “spell it out up front” principle to a different kind of trap —domain rules.

9.2 Mandal-Art cell ID —when a generic pattern overrides a domain rule

After state synchronization, the next trap I ran into most often was **domain model drift**. The AI does not know our project’s specific rules, so it falls back on the generic patterns it has seen most often in training data. The cell ID case is the clearest example.

In chapter 6 I decided that cell IDs would follow the format `{center_idx}-{sub_idx}-{cell_idx}`. I also wrote that decision into `.cursorrules`. And yet, when the AI wrote UI components, **it invented a completely different format**.

Where it went wrong

The `MandalaGrid.svelte` Claude generated rendered cells with `id="cell-${row}-${col}"`, where `row` and `col` were numbers from 0 to 8. The result: the DB tables used the format `1-3-5`, but the DOM in the screen used `cell-3-5`. The moment the click handler tried to look up the DB using the DOM id, the two formats did not match and threw an error.

It took me two hours to find the cause. The console kept printing `No cell found with id 'cell-3-5'`, but the DB only had things like `'1-1-1'` or `'1-1-2'`. **The ID scheme had split in two.**

Why this happens

My prompt was too generic. I had only said “make a 9×9 cell grid.” I never explained that the grid is in fact a **structure of overlapping 3×3 blocks**. Claude naturally picked the generic grid pattern (`row/col`). From the AI’s point of view it was a reasonable choice, but it did not match Mandai’s domain model.

The format was written in `.cursorrules`, but I had not explicitly forbidden any other format. So the AI went with the more familiar generic pattern.

How I fixed it

The fix started in the rules file, not the code. I added one line to `.cursorrules`.

A Mandal-Art (9×9 goal grid) cell ID is a string in the format `{center_idx}-{sub_idx}-{cell_idx}`.

The DOM `data-cell-id` uses the same format. No other format is allowed.

I also made the prompt more specific. Instead of just “9×9 grid,” I wrote: “a structure of 3×3 sub-grids, with IDs in the format `{center_idx}-{sub_idx}-{cell_idx}`.” The fixed component also uses the same format for the DOM’s `data-cell-id`.

What to look for: when you click a cell, the console should print something like `1-3-5`. If it prints `cell-3-5`, old code is still around.

After that one line went into `.cursorrules`, the same mistake never came back. **If a rule is not written down explicitly, the AI guesses again from scratch every time.**

Warning: the AI confidently uses “standard” patterns even when they collide with the domain model. Unless you explicitly forbid them, generic patterns will overwrite domain-specific rules.

The conclusion is simple. The AI’s guesses lean toward “what is commonly used.” Every time your domain departs from the common pattern, you have to point out the difference. These three in particular belong at the top of the prompt or the rules file.

- ID schemes that differ from common idioms
- Data flows that differ from common practice
- Naming conventions that differ from standard library conventions

These three are not matters of taste; they are part of the data model. In the next section we look at how the same pattern repeats not in the domain but in the **boundary values of the data itself**.

9.3 Productivity score calculation —naming the edge cases makes the AI better

Mandai shows a “weekly productivity score.” I defined the formula myself.

```
score = (completed_tasks × 10 + pomodoro_minutes × 1) / effective_time ×
correction
effective_time = sum of "focus hours" from the work-day settings
```

I asked the AI to turn this formula into a SQL query. My first prompt was simple —“calculate this formula per week.”

Edge cases that came out loop by loop

Claude’s query worked fine when the week was full, Monday through Sunday. But the people actually using the app do not only look at completed weeks. Checking on a Wednesday afternoon, after just Monday has passed, or during a week with a holiday in it —those mid-week checks are far more common.

Each loop revealed a new edge case.

Loop	Problem found	Fix
1	Always divided by 7. Looking on Wednesday gave a too-low score	For the current week, divide by days elapsed
2	Weekends were in the denominator even when no work happens	Sum only weekdays
3	Holiday settings ignored	LEFT JOIN the holiday table
4	If effective time is 0, division error	Treat 0 as score 0

The query grew by 4-5 lines per loop. The biggest change was how to set the denominator. At first it was always 7; after fixes, it branched into “if this is the current week, only count days elapsed so far; if it is a past week, 7.”

What to look for: when you view the score on Monday of a new week, it should not be unreasonably low. Also check that the denominator is not 1, which would inflate the score.

What I took away

The pattern is simple. **The AI only handles what you spell out. What you do not say, it skips.** The developer has to find and name the edge cases. Once you name them, the AI carries the pattern into later work.

After this experience, when I asked for other features Claude started checking these cases on its own first:

- “What if the denominator is 0?”
- “What if the date range is empty?”
- “What if the settings value is null?”

Teach an edge case once and the AI applies the lesson to other features. But you have to think of it first and feed it in.

Warning: the AI is satisfied when the logic is correct. Whether the result is meaningful under extreme inputs is something you have to test yourself.

In the next section we look at a case where it is not our code but **a change in the framework itself** that confuses the AI.

9.4 Version confusion —mixing Tauri 1.x and 2.x APIs

The AI’s training data contains a mix of Tauri 1.x and 2.x examples. The faster a framework evolves, the more old tutorials linger on the internet, and the AI learns from them too. As a result, if you just say “Tauri code,” the model statistically falls back to the more common v1 patterns and confidently slips them in among v2 code. Code like that compiles fine but fails at runtime. The trickier variant is the pattern that works in dev but breaks only in the production build.

A classic example: window management API

I asked for a “minimize to tray” feature. The code Claude returned mixed v1 and v2 imports. In v1 you used the global `appWindow` object; in v2 you get the window through the `getCurrentWindow()` function. The AI used both styles in the same file.

TypeScript saw the imports as valid, so it compiled. But at runtime the error said “no such function exists.”

When it fails, check this: errors like “function not found” are almost always a sign of version mismatch. Start with the frontend imports, not the Rust commands. I did not know that and spent four hours staring at the Rust side.

Pinning the version in the prompt

The fix was to nail down the version in the prompt itself.

Use Tauri 2.x (not 1.x). Implement minimize-to-tray. Key differences:
 – Import `getCurrentWindow()` from '@tauri-apps/api/window'; do not use the `appWindow` global

- For basic window operations, call window methods directly; do not go through `invoke()`
- The tray API is in `@tauri-apps/api/tray`, not the old `system-tray` plugin
- Window events use `window.listen()`, not `appWindow.listen()`

Show only Tauri 2.x. If you are not sure about an API, do not guess — ask.

After that, every import landed on the v2 path.

What to look for: the `@tauri-apps/api` version in `package.json` and the import paths should agree. If the v1 `appWindow` global is still in the code, that is a red flag.

Common confusion points in v1 -> v2

Area	v1	v2
Window object	<code>appWindow</code> global	<code>getCurrentWindow()</code>
File system	<code>@tauri-apps/api/fs</code>	<code>@tauri-apps/plugin-fs</code> + plugin registration
HTTP	<code>@tauri-apps/api/http</code>	<code>fetch()</code> from <code>@tauri-apps/plugin-http</code>
Tray	<code>system-tray</code> plugin	<code>@tauri-apps/api/tray</code>

The file system has the most pitfalls. In v2 you have to declare permissions explicitly in `tauri.conf.json` and register the plugin on the Rust side too. If the AI proposes a v1 pattern, the code compiles but quietly fails at runtime because the module cannot be found.

Every Tauri prompt I write now starts with one line: **“Use TAURI 2.x —verify every API.”** And I attach the key rows from the table above. That single guard line wiped out most of my version confusion.

Warning: the AI’s training data is a mix of old and new versions. The faster a framework evolves, the worse the mixing. Always pin the exact version, and check that the imports match `package.json`.

We have looked at the four traps separately. The next section pulls together **three tools to prevent them** in one place.

9.5 Prevention: the prompt + review + test trio

The four mistakes show up differently, but the way to prevent them goes through the same three steps. **Spell things out before you start** (a checklist), **make the AI re-review its own output** (a review prompt), and **have a human actually exercise the behavior** (a test protocol). The three are tools at the same level. If any one of them is missing, mistakes the other two cannot catch leak through.

A short bit of background helps before diving in.

Fake productivity

Speed does not equal productivity.

Claude can generate 200 lines in 30 seconds. It feels like fast progress. But because the AI wrote that code **without handling edge cases**, you later spend seven hours debugging a single state-synchronization bug. The AI gives you the feeling of motion even as technical debt piles up.

- **Traditional development:** slow to write, fast to debug.
- **AI-assisted development:** fast to write, slow to debug.

Over six weeks of Mandai I spent more than a day per week undoing AI mistakes. Without guardrails, AI use can be net negative. The numbers in the table below are illustrative, not precise totals.

Mistake type	Frequency	Main cause
State synchronization issues	3 features	Lifecycle spec missing
Version confusion	8 times	Mixed API training data
Edge case blind spots	Multiple features	Only happy path tested
Domain model drift	4 features	Generic pattern overrode domain rule

This table is the starting point for the trio. Let’s look at each tool.

Tool 1 —Prompt checklist

Anything more complex than CRUD (create, read, update, delete) starts from this checklist, no exceptions.

State management:

- [] Sync plan between frontend and backend state is defined
- [] Recovery behavior on crash / sleep is specified
- [] Edge cases listed: no data, mid-flight interruption, time zones

API versioning:

- [] Framework version is named (Tauri 2.x, SvelteKit 2 + Svelte 5, etc.)
- [] Import statements match the current version's docs
- [] No deprecated patterns from training data

Domain specifics:

- [] Custom ID scheme is documented in .cursorrules
- [] Non-standard rules are recorded explicitly
- [] Business rules that override "generic patterns" are listed

Test scenarios:

- [] Happy path test cases
- [] Edge case scenarios (empty, partial, overflow)
- [] Failure recovery scenarios (network, crash, data corruption)

This checklist maps directly onto the four traps from 9.1–9.4. An empty box means the opening for that trap is still there.

Tool 2 —Review prompt

Even when you start with the checklist, the AI still slips. So once you have the code back, you make the AI review it again. **When to run it?** The most efficient point is right after you have made two or three rounds of fixes on the AI-built feature, but before you start manual testing. Run it too early and it will critique parts that do not exist yet; run it too late and the only bugs left are the ones you have already caught.

Review this feature for AI traps:

State synchronization

- Does it safely recover state on app restart or crash?
- Are frontend and backend state consistent across the whole lifecycle?
- What happens if the user backgrounds/sleeps the app mid-flight?

API version accuracy

- Do all imports use the [FRAMEWORK_VERSION] API?
- Are there any patterns that look like an old version?
- Any deprecated methods or outdated import paths?

Edge case coverage

- What happens when data is missing or empty?
- How are boundary conditions handled (midnight, end of month, etc.)?
- Are time zones, holidays, and work-schedule changes covered?

Domain model alignment

- Does the code follow the custom ID scheme and rules?
- Do "generic best practices" override domain-specific rules?
- Confirm .cursorrules compliance

If you find any issues, propose specific fixes and an updated prompt.

One example of where this review template caught a bug Claude had missed: the AI's timer restore logic blindly trusted the `remaining_minutes` value stored in the DB and restarted the timer from there. The problem is that this value does not reflect time elapsed during sleep. If the laptop slept for 30 minutes, the displayed remaining time is too high. I ran the review prompt and it pointed at the problem right away: "do not trust the stored value, recompute the elapsed time from `session.start_at`." That is exactly the conclusion I reached the hard way after five rounds in section 9.1.

For the review tool to work, one thing has to be true. **A human has to be able to read the AI's code at least to some degree.** Not line-by-line scrutiny —just enough to recognize risky patterns. When you see patterns like these, stop and get suspicious.

- **Missing `await`:** if `await` is missing in front of an async call like `invoke(...)`, you get a Promise object instead of the result.
- **No error handling on critical paths:** code that shows a "success" message even when `save_data` failed.
- **State mutation without a reactive update:** in frameworks (like Svelte) where directly mutating an object does not refresh the screen, code that mutates an array element in place.

These three do not require domain knowledge and come up most often. Even if you do not read every line of AI code, having an eye for these patterns lets you judge what the review prompt is pointing at.

Warning: debugging AI-written code is harder than debugging code written by a human. The AI produces logic that looks plausible but is subtly wrong. You absolutely need a review habit to catch these before production.

Tool 3 —Test protocol

Even code that passed the checklist and the review prompt still needs a human to run it. The verification pattern has three stages and takes about 13 minutes per feature.

Stage 1: smoke test (3 minutes). Build, run, and check that the basic flow works. If not, back to the fix loop.

Stage 2: edge case exploration (5 minutes). Follow the checklist. Change the day of week, change the time zone, delete the data, click the button three times in quick succession.

Stage 3: state recovery test (5 minutes). Force-quit the app and restart it. Does the data come back? Is it stuck in a mid-state? If so, does the app handle that situation gracefully?

In Mandai the actual scenarios looked like this. Stage 1: start a Pomodoro and check the 30-second countdown, confirm the task list refreshes after completing a task, confirm a Mandala sub-goal saves. Stage 2: minimize the app during a Pomodoro and restore after two minutes, try to complete a task with an empty title, change the OS time zone and restart. Stage 3: force-quit the app during a Pomodoro and restart, force-quit while editing the Mandala, and restart with a corrupted DB file. If you apply the three stages consistently, you can catch most AI-introduced bugs before your users do.

Warning: AI-written code often looks fine in isolation but breaks during real use. Do not test only the one function you asked for —test the whole user flow.

How the three tools relate

The three tools block the same traps at different moments.

- **Checklist** —before writing the code, to reduce the opening.

- **Review prompt** —just after the code is generated, to make the AI re-check itself.
- **Test protocol** —at the end, where a human watches the real behavior.

Any one of the three helps on its own, but using them together is what shrinks the traps the most.

9.6 Summary and next chapter

I spent a real amount of time debugging while building Mandai. Four things to take away from this chapter.

First, the AI repeats four mistakes. State synchronization issues where the frontend and backend get out of sync; version confusion that mixes old and new APIs; edge case blind spots that only handle the happy path; and domain model drift that ignores domain-specific rules in favor of generic patterns. Knowing the patterns lets you head them off.

Second, the way to prevent these is to spell things out up front. Before asking for code, write the lifecycle (when it starts, stops, and restores), the exact version, the edge case list, and the domain rules into the prompt. One spelled-out line saves hours of debugging.

Third, verification comes as a trio. A checklist before starting, a review prompt right after writing, a three-stage test at the end. Any one of the three helps, but using all three together shrinks the traps the most.

Fourth, time saved by the AI comes back as debug time. Without guardrails, AI piles up code fast but leaks time later. The flow needs to change like this.

- **Traditional development:** think -> code -> debug
- **AI-assisted development:** think -> constrain -> prompt -> verify -> debug less

Adding that one “constrain” step is the key to cutting debug time.

Next chapter

The Mandai chapters are done. Next is **readbooks.ai**. We will re-implement, in Tauri 2, the PDF translation app I built in Next.js back in 2025. The loop this time has a different character. It is not a loop for catching code bugs; it is a loop for verifying **output quality**. The approach

is to use a different AI to check the AI's translation results. The mental model of “specify up front + verify after” that we built in this chapter applies, in the next one, to a different kind of output: translation quality.

Chapter 10. read-books.ai —Designing AI Output Quality with Prompt Engineering

Part III —readbooks.ai (Chapters 10-11) If Mandai dealt with “whether the code is right or wrong,” Part III deals with **output quality**. How do you decide whether an AI translation into 36 languages is actually correct? That question drives the whole part. Chapter 10 covers prompt design, and Chapter 11 covers the verification pipeline. On a first read, don’t get distracted by the scale of 36 languages. Hold on to one idea first: “If AI output is the product, you need standards and verification too.”

When an app’s **product is AI output**, feedback works differently. With code, a type error or a runtime exception shows up quickly. But a translation can read fluently and still be wrong in meaning. Neither the compiler nor your tests will catch that gap. So the central question of this chapter is not about implementing a feature. It is: **how do you design translation quality through prompts (the instructions you give the AI)?**

readbooks.ai translates PDFs into 36 languages. There is one decisive constraint, though. **Out of those 36 languages, I can only read 4 of them myself.** For the other 32, I cannot visually confirm whether the translation is correct. This single fact drives every decision in the chapter. The reviewer-role AI, the automatic format checks, even the back-translation in the next chapter—all of these answer the same question: “What does it take to keep a language I cannot read at product quality?”

When a prompt is wrong, the damage is not limited to one language. The same prompt is used for all 36 languages, so they all break in the same way. Prompt quality is product quality.

Background: readbooks.ai was a PDF translation app I built with Next.js in 2025. The service is no longer running. In this book I have re-implemented the source as Tauri 2 + SvelteKit 2 + Svelte 5. The migration itself is not the topic of this chapter. The focus here is the translation prompt system I designed during that re-implementation.

Flow of This Chapter

This chapter starts with how the translation prompt failed and then refines it toward verifiable quality standards.

- **10.1 Background** —Why I moved from depending solely on DeepL to a combination of LLMs.
- **10.2 The first prompt** —Why “just translate this” does not work.
- **10.3 Branching by language** —Why a single prompt cannot handle 36 languages.
- **10.4 The meta loop** —Using AI to refine the prompt itself.
- **10.5 Quality standards** —Turning “good translation” into a checkable list.
- **10.6 Wrap-up and summary.**

Learning Goals

- Understand how the development loop changes when AI output is the product.
 - Learn how to design the constraints that decide translation quality.
 - Follow the process of building language-specific profiles.
 - Learn how to use AI in three different roles: translator, reviewer, and language expert.
-

10.1 Background

In the beginning, readbooks.ai translated all 36 languages through the DeepL API alone. The output was, broadly, “readable.” But in a technical book, “readable” and “accurate in meaning” are not the same thing.

Even fluent sentences can cause problems. If code-block explanations are wrong, API names get localized, or part of a command gets translated, the reader ends up more confused than they would be with the original. In this book I rebuilt that structure on Tauri 2 + SvelteKit 2 + Svelte 5.

The central question of this chapter is this: **how do you keep translation quality consistent across 36 languages?** Picking a “good model” alone is not enough to answer it. You also have to design per-language profiles, decide which technical elements must be preserved, and write a reviewer-role prompt.

A provider boundary for cloud and on-device translation

A provider can be an external service or a small model running on the user’s computer. readbooks.ai places Claude, OpenAI, Gemini, DeepL, and its desktop runtime behind one `translate_with_provider` Tauri command. Rust routes the selected provider to HTTP or local inference. Swapping engines need not rewrite the UI, but consistent terminology still needs a glossary and model-aware caching.

On-device, on-demand, and offline describe different things

- **On-device:** inference runs on the user's hardware; sentences do not go to a translation server.
- **On-demand:** model downloading or loading happens when needed rather than keeping it resident from startup.
- **Offline:** after preparing model files, inference needs no network.

Selecting On-device and a GGUF file makes Rust run llama.cpp inside the app process. The first translation loads it; later requests reuse the instance. There is no localhost server or API key.

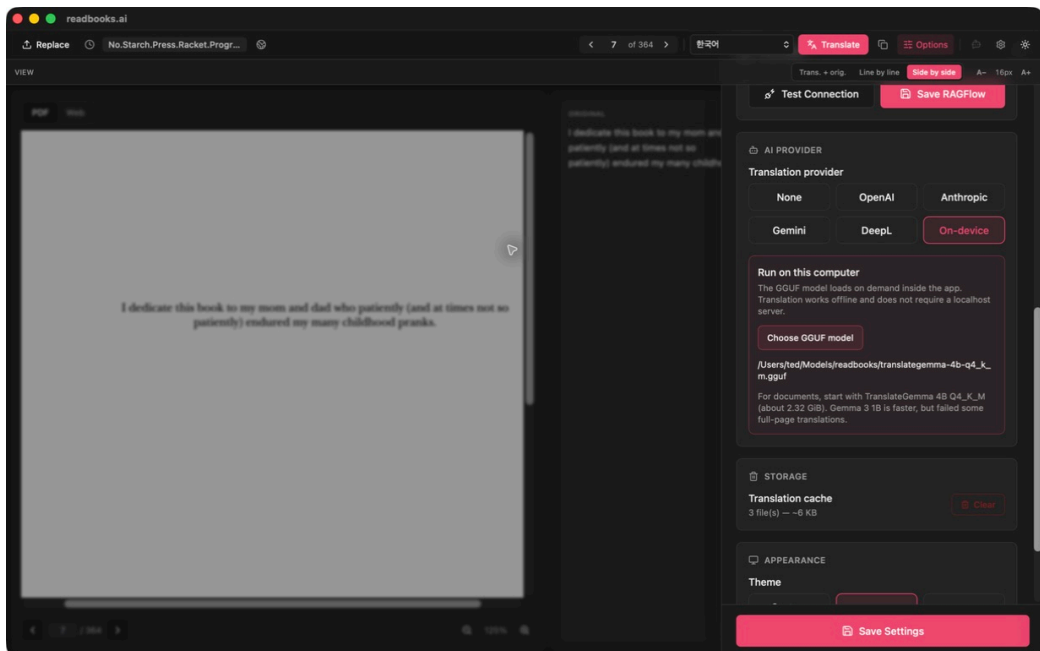


Figure 10-2. Selecting TranslateGemma 4B Q4_K_M GGUF. The app loads the model when needed.

```
Svelte -> invoke -> Tauri Rust: load/reuse GGUF
                        -> llama.cpp -> result + DuckDB cache
```

The routing sketch is:

```
match provider.as_str() {
    "openai" => do_translate_openai(/* ... */).await,
    "deepl"  => do_translate_deepl(/* ... */).await,
    "on_device" => {
        let path = model.ok_or("Settings에서 GGUF 모델을 선택하세요");
        on_device::translate(path, system_prompt, text,
            target_lang).await
    }
    _ => Err("지원하지 않는 Provider".into()),
}
```

`on_device::translate` sends expensive inference to `spawn_blocking` so it does not block the async executor. It uses the GGUF chat template; for a Gemma file without one, a fallback constructs user/model turns with `.`. The implementation uses a 4096-token context and up to 512 generated tokens. Oversized pages produce a request to split the input instead of silently truncating it.

Small models require workload-specific validation

The documented starting point for PDF translation is **TranslateGemma 4B Q4_K_M**, a 2.32GiB text-only GGUF with verified SHA-256 `e7afd56008be424d2ba36ed3b371d40039ae15fa467e8cf71442570d842b1119`.

The model is not committed to Git or bundled with the app. Users review Gemma terms and the file hash before importing it.

In the author's checks, Gemma 3 270M sometimes returned English unchanged. A 1B model handled short EarLoop conversation but produced mixed characters on PDF pages. TranslateGemma 4B translated the coffee question and a one-sentence dedication into Korean. These observations distinguish pipeline smoke tests, short conversation, and document translation; they are not a broad language benchmark.

The Rust test checks the key meaning, including the Korean word for coffee, rather than merely the presence of Hangul. The app rejects a Korean result containing no Hangul, but this cannot establish fluency or semantic accuracy. Longer documents still require Chapter 11's quality evaluation and a suitable larger or cloud model.

Ollama can help find or download models, but it is not an execution dependency. llama-cpp-2 opens GGUF directly. In the documented check, an Ollama vision blob lacked

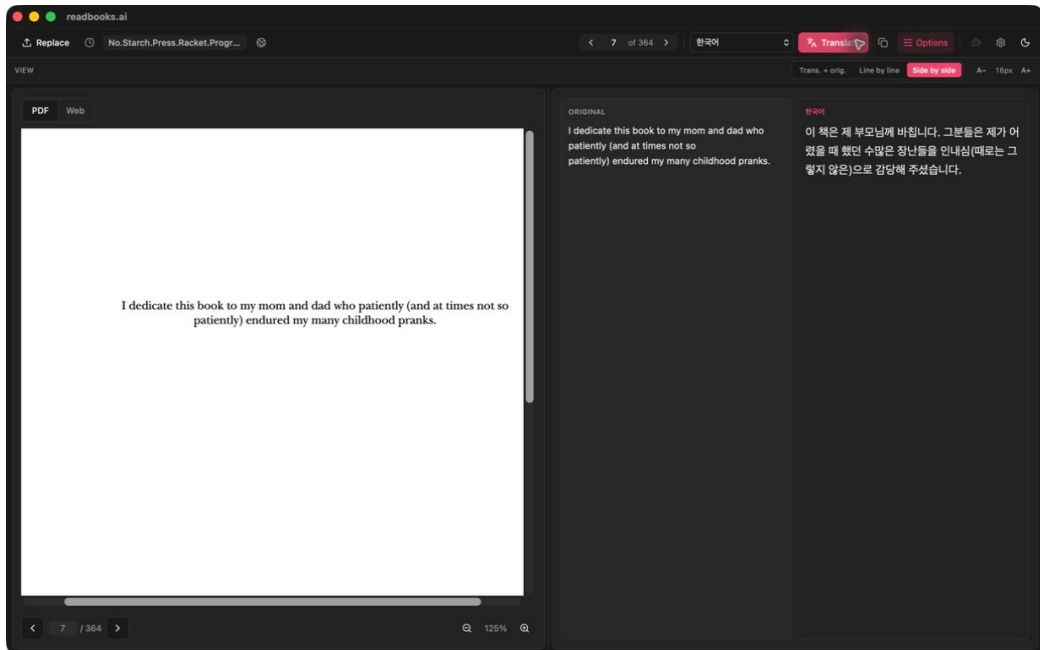


Figure 10-3. Translating a PDF dedication into Korean using TranslateGemma inside the app process.

gemma3.attention.layer_norm_rms_epsilon metadata required by llama.cpp, so a text-only conversion was downloaded and hashed. A GGUF extension alone does not prove compatibility: test actual loading and generation with the app's runtime.

Small models can struggle with long pages, low-resource language pairs, mixed code/tables/footnotes, and simultaneous glossary and context constraints. Preserve cloud choices so users can balance privacy, cost, speed, and measured quality.

For product-managed downloads, display size and license first, download to a temporary file, verify SHA-256, and move only verified files into Application Support. Include model ID, version, and hash in cache identity. The example's DuckDB key currently contains PDF ID, page, target language, source hash, provider, and model path; this distinguishes different paths but does not alone detect replacing a model at the same path. Production use should add the verified content hash. Old localStorage results are not read as translations.

The hard part remains judging quality. Prompts must distinguish decisions entrusted to a translator from technical elements that must remain unchanged.

10.2 The First Prompt —A Draft That Missed Everything

The first prompt was short. It was a translation task, so I thought the model could figure things out if I just gave it the input text and the target language. My first prompt was a single line.

“Translate the following English text into {language}. Output the translation only. No explanation.”

When I ran it across 36 languages, the output looked fine at first glance. The paragraphs flowed naturally, and most sentences were in the target language. But when I actually opened the files, the parts that matter most in a technical document were breaking again and again.

- Variable names inside code blocks were being translated (`const userName -> const 사용자이름`)
- Quotation marks were being swapped (`"foo" -> 「foo」`)
- Number formats were being localized (`1,000 -> 1.000`)

Why the AI Behaves This Way

There is one cause. **Without instructions, the AI follows general translation conventions.** Under those conventions, the right answer is to render every piece of text naturally. That is correct for literary translation. In technical documentation, it becomes an error. Filenames, code, commands, and API names are pieces of information the reader has to copy and run exactly as written.

This idea runs through the entire chapter. A prompt is not a tool for telling the AI “what to do.” It is a tool for telling the AI **where to stop following the conventions it would default to.** A technical translation prompt should not start with “translate naturally.” Define the structures that must be preserved first, then ask the AI to vary sentences only within that boundary.

A Real Failure —Five Errors in One Block

I ran the following English excerpt through the first prompt for Korean translation.

Original: ~markdown ## Chapter 3: Database Setup Set up the `database.config.js` file.

```
const config = { host: "localhost", port: 5432, database: "readbooks"
};
```

Connection timeout is 30,000ms. ~

What came back: ~markdown ## 3장: 데이터베이스 설정 데이터베이스. 설정. js 파일을 구성하세요:

```
const 설정 = { 호스트: "localhost", 포트: 5432, 데이터베이스: "readbooks" };
```

연결 시간 초과는 30.000ms로 설정됩니다. ~

Five problems showed up at once in a single small code block. The filename, variable name, and JSON keys were all translated; the number 30,000 was changed to the European-style 30.000; and as a result, the syntax highlighting of the code was also broken.

Caution: A simple “translate this” prompt **treats all text as something to translate**. Technical documents contain regions (code, URLs, commands) that must stay exactly as they are. If you do not state the constraints, the AI will localize everything.

First Fix —Pin Down the Preservation Boundary

The first fix was not an abstract demand to “make quality better.” It was to first pin down what must be preserved —code blocks, inline code, numbers, URLs —and then have the AI translate only the surrounding prose. The core is a five-line **system prompt** (an instruction that is attached to every request and that sets the AI’s role and rules).

You are translating technical documentation. Rules:

- Do not translate content inside ``` fenced code blocks. Copy it as-is.
- Do not translate `inline code` — keep it in the original.
- Preserve quotation marks, number formats, and dates as in the source.
- URLs, emails, DOIs: keep as in the original.
- Output the translation only. No explanation, no preamble.

When I ran it, code-block preservation worked. But a new problem showed up. **English sentences inside quotation marks were being left in the source language.** The AI had interpreted “preserve the quotation marks” as “preserve everything inside the quotation marks.”

Second Fix —Separate “Symbol”from “Content”

What I wanted was closer to the opposite. The quotation punctuation should stay, but the content inside the quotes should be translated like any other sentence. So I added these two lines.

- Punctuation symbols (" , ' , «» , „", etc.) are preserved as-is.
But the text inside the quotation marks is translated.
- Number formats (commas, decimal points) are preserved as in the source.

Let’s run the same sentence again.

Original: The system returns {"status": "success", "message": "File uploaded"} **Korean:** 시스템은 {"status": "success", "message": "파일이 업로드되었습니다"}를 반환합니다.

The JSON structure and keys were kept, and only the human-readable message was translated. Exactly the behavior I wanted.

The lesson is simple. **A constraint like “do not change X”must also state the precise scope of X.** If the scope is vague, the AI invents its own interpretation, and that interpretation usually follows general translation conventions.

10.3 Branching by Language —The Illusion of “One Prompt for 36 Languages”

Even getting the constraints right for a single language (Korean) took two rounds. So what happens when you have to handle 36 languages at once? At first I thought a single prompt could do it. In practice it could not. The same English sentence breaks differently in German, Russian, Arabic, and Chinese. Each language group needs its own constraints.

First, let’s list the kinds of issues each language tends to produce.

- Korean / Japanese: English proper nouns stay in English by convention (e.g., “React”stays “React”).
- German: The LLM cannot consistently apply the rule that nouns start with a capital letter.

- Russian: A language without articles, but the LLM inserts demonstratives (“this,” “that”) that were not in the original.
- Arabic / Hebrew: Right-to-left languages (RTL), which affects how the Markdown renders.
- Chinese: The same English word must be translated differently in Simplified vs. Traditional Chinese.

For example, when the English “The react component handles state management through hooks.” is translated into German, the AI sometimes outputs `react`, `komponente`, and `hooks` all in lowercase. It followed English capitalization rules instead of German orthography. For Russian, the AI may add a demonstrative (“этот”) that was not in the original, creating a new meaning of “this specific hook.” It tries to sound more natural and ends up changing the meaning.

Caution: Thinking in terms of language groups breaks the rules at the level of individual languages. Even closely related languages like Portuguese and Spanish differ in their preferred technical terminology.

Solution —A Base Prompt Plus Per-Language Profiles

The solution was to attach a **per-language addition block** to one system prompt. Shared rules live in the base prompt; the conventions that differ per language are isolated as short profiles.

```
const base = `You are translating technical documentation. Rules: ...`;

const languageProfiles: Record<string, string> = {
  ko: `– Keep English proper nouns (React, npm, API) in English as-is`,
  de: `– German noun capitalization required\n– Keep compound nouns as one word`,
  ar: `– Apply RTL text direction. Keep code blocks LTR.`,
};
```

I kept each profile under ten lines. When the rules get longer, the model starts dropping important constraints or trying to follow contradictory instructions at once.

What a Real Profile Looks Like

A one-line example is hard to picture. Here is the full German profile I actually used.

German (de) profile:

- Nouns always start with a capital letter. English loanwords used as nouns also get capitalized (Hook -> Hook, component -> Komponente).
- English proper nouns (React, npm, API) stay as-is, but capitalization applies when used as a noun.
- Compound nouns are written as one word (Datenbankverbindung, no hyphen).
- Use formal address (Sie/Ihnen). The du form is forbidden.
- Translate "you" as Sie or use the passive voice depending on context.

Use one template for each language: style, mistranslations to avoid, number and unit formatting, proper names, and output format. Add one incorrect and one correct sentence. The five-line example demonstrates this structure. Apply only rules checked by a reviewer of that language; it does not establish accuracy across all 36 languages.

Building Language Profiles: Using AI as a Research Assistant

Instead of researching 36 languages by myself, I used Claude as a domain expert. The key was not handing the whole task over with “build the profiles for me.” First I decided the criteria I needed (typography, technical terminology conventions, formatting differences). Then I asked, language by language, where the model tends to fail, and from the responses I picked only the items worth putting into the actual prompt.

That approach let me put together profiles for 36 languages in two hours. But I did not trust the output blindly. I ran a short sample in each language and confirmed that code blocks, numbers, and terminology were preserved. **AI is useful not only for writing code but also for researching domain knowledge. The final criteria and the verification, though, still have to come from the person making the product.**

Caution: Complex language profiles cause rules to collide. If you say “use formal address” and “translate naturally” together, the AI will pick formality over flow. Give the rules a clear priority.

10.4 Designing Prompts With AI —The Meta Loop

The most important idea in this chapter is here. AI is not only the translator —**you use AI to design the prompt itself**. You don't lock in a prompt once and stop. You collect failure cases and keep refining it.

What a Meta Prompt Is

Let me define the term first. A **meta prompt** is “a prompt for creating or fixing another prompt.” A regular prompt tells the AI to do work (“translate this sentence”). A meta prompt tells the AI to do work on the prompt itself (“find the weaknesses in this translation prompt”). It is the same model, but a different input changes the character of the output completely.

There are two levels of use.

1. **Level 1:** Write a prompt that produces good translations.
2. **Level 2:** Write a prompt (a meta prompt) that produces a better translation prompt.

A Level 2 prompt looks like this.

“You are a prompt engineering expert. You need to improve this translation prompt for technical documentation. Current version: [text of current prompt] Problems I'm seeing: [specific failure examples] Your task: suggest three specific improvements to the prompt. Under 200 words total.”

Rather than testing 36 languages one by one, iterating on the prompt itself with this meta-prompt pattern is much faster.

Reusing the Same AI as a Reviewer

You can also switch roles for the same model.

“You will be the translation AI that receives the following prompt. Read it and find **three places where the instructions are ambiguous**. Do not fix them.”

The three ambiguities Claude found all turned out to affect real translation output. For example, it was unclear whether “preserve number formats” also covered dates, and whether “inline

code” meant one backtick or two. Refining the prompt to address them improved the quality. **It is the same AI model, but with a different role, it brings a different perspective.**

Dual Agents —Separating Translator from Reviewer

I’m using the word **agent** here in a narrow sense for this book: “an AI instance assigned a specific role that does only one job.” Even the same model (such as Claude) becomes a different agent depending on the prompt that calls it.

The pattern is simple. **Let one agent’s output be reviewed by another agent.**

- **Agent 1 (translator):** Generates output following the translation prompt.
- **Agent 2 (reviewer):** Receives the same output and checks code preservation, terminology consistency, formatting preservation, and meaning preservation. If there are issues, it outputs them as a list; if it’s clean, it returns ‘No issues’.

For instance, if a German translation renders the filename `app.config.js` as `anwendungs.konfig.js`, the reviewer catches it: “filename was translated —code preservation rule violated.” Errors that a stand-alone translation would let slip through get caught. This pattern was especially useful for the 32 languages I cannot read.

Meta Prompts That Failed

Not every meta-prompt approach worked. Vague requests get vague answers.

- “Make it better”—> You get generalities like “be more specific.” You have to **include concrete failure cases**.
- “Score this prompt”—> You get a number like “8/10,” but it does not match real performance. Ask the model to **identify failures in test cases** rather than give a score.

Caution: Meta prompts need concrete criteria just like regular prompts.

Version Control for Prompts

As you revise a prompt many times, you start needing a change history. I managed it like a git repository.

```
prompts/
v1.0-base-naive.md          # "translate this text"
v1.1-add-code-preservation.md
v1.2-fix-quote-handling.md
v2.0-language-profiles.md
v2.1-ambiguity-fixes.md
current.md -> v2.1-ambiguity-fixes.md
```

For each version I noted what changed and why. That let me **compare prompt versions** and, when an “improvement” was actually a regression, roll back immediately. In one real case, Claude suggested adding “be creative in the translation” to improve fluency; when I applied it, technical terms became inconsistent, and I rolled it back right away.

10.5 Translation Quality Standards —How to Communicate Them to the AI

Declarations like “translation quality must be high” do not work. The AI needs concrete instructions. **You have to break the goal into items that can be checked.**

These are the check items pulled from the final prompt.

Quality standards (do not output — for internal checking):

1. Meaning preservation: no sentences dropped or altered.
2. Terminology consistency: same source term -> same target term throughout the document.
3. Tone match: technical documents stay formal, not conversational.
4. No AI meta-talk: phrases like "Here is the translation" are forbidden.
5. Same Markdown structure: same heading levels, lists, code fences.

When you group these five items as an “internal check,” the output changes. I cannot know what procedure the model actually runs inside, but putting a checklist into the prompt tends to make the results more consistent.

Where Each Standard Came From

All five standards came from real failure cases. Here are two of them.

- **Meaning preservation:** “The function returns null if no user is found.” was translated as “If there is no user, the function searches for null.” “Return” had become “search for.” “Translate accurately” alone is not enough—you have to **explicitly require meaning preservation**.
- **Tone match:** The English “Execute the following command” once came out in French in the overly casual *tu* form (“Va exécuter...”). If you do not specify the formal *vous*, the AI defaults to conversational tone.

Terminology consistency, blocking AI meta-talk, and Markdown structure preservation became rules in the same way—built up from similar cases.

Domain-Specific Additions

Some edge cases were not caught by the five basics, so I added a short appendix. API end-point paths, error codes, configuration keys, command-line flags, file extensions—these identifiers must never be translated. One Russian translation, for instance, tried to localize `.config.json` to `.конфиг.json`. That is a logical translation in Russian, but it breaks every code example.

Caution: Quality standards can conflict with each other. The classic example is “meaning preservation” vs. “natural phrasing.” Decide the priorities in advance. **Technical accuracy comes first, naturalness second.**

Verification Pipeline —Automatic Checks Plus a 10% Human Review

Quality standards on their own are not enough. You also need a stage that confirms the output actually meets them. I set up these four stages in order.

Stage 1 — AI translation

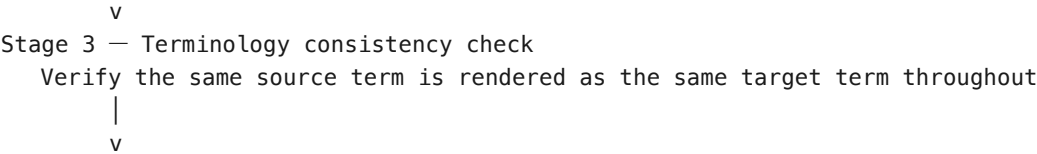
Generate translation with a system prompt that includes the 5 quality standards

|
v

Stage 2 — Automatic format check

Compare heading count, code-block count, link count between source and translation

|



Stage 4 — Human review (10% of all output)
 Cultural nuance, non-standard technical terms, and other items automation cannot catch

Stage 4 was especially important. AI-generated quality standards catch about 80% of the problems, but cultural nuance and non-standard technical terms in a given language only get caught by a human. This four-stage flow leads directly into the verification pipeline in Chapter 11.

10.6 Wrap-Up of This Chapter's Loops

Here is the iteration history summarized in one table.

Round	Content	Result
1	Initial constraints (code blocks, URLs)	Second pass needed (quotes)
2	“Symbol vs. content”split for quotes	Done
3	Language profile design	I led, AI assisted
4	Claude as a language expert	36 language profiles generated
5	Reusing Claude as a prompt reviewer	Found 3 ambiguities
6	Self-rewrite experiment	Low return for the cost; dropped
7	Import .md as a string with Vite ? raw	Solved in one shot

Sidenote—Round 6 cost analysis: I also tried adding “rewrite until all 5 items are satisfied” to the prompt. Token usage nearly doubled, but the quality improvement was very small. The internal-checklist approach delivers most of the quality gain at half the tokens.

Sidenote—Vite string imports: `import prompt from './prompt.md?raw';` loads file contents as a string. Secrets placed there may enter the bundle, so keep them out. If TypeScript rejects the syntax, check that Vite type declarations are included. Importing Markdown text and rendering safe HTML are separate concerns.

Three Patterns That Kept Repeating

Three patterns came up in every loop.

1. **Constraint specificity:** Vague instructions like “translate well” gave uneven quality. Specific constraints like “preserve code exactly” produced much more stable results.
2. **Switching the AI’s role:** Using the same AI as translator, reviewer, and language expert gave different perspectives each time.
3. **Iterative improvement:** No prompt was ever perfect at the start. Quality only goes up through systematic testing and fixing.

These three are not specific to translation prompts. They apply to any AI task where you need precise output.

Core message: AI is not only a tool that writes code for you. You can also use it as **a reviewer, an expert, and an experimentation partner**. Switching roles as the situation calls for is the heart of this chapter.

10.7 Next Chapter

Even with a solid prompt, one problem remains. **Can you actually verify that the output is correct?** Out of 36 languages, I can read only three or four. The other 32 cannot be checked by eye. The next chapter is about that problem. It covers **a pipeline that back-translates the output with another AI and automatically scores the result**.

Summary

This chapter treats **prompt engineering as a collaboration between humans and AI**. The single key idea: a prompt is not a one-shot sentence you write and throw away. It is **a quality system you maintain continuously**.

Things to take from this chapter:

- **Constraint specificity:** AI takes words literally. “Preserve quotation marks” and “preserve the quotation symbols but translate the content” produce completely different results.
- **Per-language profiles:** One prompt cannot handle 36 languages. Each language needs its own rules.
- **Meta prompting:** Use AI to produce better prompts for AI. The same model can take on the roles of translator, reviewer, and language expert.
- **Quality standards as code:** Instead of a vague goal like “high quality,” break it down into five verifiable conditions. AI follows a clear checklist far better than a subjective target.

When you implement this in code, two things are needed: **a provider abstraction** (bundling several translation services behind one interface), and **prompt management** (version control, modular assembly, automated testing).

Practically, remember these five points.

1. **Start simple, then add constraints:** Begin with a basic translation, and add concrete rules one at a time as problems appear.
2. **Use AI as several experts:** The same model gives you a different perspective depending on the role (translator/reviewer/expert).
3. **Test per target language:** If you cannot read the output language yourself, build an automated verification system.
4. **Version-control your prompts:** Track changes like code and keep the ability to roll back.
5. **Measure and monitor:** Track prompt performance in production and feed it back into future improvements.

Next chapter preview: How do you verify translation quality when you cannot read the output language? We will look at how a back-translation pipeline automatically scores translation quality.

Chapter 11. Translation Verification —How Do You Trust AI Output?

readbooks.ai supports 36 languages. Most developers cannot read 32 of them. When you cannot visually check whether a translation reads naturally, whether the terminology is right, or whether a paragraph has gone missing, quality control has to be designed in a different way.

The prompt is well written and the output looks plausible. But there is no way to tell where it went wrong. This problem is not limited to translation. The same situation appears in any field where AI produces specialized content faster than a human can keep up.

The solution is a **verification pipeline that uses a second AI**. First, you list the failure types that actually happen often. Then you let an AI automatically check whether each failure has occurred.

It is not a perfect method. It can only catch the failure types you anticipated, and unexpected errors still need human eyes. That is why verification is separated into its own stage instead of being folded into generation —the same reason a code reviewer is separated from the author. When the same person writes and reviews, they tend to miss the same things twice. Instead of trusting the output of the generating AI directly, you ship it only after it passes an independent verification process.

The flow of this chapter

In broad strokes, there are four steps.

1. **Define the problem** —the four shapes of “plausible but wrong” translations.
2. **Try back-translation once** —the simplest form of verification.
3. **Expand to three axes** —split into meaning, length, and terminology.
4. **Gather user feedback again** —feed what automatic verification missed back into the prompt.

Operational details (calibrating synthetic test cases, regression tests per language family, canary deployments) are sketched briefly at the end of the chapter. On the first read, focus on following the four steps above.

Learning goals

- Understand why verification is hard when you cannot read the AI translation yourself, and how to approach it.
- Learn how to verify with back-translation (translating the result back into the original language and comparing) and how to set scoring criteria.
- Get the big picture of automatically testing 36 languages.
- See the pattern of feeding user feedback back into prompt improvements.

Minimum deliverable for this chapter: one `verify_translation` Tauri command and one screen that shows the three-axis score (meaning, length, terminology).

11.1 The Problem —“Plausible but Wrong”Translations

The trickiest mistake an LLM makes in translation is when the sentence reads naturally but the meaning is wrong. On the surface it looks fine, so users rarely notice.

It is especially dangerous in technical documentation. Even if the prose flows well, when a code explanation or API name is wrong, the reader will follow the wrong procedure.

There are four common failure types.

- **Omission:** an entire sentence drops out of a paragraph. For example, the second of two sentences explaining a database connection pool disappears completely.
- **Hallucination:** an example that was not in the source gets added. The English source mentions only the `fetch` API, but `XMLHttpRequest` slips into the Spanish translation.
- **Terminology drift:** the same word is translated as “component” earlier in the document and as “constituent” later. The reader cannot be sure whether the two words refer to the same concept.
- **Tone collapse:** a casual conversational tone suddenly appears in the middle of a technical document. Friendly informal speech pops up in the middle of a guide written in a formal register.

Of these, **omission and hallucination** are hard for users to spot without seeing the source.

Concern: errors like these do not surface immediately the way compile errors do. They are delivered as-is to thousands of readers until someone notices.

11.2 Back-translation —Trying It by Hand Once

The first strategy is a classic one. **Translate the result back into the original language and compare it with the source.** Even if you cannot read the target language, the back-translation comes back in English, so you can compare directly.

Let us walk through it by hand first, without any code.

1. Write one paragraph of English source. Example: “Use the fetch API to make HTTP requests.”
2. Translate it into Spanish (let the generating AI do this). Result: “Use the fetch API y el método XMLHttpRequest para hacer solicitudes HTTP.”
3. Translate that Spanish back into English (let the verifying AI do this). Result: “Use the fetch API and XMLHttpRequest method to make HTTP requests.”
4. A human compares step 1 with step 3 by eye. It is immediately clear where XMLHttpRequest came from.

The key point is that **even if you do not know what language step 2 went to, step 4 compares English with English**. That is why verification is possible without being able to read all 36 languages.

It is not a complete method, however. Errors can be hidden during the back-translation step. So when you automate this approach, you also need to set scoring criteria.

11.3 Building the Verification Command —The Simplest Skeleton

Now we move the flow above into a Tauri command. At first, instead of a finished quality evaluator, I wanted the smallest possible skeleton. Pass in the source and the translation, translate it back to English, and return a single score.

prompt:

“Translation verification command. Input: source (English), translation, target language. Internally back-translate to English. Similarity score against source. Reuse the `translate_with_provider` command. Skeleton only, under 10 lines.”

The first skeleton Claude returned creates a Tauri command called `verify_translation` and reuses `translate_with_provider` from Chapter 10 to translate the result back into English. It then returns one score that compares the back-translation to the source.

The `translate_with_provider` function from Chapter 10 was a general-purpose function whose arguments said “with which provider (OpenAI/Anthropic/...) and which model, translate from which language to which language.” In Chapter 11, we **call this function twice**: once from

source to target language, and once from target language back to English. We use the same function twice, only flipping the direction.

11.4 Limits of a Single Score —BLEU and Cosine Similarity

Once the skeleton ran, the scoring part became the next problem. What number should express “similar”?

I first tried BLEU (a classic metric that measures translation quality by overlap of word combinations), but **it did not fit back-translation verification**. BLEU counts how much the word combinations overlap, so even when the meaning is identical, a different choice of words drives the score down.¹

So I switched direction and decided to use **an API that returns a semantic similarity score**. Feeding the source and the back-translation into an embedding API gives a number between 0 and 1 expressing how close the two sentences are in meaning. The closer to 1, the closer in meaning.

It would be nice if that were the end of it, but this method has its own limits. Semantic similarity is **insensitive to length differences**. Even if an entire paragraph drops out of the translation, the remaining content can be close enough to the source that the score still comes out above 0.95.

To test this, I made a German translation in which the “mounting” explanation was missing from an English paragraph about React component lifecycle. Even with the section missing, the back-translation similarity came out at 0.95. This is where it becomes clear that **a single score is not enough**.

¹BLEU was originally designed to compare how closely different translation models reproduce the same reference sentence, so it is unsuited to back-translation, where “the meaning is the same but the words differ.”

11.5 Splitting into Three Axes

Translation quality is hard to express with a single number. The meaning can be close while a paragraph is missing, or the length can match while a key term is wrong. So I split the verification criteria into **three axes**.

- **Semantic similarity:** a 0-1 score returned by an embedding API.
- **Length ratio:** translated word count ÷ source word count. Normal range is usually 0.85 to 1.2.
- **Term consistency:** how well the frequency of key terms (proper nouns, API names) matches.

Each metric is implemented as a separate function, and any metric that falls outside its threshold raises a flag (an error marker) on that translation.

Aligning the Result Type With the Frontend

For the frontend to receive the verification result and show it on screen, the shape of the result must be defined identically on both sides.

- On the TypeScript side, create a `VerifyReport` type. It holds the semantic score, the length ratio, term consistency, and an array of flags.
- On the Rust side, create a struct with the same shape. Then attach `serde::Serialize`.

The reason for `serde::Serialize` is simple. A Rust struct cannot be handed directly to JavaScript. Tauri first turns the struct into JSON, then sends it to the frontend. The flow comes down to three lines.

1. Rust constructs a `VerifyReport` struct.
2. `serde` serializes it into a JSON string.
3. The return value of `invoke('verify_translation', ...)` arrives in JavaScript as a `VerifyReport` object.

What to check in the output: the most important signal is which keywords ended up in the `flags` array on the object the frontend receives. Empty means GOOD, one or two flags means REVIEW, several flags means FAIL.

The three metrics caught 80% of omission errors. But hallucination errors slipped through. I had left a place in the `flags` array for them, but the detection logic was not there yet. **A verification pipeline is never finished from the start.** It grows stronger one metric at a time, as you meet new failure cases.

Concern: the length ratio can produce false positives depending on the language. German compound words reduce word count, while Indonesian adds particles. The acceptable range has to be tuned per language.

Adding Hallucination Detection

Hallucination is an error where content that was not in the source slips into the translation. Semantic similarity and length ratio alone struggle to catch it.

The detection method is straightforward. Ask the LLM to extract proper nouns, product names, and API identifiers from the source and the translation, one per line, for each side. Then compare the two lists and treat any **word that appears only in the translation, not in the source** as a hallucination candidate. Not perfect, but practical enough.

The `XMLHttpRequest` example from 11.1 is caught exactly this way.

Summarizing with a Three-Level Badge

Setting the thresholds also took trial and error. The first version was so strict that every page lit up with a “this translation looks off” badge. After loosening it, the badge almost never appeared. The final form was a **three-level badge**.

- **GOOD:** all metrics inside their ranges.
- **REVIEW:** one metric in the warning range.
- **FAIL:** two or more metrics in the warning range.

After splitting it into three levels, complaints about false positives dropped. **Automatic verification cannot catch every error.** A realistic goal is to separate the translations that need a human eye from the ones that do not.

11.6 Refining It in Operation

That covers what you really need from the first read. This section briefly mentions the additional steps you take when you put the system into a real service. Not in depth —just enough to know “there is more after this.”

- **Calibrate thresholds with synthetic tests:** have the AI generate scenarios like “a case where only one axis is broken,” then use the results to fine-tune the GOOD / REVIEW / FAIL boundaries.
- **Per-language test matrix:** group 36 languages by family (Romance, Germanic, agglutinative, logographic) and auto-generate the error types each family is prone to. Compound words come up for Germanic, length ratio for agglutinative —characteristics like these surface.
- **Per-language regression tests:** cross-interference is common —fixing a German prompt can break French. Keeping each language’s baseline in its own JSON file and re-verifying only the affected languages on a change prevents that.

A verification table needs language, input type, expected condition, actual result, judgment, and reviewer/tool version. Assess preservation of “12.5%”, three paragraphs, translation-only output, and consistent proper names separately. Separate synthetic pass rates from human review of real documents. Do not report accuracy for untested languages without supporting measurement.

11.7 User Feedback Loop

No matter how refined automatic verification gets, it cannot fully replace the problems real users point out. I added a feedback button in the Tauri 2 rebuild.

The feature

- A small “this translation looks off” button at the end of each paragraph.
- Clicking sends the paragraph content, the source, the target language, and the provider info to a log database.

- The log is reviewed every two weeks.

Updating Prompts From Real Data

Analyzing three weeks of logs surfaced these problems:

- German “compound noun splitting”: 12 cases
- French “gender agreement errors”: 8 cases
- Spanish “vos/tu mixing”: 5 cases

I handed this data to Claude and updated the per-language profiles. For German, for instance, I added a rule like “keep German compound nouns as a single word (Datenbankverbindung, not Datenbank-Verbindung). Do not split them unless the source clearly uses a hyphen.”

What to check in the output: the verification signal is whether the same problem is reported again over the next two weeks. With this one line, the same 12 cases did not reappear. **Feed-back -> prompt update** is one of the clearest cost-effective improvements you can make.

Sorting Feedback Into Three Simple Tiers

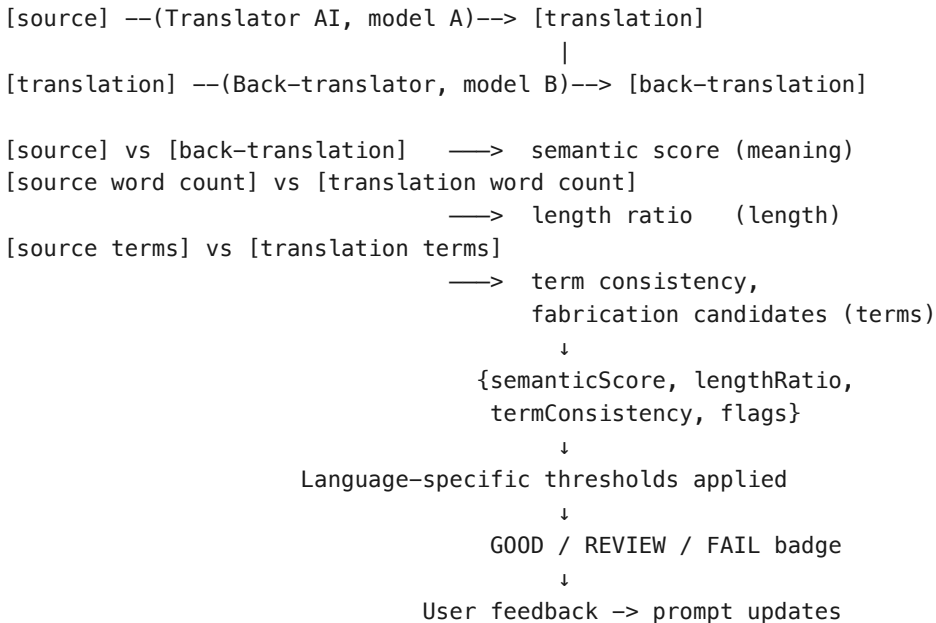
Not all user feedback has the same priority. Group it by impact into three tiers.

- **Fix immediately:** meaning reversals (translating enable as disable), security-related mistranslations.
- **Fix in a weekly batch:** terminology inconsistency, culturally inappropriate tone.
- **Review monthly:** technically correct but awkward phrasing, simple preference differences.

The categorization itself can be handed to Claude. Without categorization, minor style preferences pile up, and serious errors are discovered too late.

11.8 Visualizing “Verifying AI Output With Another AI”

Here is the core pattern of this chapter in a single diagram. It flows from top to bottom.



The key point is that **using different providers for model A and model B makes verification more reliable**. When generation and verification use the same model, they share the same biases. Translate with Claude and verify with GPT, or the other way around.

11.9 Tips for Applying This to Your Reader App

This pattern is not limited to translation. It applies anywhere there is AI output.

Tip 1. No single score. Split into axes.

Expressing quality as a single number hides where the problem is. Splitting the criteria by item —meaning, length, formatting, consistency —makes it immediately clear which part is bad.

Tip 2. Use different models for generation and verification.

The same model makes the same kinds of mistakes. Set it up so Claude’s output is verified by GPT, and OpenAI’s output is verified by Gemini. Different models reduce the chance of missing the same error together.

Tip 3. Use three threshold levels.

Splitting into only pass and fail forces the bar to be unnecessarily strict. Three levels —good, needs review, fail —let you tune the amount of human review to a realistic level.

Tip 4. Feed user feedback into the prompt.

What automatic verification misses, users catch. Every two weeks, gather the feedback logs and update the per-language prompt profiles. It is a low-cost way to keep raising output quality steadily.

11.10 Chapter Summary

In plain language, the four points are:

1. **Quality cannot be caught with a single score.** Look at it on at least three axes: meaning, length, terminology.
2. **The AI that produces and the AI that checks must be different.** The same model makes the same mistakes.
3. **Pass/fail in two tiers is too coarse.** GOOD / REVIEW / FAIL in three tiers is what people actually find usable.
4. **Automatic verification is not the end.** Quality stays high only if you collect user feedback and regularly revise your prompts.

The operating principles for this chapter:

Pattern: verify the bias of one AI’s output with a different AI model.

Architecture: for complex output, multi-axis scoring beats a single quality metric.

Testing: use AI to generate test cases that cover the whole range, and always run regression tests before changing thresholds.

Operations: prevent cross-language interference with per-language baselines, and keep calibrating with real user feedback.

11.11 Looking Back at Part II and Part III

There is something the two apps from Part II (Mandai) and Part III (readbooks.ai) showed in common. **Even when AI speeds up implementation, if you do not set the direction yourself, you go in the wrong direction quickly.** The request-implement-verify loop is the device that controls both speed and direction.

- In Part II, the loop was mostly about **whether the code was right or wrong**. Does it compile? Does it work? Are there any edge cases?
- In Part III, the loop was about **the quality of the output**. There is no single correct answer for a translation. You have to design the evaluation criteria yourself.

11.12 Next Chapter Preview

The next chapter moves beyond translation quality verification to a dashboard-scale project. The focus of verification shifts as well, from linguistic accuracy to data accuracy and real-time rendering.